# Machine Learning

## 01 - Regression

**Linear Regression**: regression that assumes a **linear relationship between inputs and the output**, so $y_n \approx f(x_n) = w_0 + w_1 x_{n1} + \cdots + w_D x_{nD} = w_0 + w x_n^T = \tilde{x}_n^T \tilde{w}$ where $\tilde{x}_n$ starts with a 1 (to have an **intercept or bias term**)

**Loss/cost function**: quantifies **how well a model does**, **used to learn the optimal parameters** that explain the data well. **Positive and negative errors should be penalized the same** so loss function should be **symmetric around 0** and **penalize "large" and "very-large" mistakes similarly** (so that outliers cannot mess up our model too much as it's not always easy to just remove them)

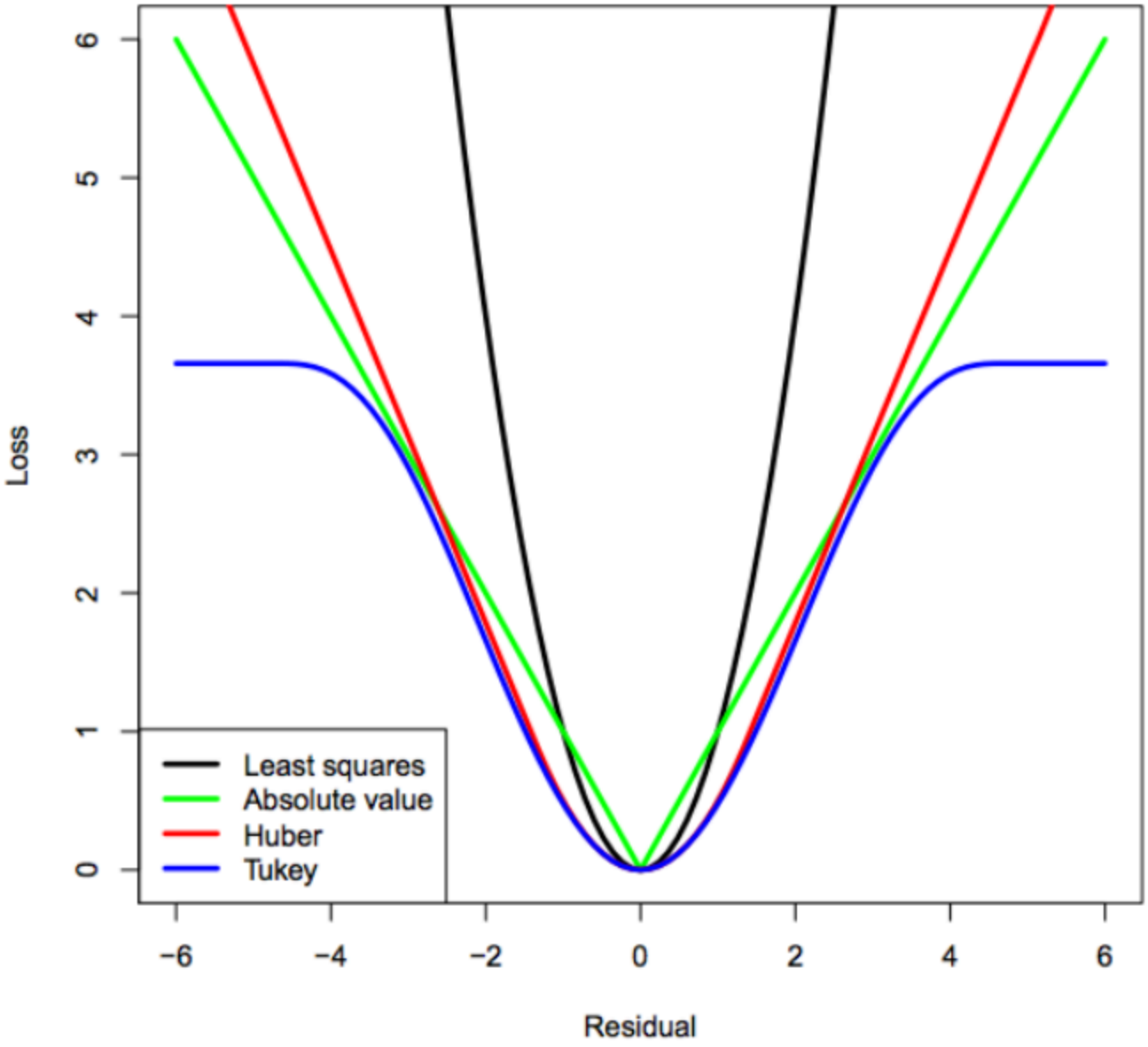
**Mean Square Error (MSE)**: loss function that's **easy to compute as it's differentiable**, $MSE(w) = \frac{1}{N} \sum_{n=1}^{N} [y_n - f_w(x_n)]^2$, **minimized by the mean** (we want to be closer to where most of the points are)

**Mean Absolute Error (MAE)**: **not differentiable at 0** but with **better statistical properties** as it **handles outliers better** than MSE (does not amplify them), $MAE(w) = \frac{1}{N} \sum_{n=1}^{N} |y_n - f_w(x_n)|$, **minimized by the median** (we want to be closer to most points)

**Convex functions**: $f(x)$ is convex if $f(\lambda u + (1-\lambda)v) \leq \lambda f(u) + (1-\lambda) f(v)\ \forall u, v \in \mathbb{R}^D, \forall \lambda\ s.t.\ 0 \leq \lambda \leq 1$, $f$ is below the line that connects any two points of $f$, **unique global minimum** $w^*$, **MSE and MAE with linear models** are convex. **All linear/affine functions and sums of convex functions are convex**, **composition of linear/affine functions and convex functions are convex**

# Computational VS statistical trade-off

So which loss function is the best?

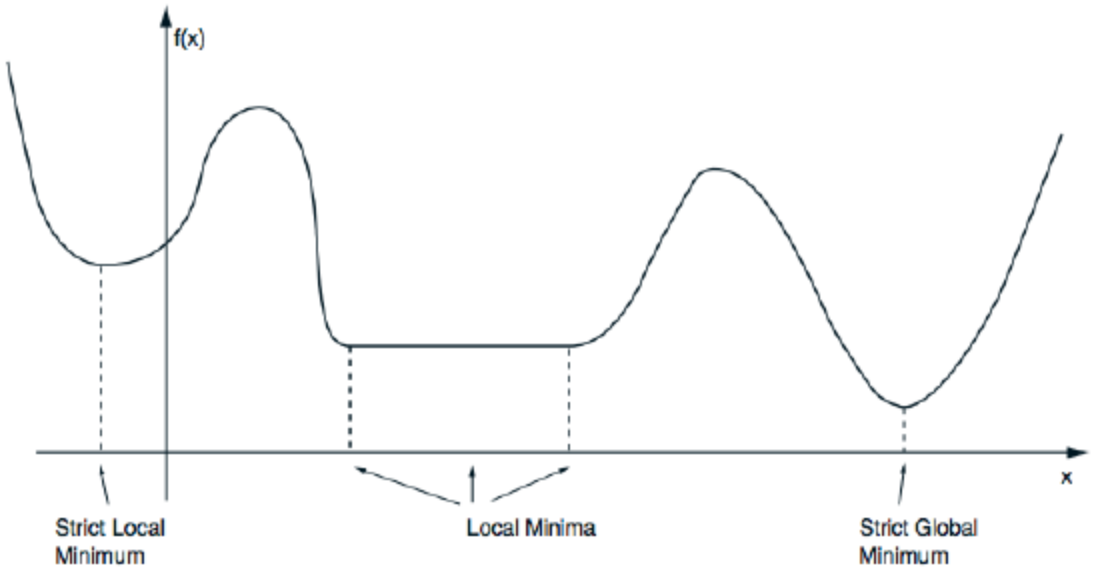



## 02 - Optimization

**Optimization**: finding $w_* = argmin\ \mathcal{L}(w),\ w \in \mathbb{R}^D$

**Local minimum**: $\mathcal{L}(\tilde{w}) \leq \mathcal{L}(w),\ \forall w \in [\tilde{w} - \epsilon, \tilde{w} + \epsilon]$, strict if the inequality is strict

**Global minimum**: $\mathcal{L}(w_*) \leq \mathcal{L}(w),\ \forall w \in \mathbb{R}^D$, strict if the inequality is strict



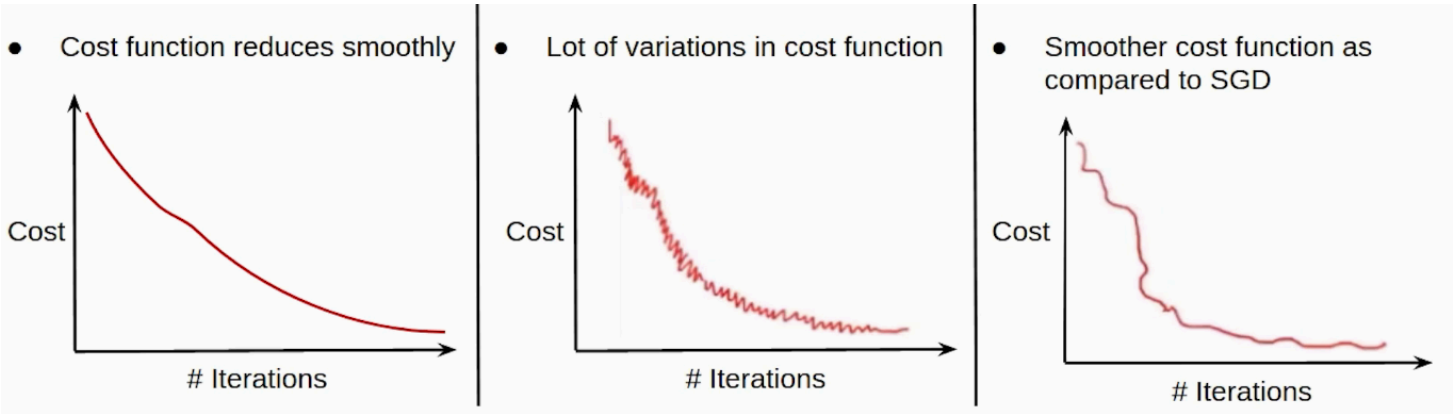

**Gradient**: **slope of the tangent** (line, plane, hyperplane) to the function at a specific point. It's the **vector that points to the direction of largest increase** of the function, $\nabla \mathcal{L}(w) = [\frac{\partial \mathcal{L}(w)}{\partial w_1}, \dots, \frac{\partial \mathcal{L}(w)}{\partial w_D}]^T$, each partial derivative estimates how much I would go up the loss function I were to go along that specific dimension

**Gradient Descent (GD)**: **minimize the function** by **iteratively taking a step in the opposite direction of the gradient**, $w^{(t+1)} = w^{(t)} - \gamma \nabla \mathcal{L}(w^{(t)})$ where $\gamma > 0$ is the **step-size** (or **learning rate,** generally **below 2 to "guarantee" convergence**)

**Gradient Descent for Linear MSE**: $y = [y_1, \dots, y_n]^T, X \in \mathbb{R}^{NxD}$ the **error vector** $e = y - Xw$ (computational cost $O(N * D)$), **MSE becomes** $\mathcal{L}(w) = \frac{1}{2N} \sum_{n=1}^{N} (y_n - x_n^T w)^2 = \frac{1}{2N} e^t e$ ($N$ became $2N$ to make gradient prettier, computational cost $O(N * D)$) and the **gradient** therefore is $\nabla \mathcal{L}(w) = -\frac{1}{N} X^T e$

**Stochastic Gradient Descent (SGD)**: applying the **loss function on only a random data point** $n$ **each time to save on computation** (**same possibility of getting to the optimum weights because picking a random point is an unbiased** estimate, $\mathbb{E}[\nabla \mathcal{L}_n(w)] = \nabla \mathcal{L}(w)$)

**Mini-batch Stochastic Gradient Descent**: pick a **random subset** $B$ **of points each time**, $g = \frac{1}{|B|} \sum_{n \in B} \nabla \mathcal{L}_n(w^{(t)}), \;\; w^{(t+1)} = w^{(t)} - \gamma g$, **size of** $B$ **can be customized** per computer/system, also the **computation of all** $\mathcal{L}_n$ **in** $g$ **can be parallelized**




**Stochastic Gradient Descent with Momentum**: compute **momentum** (a **moving average of the gradient**), $m^{(t+1)} = \beta_1 m^{(t)} + (1-\beta_1)g, \;\; w^{(t+1)} = w^{(t)} - \gamma m^{(t+1)}$, **momentum softens discrepancies between the single data points** picked for SGD, helps in **not getting stuck in local minimums**

**Adam**: **SGD with Momentum plus the 2nd-order statistic** $v_i^{(t+1)} = \beta_2 v_i^{(t)} + (1-\beta_2)g_i^2 \;\; \forall i$, then $w_i^{(t+1)} = w_i^{(t)} - \frac{\gamma}{\sqrt{v_i^{(t+1)}}} m_i^{(t+1)}$, **softens big variations in the gradient's coordinates** (it **normalizes** it, **scale-invariant to gradient magnitude**), **dynamic adjustments** (slow around cliffs, quick in valleys), forgets older weights faster, **coordinate-wise adjusted learning rate**

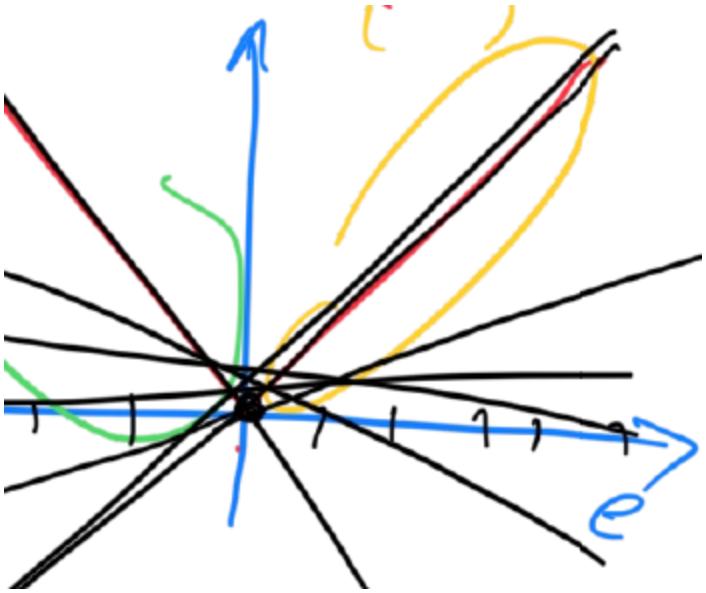
**SignSGD**: has **convergence issues**, **uses only one bit** (the sign) for each gradient coordinate (**great for distributed training**), pick a stochastic gradient $g$, $w_i^{(t+1)} = w_i^{(t)} - \gamma sign(g_i) \;\; \forall i$

**Convexity of Differentiable functions**: a **function to be convex must lie above its linearization**: $\mathcal{L}(u) \geq \mathcal{L}(w) + \nabla \mathcal{L}(w)^T (u-w) \;\; \forall u, w$

**Subgradient**: vector $g \in \mathbb{R}^D \; s.t. \; \mathcal{L}(u) \geq \mathcal{L}(w) + g^T(u-w) \;\; \forall u, w$ is a subgradient to $\mathcal{L}$ at $w$, multiple subgradients if $\mathcal{L}$ is not differentiable at $w$

**Subgradient descent**: $w^{(t+1)} = w^{(t)} - \gamma g$ with $g$ being a subgradient to $\mathcal{L}$ at $w^{(t)}$, **MAE can be used**

**Stochastic Subgradient Descent**: $w^{(t+1)} = w^{(t)} - \gamma g$ with $g$ being a subgradient to $\mathcal{L}_n$ at $w^{(t)}$



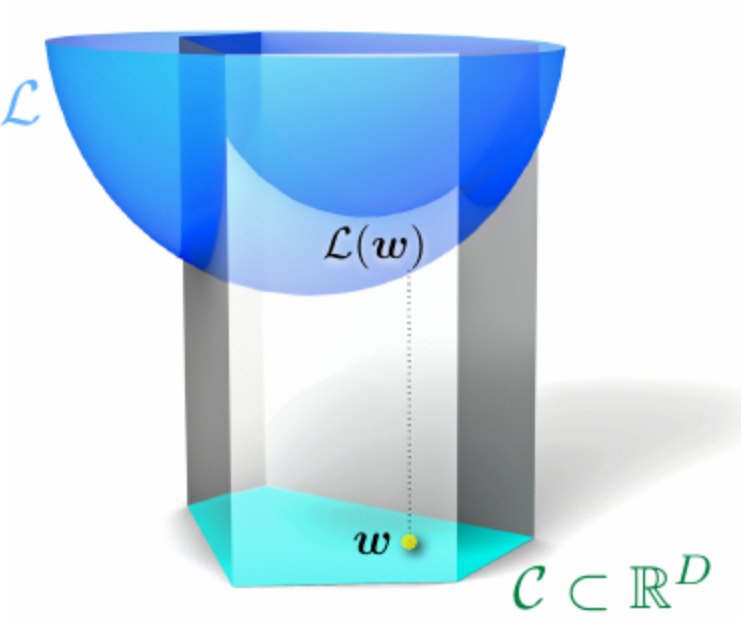
**Feature normalization**: **GD** is very **sensitive to features having change rates very different** each other (**ill-conditioning**), so **input is normalized** by subtracting the mean and dividing by the standard deviation (**pre-conditioning**), allowing all “directions” to converge at similar speeds

**Stopping criteria**: when $\nabla\mathcal{L}(w)$ **is close to zero** we are often **close to the optimum** (or if the **loss stops changing** significantly)

**Critical points**: points where the **gradient is zero**, **if** $\mathcal{L}$ **is convex** then a **critical point is also a global optimum**

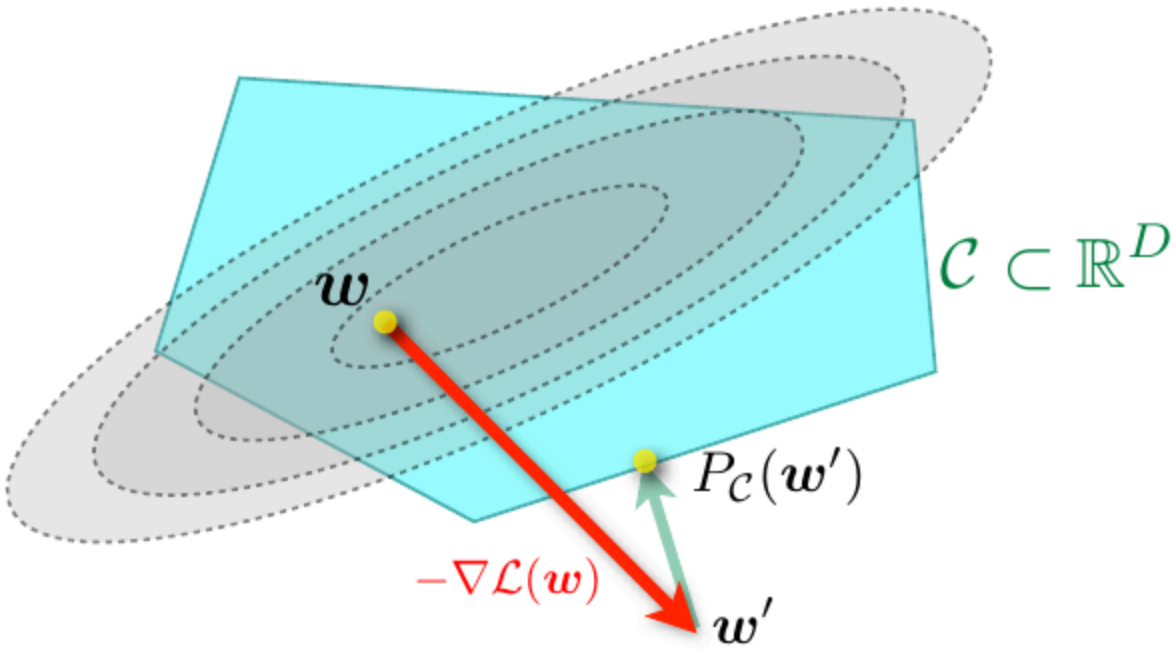
**Hessian matrix**: $\nabla^2\mathcal{L}(w) = \frac{\partial^2\mathcal{L}}{\partial w \partial w^t}(w)$, matrix containing the **partial derivatives in each combination of directions**, if the **second derivative in a critical point is positive** then it means that function increases in all directions you go, so you are at a **local minimum** (formally $\nabla\mathcal{L}(w) = 0$ and $\nabla^2\mathcal{L}(w) \succ 0$, positive definite). All **convex functions have positive semi-definite Hessian matrices** in all points

**Constrained Optimization**: solution costrained to a **constrain set** $C$, $argmin\ \mathcal{L}(w), w \in C$




**Convex Sets**: **line segment between any two points in the set lies in it**, $\theta u + (1-\theta)v \in C \ \ \forall u, v \in C$, **intersections of convex sets are convex**, **projections** onto convex sets are **unique and efficient** to compute ($P_C(w') = argmin_v ||v - w'||$)

**Projected Gradient Descent**: add a projection onto $C$ after every step, $w^{(t+1)} = P_C[w^{(t)} - \gamma \nabla \mathcal{L}(w^{(t)})]$, **stochastic version also exists**




**Turning Contrained problems into Uncontrained**: use **penalty functions**:

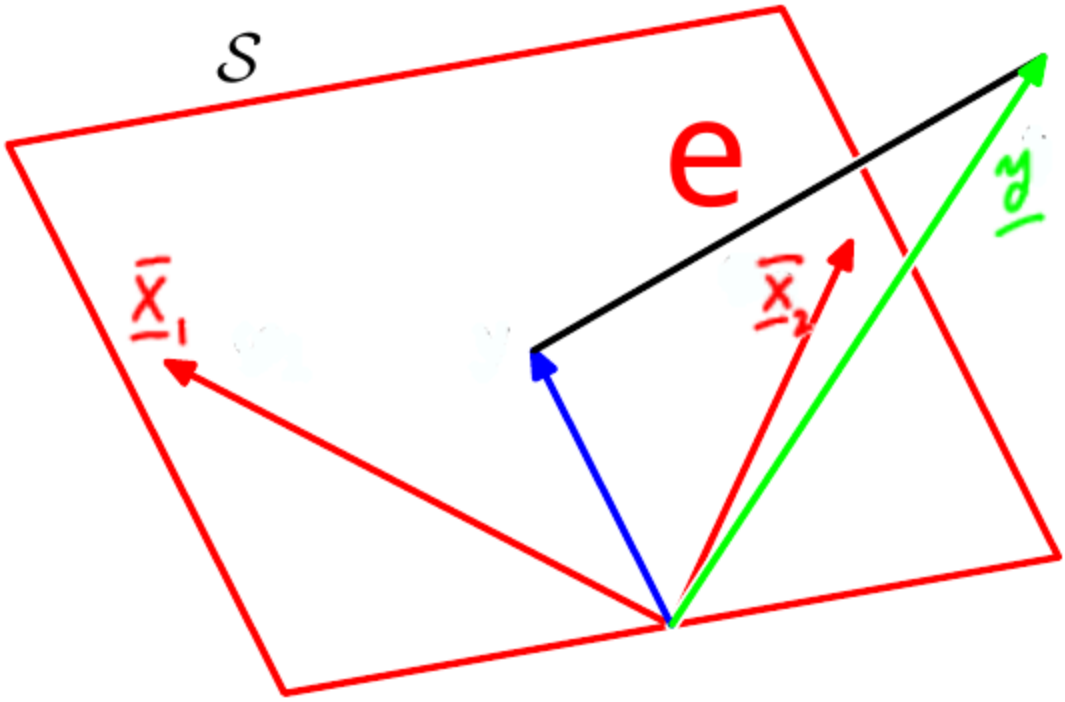
- **brick wall**, $I_C = \{0 \ w \in C, \ \ \infty \ w \notin C, argmin \ \mathcal{L}(w) + I_C(w)$, loss becomes non differentiable and discrete

- **penalize error**, relaxation of the brick wall, increase loss a lot for points outside $C$ ( $argmin\ \mathcal{L}(w) + \gamma||Aw - b||^2$)
- **linearized penalty functions**

# 03 - Least Squares

**Least Squares method**: **solving analytically MSE linear regression** (**linear system of (normal) equations**), **predicting the mean $y$ at each $x$ minimizes the MSE**. We need to **prove that the MSE cost function is convex** in $w$: cost function is a **sum of** $(y_n - x_n^T w)^2$ terms which are the **composition of a linear function with a convex function** (the square); prove convex definition $\mathcal{L}(\lambda w + (1-\lambda)w') \leq \lambda\mathcal{L}(w) + (1-\lambda)\mathcal{L}(w'),\ \ \forall\lambda \in [0,1],\ \ \forall w, w' \ \ \rightarrow \ \ -\frac{1}{2N}\lambda(1-\lambda)||X(w-w')||_2^2$ (clearly positive); or compute the **second derivative** (the **Hessian**) and **show that it's positive semidefinite** (all **eigenvalues are positive**) $\rightarrow \frac{1}{N}X^T X$ (eigenvalues are just the squares of the singular values of $X$). Loss function $\nabla\mathcal{L}(w) = -\frac{1}{N}X^T(y - Xw) = 0$

You want to pick the point in the feature space closest to the $y$ you are predicting (which is not in the feature space), the **best guess is to take the projection of $y$ onto the feature space**, the **error vector** (which is **orthogonal to the feature space**) **connects the projection to the $y$**

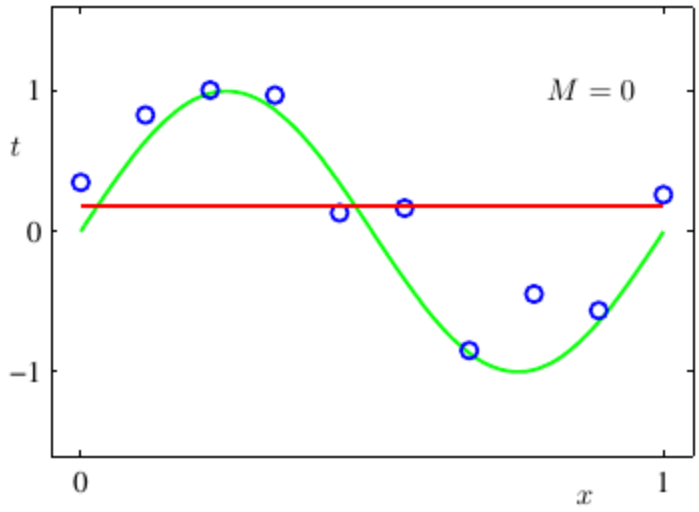



If the **Gram matrix** $X^T X \in \mathbb{R}^{DxD}$ is invertible (if $X$ has **full column rank** ($rank(X) = D$)) we **make the normal equation** $w_* = (X^T X)^{-1} X^T y$. Problem is that $X$ **is often rank deficient** (

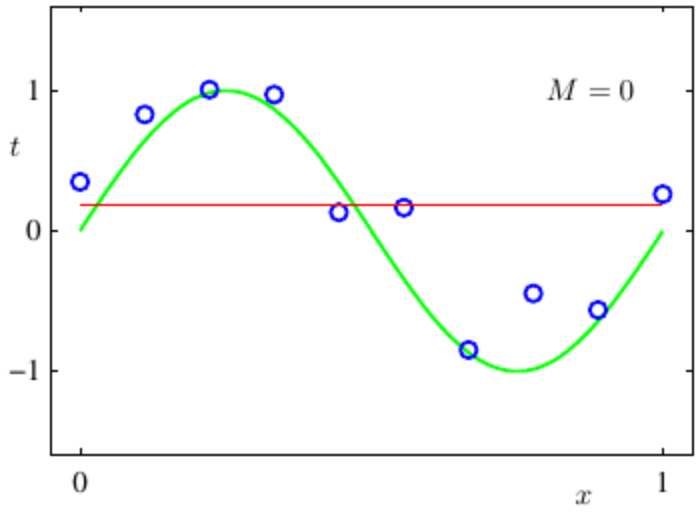
$D > N$ or $D \leq N$ but some columns are nearly collinear (ill-conditioning)), we can still solve Least Squares using a **linear system solver** ( `w_star = np.linalg.solve(X.T @ X, X.T @ y)` )

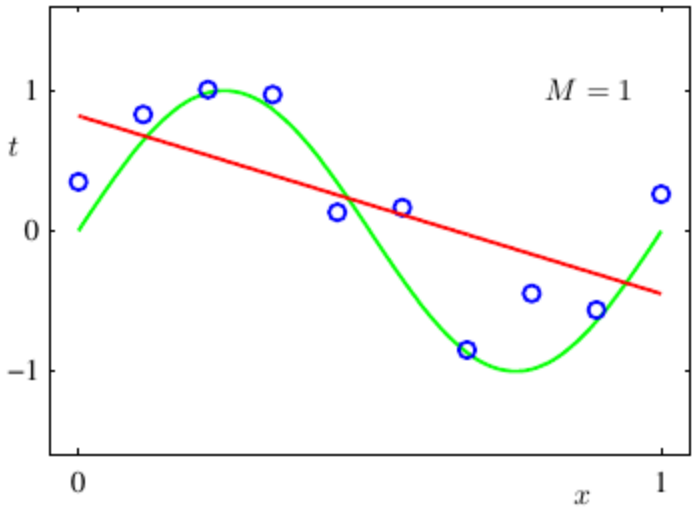
# 04 - Overfitting & Underfitting

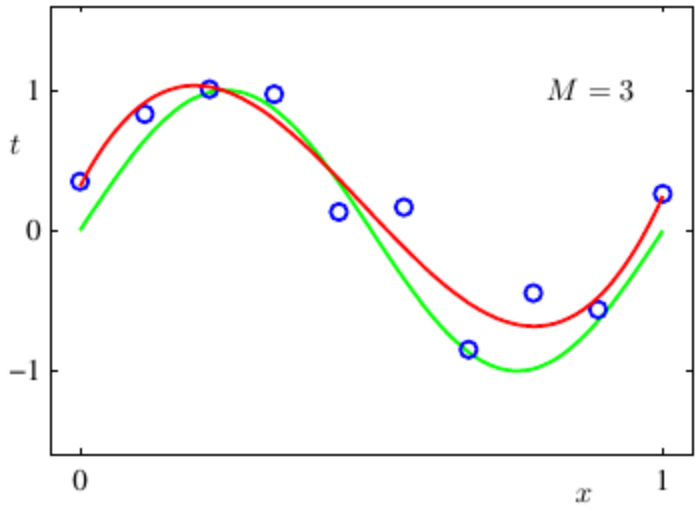
**Underfitting**: model that **does not fit our data well**, **common for linear models with no linear transformation of data**

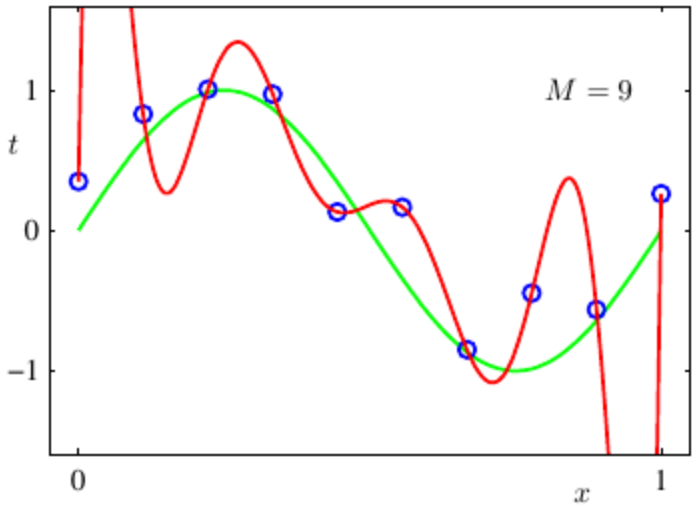


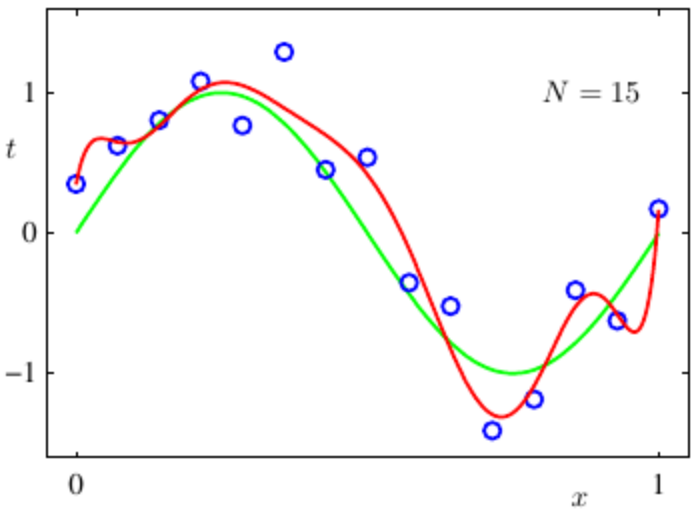

**Overfitting**: model that **does not fit real data well**, **linear model with non linear transformations of the data** (e.g. $\phi(x_n) := [1, x_n, x_n^2, x_n^3, \ldots, x_n^M] \rightarrow y_n = \phi(x_n)^T w$), **increasing the amount of data** while keeping $M$ the same helps

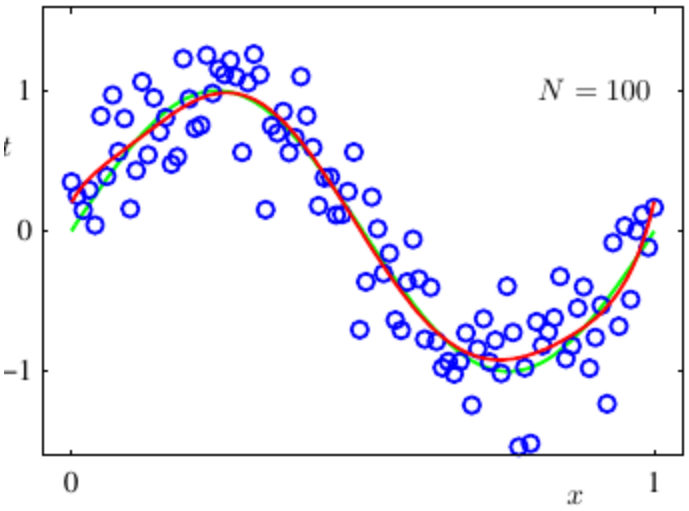

$M = 0$
1
$t$
0
−1
0
$x$
1


$M = 1$
1
$t$
0
−1
0
$x$
1


$M = 3$
1
$t$
0
−1
0
$x$
1


$M = 9$
1
$t$
0
−1
0
$x$
1


$N = 15$
1
$t$
0
−1
0
$x$
1




**Gaussian random vector**: used for **noise**, mean $\mu$, covariance $\Sigma$, the density of probability is $p(y|\mu,\sigma^2) = \mathcal{N}(y|\mu\sigma^2) = \frac{1}{\sqrt{(2\pi)^D det(\Sigma)}} exp[-\frac{1}{2}(y-\mu)^T\Sigma^{-1}(y-\mu)]$

**Probabilistic Least Squares**: we assume $y_n = x_N^T w + \epsilon_n$ where $\epsilon_n$ is the **noise** independent of each sample (a zero-mean Gaussian random variable with variance $\sigma^2$). **Likelihood of producing the correct $y$ vector** given $X, w$ is $p(y|X,w) = \prod_{n=1}^N p(y_n|x_n,w) = \prod_{n=1}^N \mathcal{N}(y_n|x_n^T w, \sigma^2)$ (distribution of the error has center in the perfect prediction $x_n^T w$). The **best model maximizes this likelihood**

**Log-likelihood**: **log of the likelihood** (we can easily do it since the **log is monotonic** and it allows use to switch out the products for sums), $\mathcal{L}_{LL}(w) = log\ p(y|X,w) = -\frac{1}{2\sigma^2}\sum_{n=1}^N (y_n - x_n^T w)^2 + cnst.$, we are **maximising a negative value** (before we were minimizing a positive value)

**Maximum-likelihood estimator** (**MLE**): $argmin_w\ \mathcal{L}_{MSE}(w) = argmax_w\ \mathcal{L}_{LL}(w)$, finds the **model under which the observed data is most likely to have been generated from**, can be applied to any type of distribution (e.g. **Laplace** $\rightarrow p(y_n|x_n,w) = \frac{1}{2b}e^{-\frac{1}{b}|y_n - x_n^T w|}$, uses **MAE instead of MSE, values in different dimensions are not independent**). It's **consistent**, **will give us the correct model if we have enough data** (**Law of Large Numbers**) because the estimator will be the sample **approximation of the expected log-likelihood** (from new unknown data)

**Regularization**: **make complex models simpler so that they generalize better** (**less overfitting**), $min_w\ \mathcal{L}(w) + \Omega(w)$ with $\Omega$ being the **regularizer function** (measures complexity of the model)

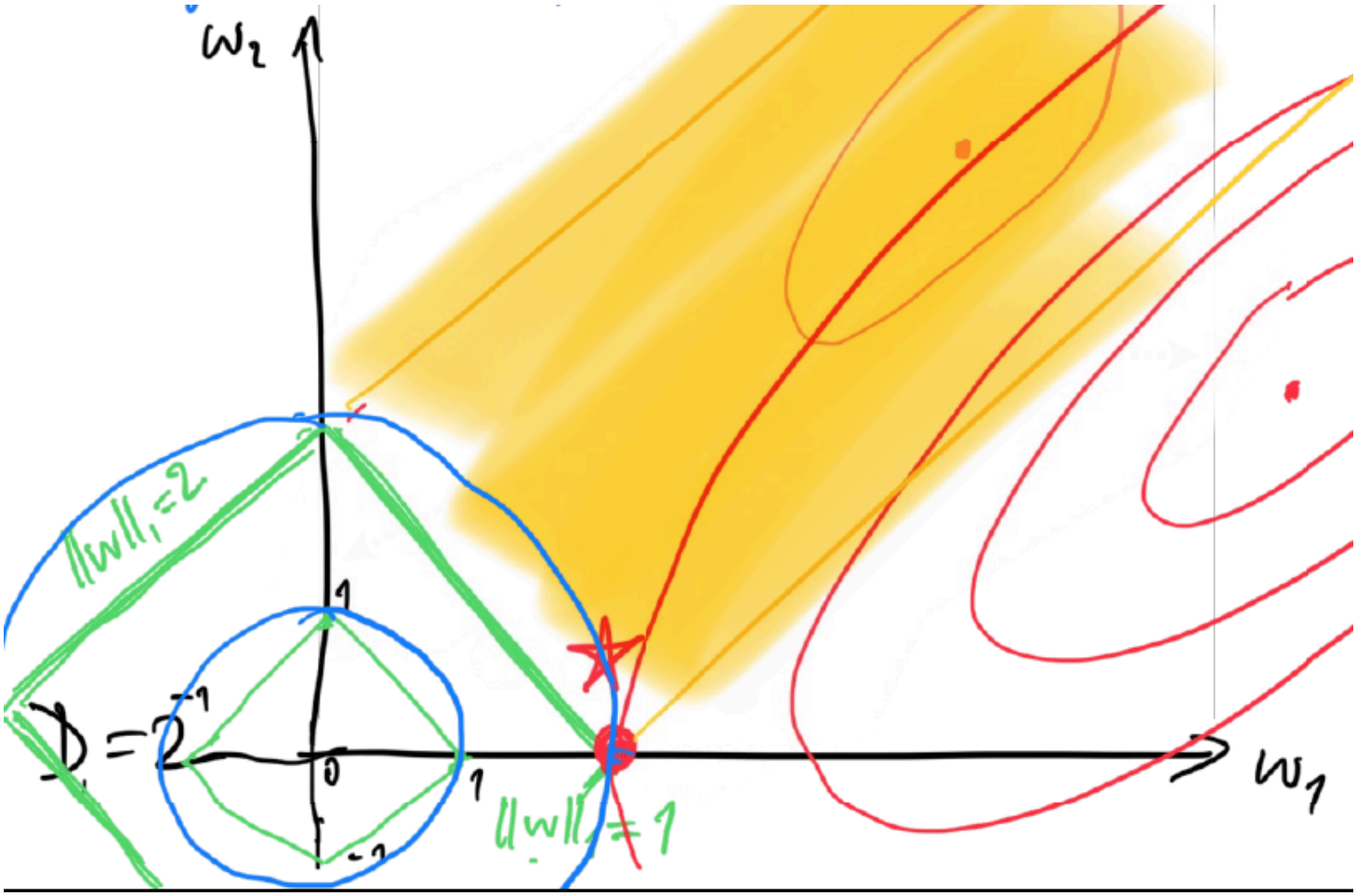
**$L_2$-Regularization**: most **common**, $\Omega(w) = \lambda||w||_2^2 = \lambda\sum_i w_i^2$ (**Euclidean-$L_2$ norm**), **large model weights will be penalized** and considered unlikely

**Ridge Regression**: **MSE** cost function with **$L_2$-Regularization**, $min_w \frac{1}{2N}\sum_{n=1}^N [y_n - x_n^T w]^2 + \lambda||w||_2^2$, Least Squares with $\lambda = 0$, $w_* = (X^T X + 2N\lambda I)^{-1} X^T y$ with $\nabla\mathcal{L}(w) = -\frac{1}{N}X^T(y - X^T w)$, $\Omega(w) = \lambda 2w$, the eigenvalues of the inverted matrix are all at least $2N\lambda > 0$ so the

**inverse always exists** (**lifting the eigenvalues**, useful to make the highest and lowest eigenvalue close to each other and have cheaper computation), proved by decomposing $X^TX = USU^T \rightarrow X^TX + 2N\lambda I = USU^T + 2N\lambda UIU^T = U[S + 2N\lambda I]U^T$ ($[S + 2N\lambda I]$ are the eigenvalues)

$L_1$**-Regularization**: $\Omega(w) = \lambda|w|_1 = \lambda \sum_i |w_i|$ ($L_1$**-norm**)

**Lasso Regression**: **MSE** cost function with $L_1$**-Regularization**, $min_w \frac{1}{2N} \sum_{n=1}^{N} [y_n - x_n^T w]^2 + \lambda||w||_1$, **drives some of the coordinates of** $w$ **to** $0$, sometimes you start with $\lambda = 0$, get the best $w$, the re-iterate starting from the previous best $w$ and increasing $\lambda$, you keep going until the tradeoff is too big. As optimum $w$ we need to pick a point where the **ellipsis defined by the cost function** and the **diamond defined by the** $L_1$**-Regularization** touch (**most likely a corner** of the diamond (coordinate is 0, **sparse solution**))




# 05 - Model Selection

**Data model**: unknown distribution $\mathcal{D} \in X \times Y$, we have a **dataset** $S$ **of independent samples** of $\mathcal{D}$, $S = \{(x_n, y_n)\}_{n=1}^N \sim \mathcal{D}$ i.i.d. (independent and identically distributed)

**Learning Algorithm**: $\mathcal{A}(S) = f_{S,\lambda}$

**Expected/True/Generalized Error/Risk/Loss**: **over all data points** in $\mathcal{D}$, $L_{\mathcal{D}}(f) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[l(y, f(x)]$ where $l$ is the loss function, **what we are fundamentally interested in**. Problem is that $\mathcal{D}$ is unknown

**Empirical Error**: **approximate the true error by averaging the loss function over the dataset** $S$, $L_S(f) = \frac{1}{|S|}\sum_{(x_n,y_n)\in S} l(y_n, f(x_n))$, **law of large numbers** makes it so with a **big enough dataset it's the same to the True Error**. The **loss is the sum of the probability of each sample to be drawn times the probability that our model makes the correct prediction**. Since the **samples in** $S$ **are randomly taken from** $\mathcal{D}$. it's an **unbiased estimator of the True Error**

**Generalization gap**: $|L_{\mathcal{D}}(f) - L_S(f)|$

**Training error**: how **empirical error** is called when it's **calculated on the data the model has been trained on**, **not representative of the error we see on new unknown samples**. We solve this by **splitting the data in training and test set**, $L_{S\,test}(f_{S\,train}) = \frac{1}{|S_{test}|}\sum_{(y_n,x_n)\in S_{test}} l(y_n, f_{S\,train}(x_n)) \rightarrow L_{S\,test}(f_{S\,train}) \approx L_{\mathcal{D}}(f_{S\,train})$, **tradeoff is having less training data**

**Generalization gap with the test set**: $\mathbb{P}[|L_{\mathcal{D}}(f) - L_{S\,test}(f)| \geq \sqrt{\frac{(b-a)^2 ln(2/\delta)}{2|S_{test}|}}] \leq \delta$ with $l \in [a, b]$ and $\delta$ confidence threshold ($1-\delta$ confidence level), the bigger the loss range or the lower the confidence threshold or the smaller the test set the higher generalization gap we allow

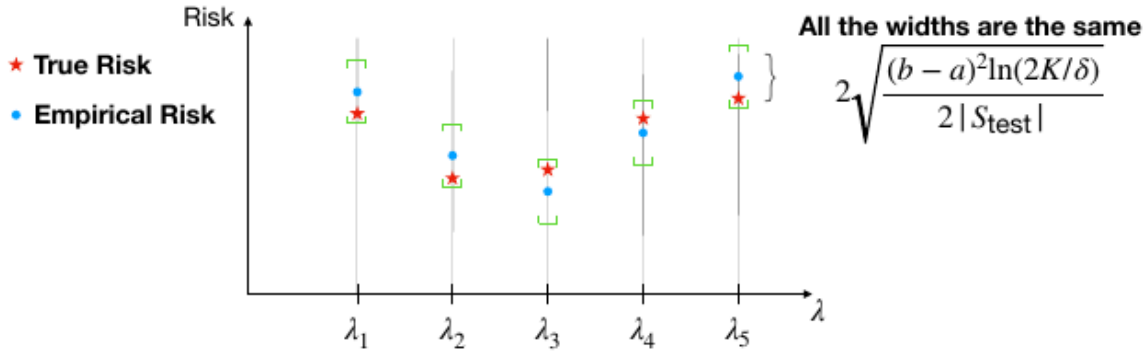
**Probabilistic upper bound of the true error**: $\mathbb{P}[L_{\mathcal{D}}(f_{S\,train}) \geq L_{S\,test}(f_{S\,train}) + \sqrt{\frac{(b-a)^2 ln(2/\delta)}{2|S_{test}|}}] \leq \delta$, tells us the probability $\delta$ that the **true risk using the model we trained** $f_{S\,train}$ **is higher than the empirical risk plus the error margin**

**Concentration inequalities**: estimate the **chance that the empirical error on the test set deviates from the true loss more than a given constant**

**Hoeffding inequality**: $\Theta_i = l(y_i, f(x_i))$, i.i.d losses with mean $\mathbb{E}[\Theta]$ and range $[a, b]$, $\mathbb{P}[|\frac{1}{N}\sum_{n=1}^{N}\Theta_n - \mathbb{E}[\Theta]| \geq \epsilon] \leq 2e^{-2N\epsilon^2/(b-a)^2} \;\; \forall \epsilon \geq 0$, the **empirical mean is concentrated around its true mean** (if you switch out $\Theta$ for it's definition you get what we had before)

**Bounding K test errors from K models with different hyperparameters**: how **can we be sure that the model with the lowest loss is the best one**, $\mathbb{P}[max_k|L_{\mathcal{D}}(f_k) - L_{S\,test}(f_k)| \geq \sqrt{\frac{(b-a)^2 ln(2k/\delta)}{2|S_{test}|}}] \leq \delta$, the higher $K$ testing hyperparameters we use the higher generalization gap we allow (by a $\sqrt{ln(K)}$ factor, we **can test many different models without having them diverge a lot between one another**)

**If we choose the "best" function according to the empirical risk then its true risk is not too far away from the true risk of the optimal choice**




Let $k^* = \text{argmin}_k L_{\mathcal{D}}(f_k)$ and $\hat{k} = \text{argmin}_k L_{S_{\text{test}}}(f_k)$ then

$$\mathbb{P}\left[ L_{\mathcal{D}}(f_{\hat{k}}) \geq L_{\mathcal{D}}(f_{k^*}) + 2\sqrt{\frac{(b-a)^2\ln(2K/\delta)}{2|S_{\text{test}}|}} \right] \leq \delta$$

**Function with the smallest empirical risk** — **Function with the smallest true risk**

**K-Fold Cross-Validation**: **partition data in $K$ groups**/folds, **train $K$ times leaving out one of the groups** each time for testing, **average the $K$ results**

# 06 - Bias-Variance decomposition

**Data model with error**: assume you have the true model for the data $X \sim \mathcal{D}_x,\ y = f(x) + \epsilon$ where $\epsilon \sim \mathcal{D}_\epsilon$ i.i.d. is the noise independent of $X$ with $\mathbb{E}[\epsilon] = 0$

*We are interested in* ***how the expected error of $f_s$: $\mathbb{E}_{(x,y)\sim\mathcal{D}}[(y - f_S(x))^2]$ changes as the train set $S$ and the model complexity change***

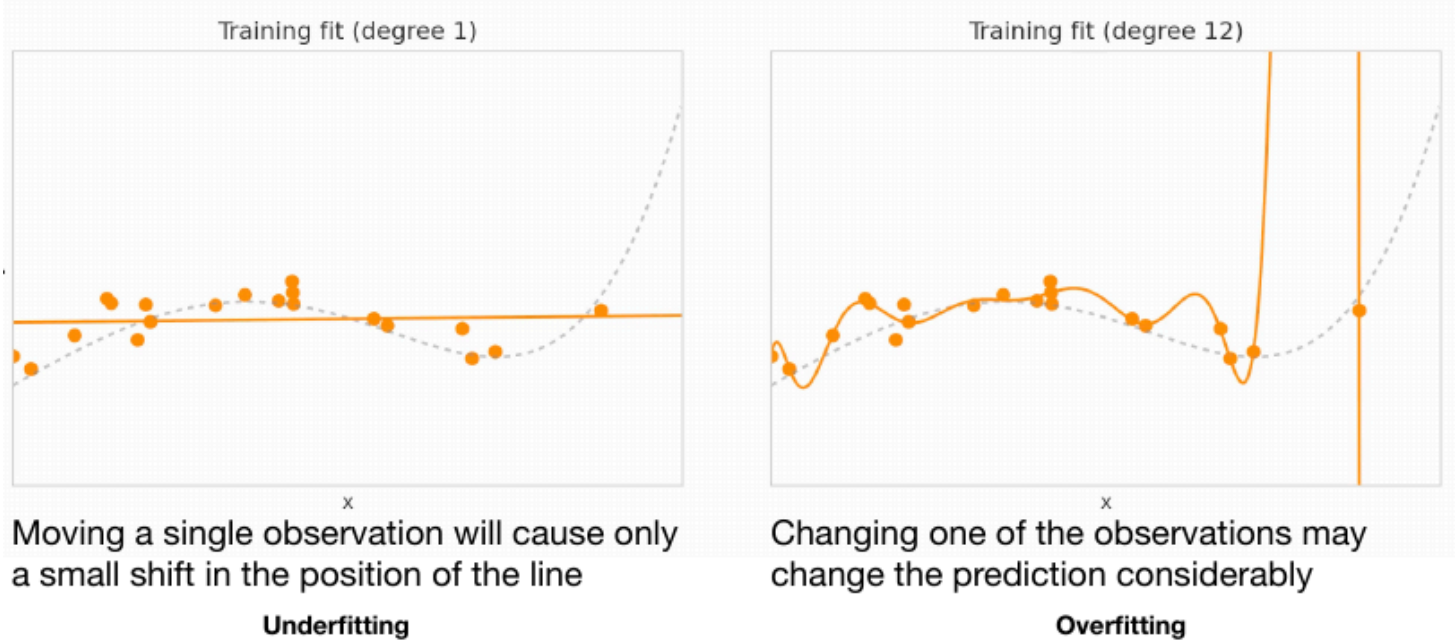
**Error decomposition**: simplified for a fixed element $x_0$, $L(f_S) = \mathbb{E}_{\epsilon\sim\mathcal{D}_\epsilon}[(f(x_0) + \epsilon - f_S(x_0))^2]$, randomness comes from the train set $S$

**Decomposing true risk over the training set**: we are **interested in its expected value**, $\mathbb{E}_{S\sim\mathcal{D}}[L(f_S)] = \mathbb{E}_{S\sim\mathcal{D}}[\mathbb{E}_{\epsilon\sim\mathcal{D}_\epsilon}[(f(x_0) + \epsilon - f_S(x_0))^2]] = \mathbb{E}_{S\sim\mathcal{D},\epsilon\sim\mathcal{D}_\epsilon}[(f(x_0) + \epsilon - f_S(x_0))^2] = Var_{\epsilon\sim\mathcal{D}_\epsilon}[\epsilon] + \mathbb{E}_{S\sim\mathcal{D}}[(f(x_0) - f_S(x_0))^2] = Var_{\epsilon\sim\mathcal{D}_\epsilon}[\epsilon] + (f(x_0) - \mathbb{E}_{S'\sim\mathcal{D}}[f_{S'}(x_0)])^2 + \mathbb{E}_{S\sim\mathcal{D}}[(f_S(x_0) - \mathbb{E}_{S'\sim\mathcal{D}}[f_{S'}(x_0)])^2]$ with $S'$ a second dataset independent of the training set $S$

**Noise variance term**: positive, $Var_{\epsilon \sim \mathcal{D}_\epsilon}[\epsilon]$, **imposes the lower bound** $L(f) = \mathbb{E}[\epsilon^2]$, even if you knew the true real model $f$

**Bias term**: positive, $(f(x_0) - \mathbb{E}_{S' \sim \mathcal{D}}[f_{S'}(x_0)])^2$, (actual value - predicted value)^2, measures in general **how far off the model's predictions are from the correct values**

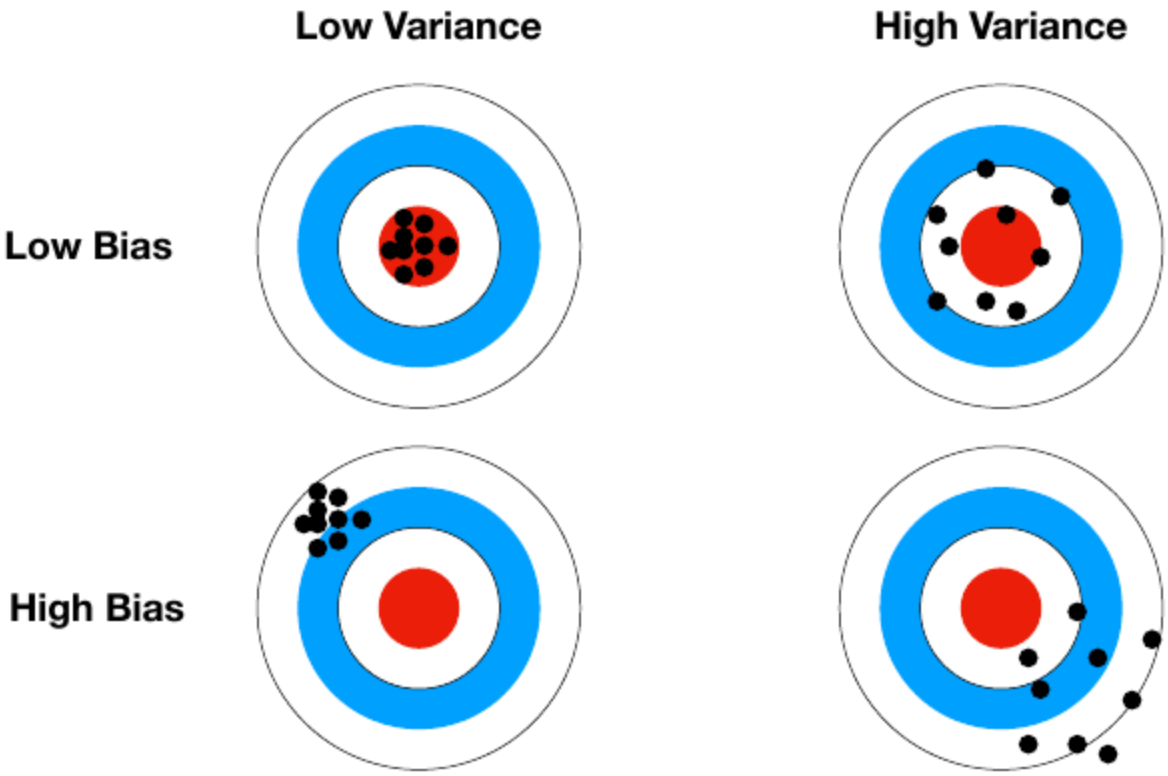
**Variance term**: positive, $\mathbb{E}_{S \sim \mathcal{D}}[(f_S(x_0) - \mathbb{E}_{S' \sim \mathcal{D}}[f_{S'}(x_0)])^2]$, **variance of the prediction function**, measures the **variability of predictions at a given point across different variations of the training set**




Simple models are less sensitive

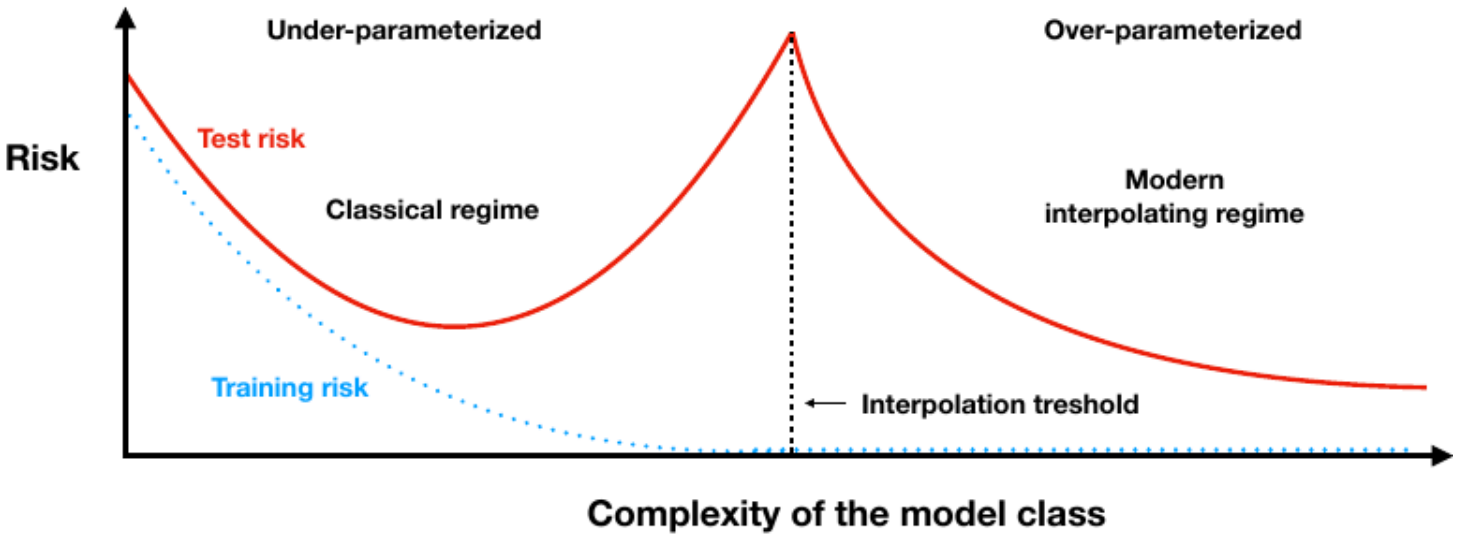
**Bias-Variance tradeoff**: **in practice you don't know the bias** (because you dont have the true function) and **the variance** (because you train a model per complexity level →cannot check how across different variations of the training set your model at a certain complexity level changes). In reality the **only thing you can do is drawing the learning curve**. In the picture below the **total error is given by the variance + the bias + noise** (**decomposition** from before)


If model complexity is too low, approximation will be poor (underfitting)
If model complexity is too high, it may cause issues with variance (overfitting)



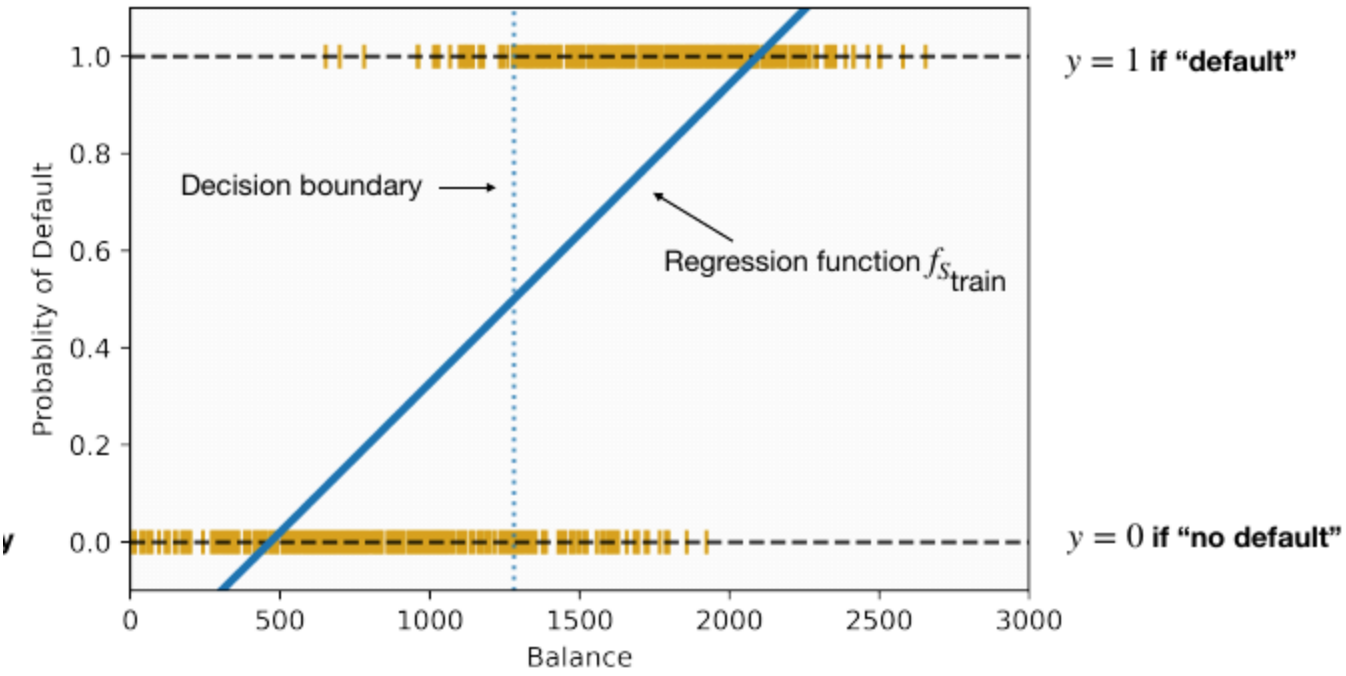

**Double descent curve**: **first descent** is **traditional Machine Learning**, **second descent** is **Deep Learning**, when you **add more parameters than what you need** and **even if the model is complex the test risk goes down** (you don't overfit anymore with a model swinging wildly to hit all training set points, instead **since you have a lot more parameters you have some wiggle room to not overfit**).
**EXPLAINS WHY IN DEEP LEARNING WE JUST ADD MORE PARAMETERS TO LEARN VIA BACKPROPAGATION, TRUSTING THE MODEL WILL LEARN THEM APPROPRIATELY AND GIVING THE MODEL WIGGLE ROOM TO OPERATE AND LEARN**

# Double descent curve



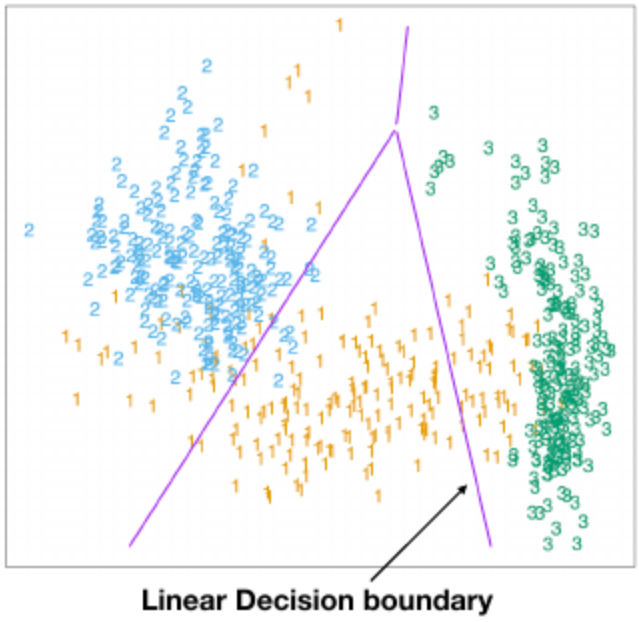

## 07 - Classification

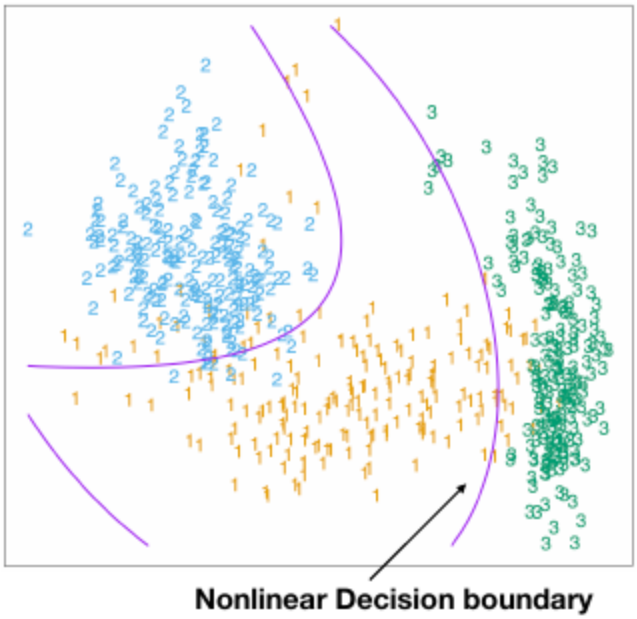
**Classification**: **regression problem with discrete labels**, can use **models from regression by mapping the y to probabilities of belonging to a class**, not a great idea as it's **sensitive to unbalanced data and outliers**




**Classifier**: model/function that **divides the input space in regions belonging to each class**. Can be nonlinear even with a linear model if we apply a **nonlinear trasformation of the input space** (e.g.

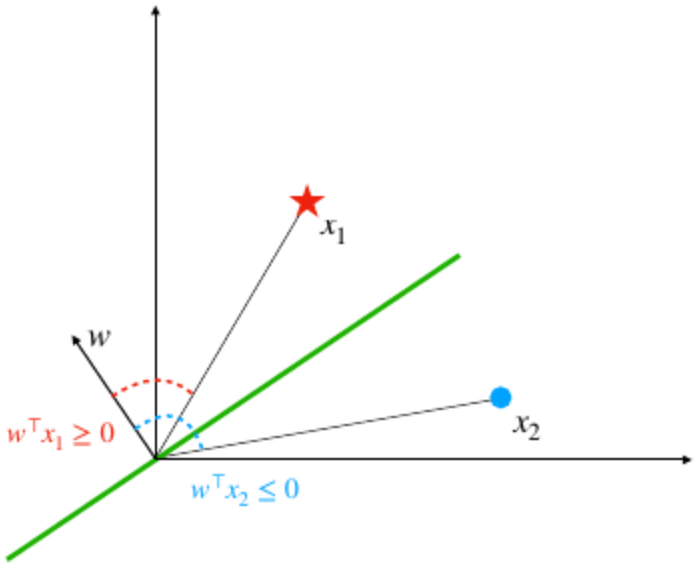
**kernel method**)




**k-Nearest Neighbor (kNN)**: assume that **nearby points in the input space are likely to have similar labels**, in **binary classification k needs to be odd**. Variance decreases as you increase k, bias does the opposite (**bias-variance tradeoff**). **No optimization** needed, **easy** to implement, **works well only in low dimensions** with complex decision boundaries. **Slow at query time** and requires **choosing the right distance measure**

**Linear Decision Boundaries**: for every point $x$ on the green line $\rightarrow w^T x = 0$ ($w^T \perp x$), the **dot product** $\forall y, w^T y$ is the distance from the green line of the projection of $y$ to $w$




**Margin**: **distance from the separating hyperplane** to the closest point, $min_{n \leq N} |w^T x_n|$

**Max-margin separating hyperplane**: hyperplane that **maximizes the margin** so that **if the training set slightly changes the misclassification will stay low**, $max_{w,||w||=1}(min_{n \leq N} |w^T x_n|) \;\; s.t. \; \forall n, y_n x_n^T w \geq 0$ (last part makes it so we are **assigning the right classes**)

**Loss function for Binary Classification**: $l(y, y') = 1_{y \neq y'}$ (indicator variable)

**True error for Binary Classification**: $L_{\mathcal{D}}(f) = \mathbb{E}_{\mathcal{D}}[1_{Y \neq f(X)}] = \mathbb{P}_{\mathcal{D}}[Y \neq f(X)]$, first is the **classification error**, second one is the **probability of making an error**

**Bayes Classifier**: classifier $f_* = argmin_f \; L_{\mathcal{D}}(f)$, we say $f_*(x_0) \in argmax_{y \in \{-1,1\}} \mathbb{P}(Y = y | X = x_0)$ **after fixing the input $x_0$ we take the most probable class** to be assigned to $x_0$. **Unattainable gold standard** as we never know the underlying $\mathcal{D}$ data distribution

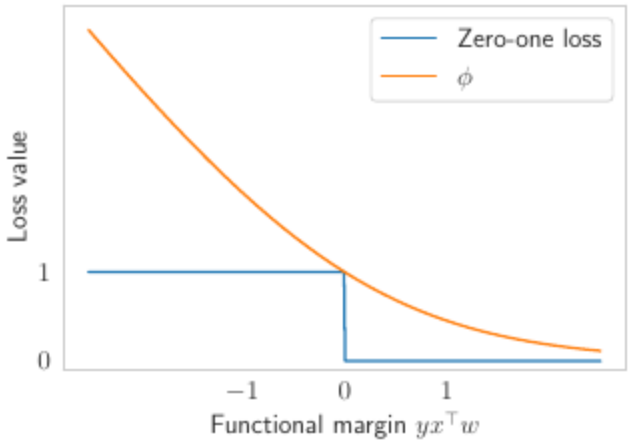
**Non-parametric classification algorithm**: **approximate the conditional distribution** $\mathbb{P}(Y = y | X = x)$ via **local averaging** (KNN)

**Parametric classificatiolgorithm**: **approximate the true distribution** $\mathcal{D}$ via training data, **minimize the classification/empirical risk** on training data (**Empiral Risk Minimization**, **ERM**) mimicking what you would do if you had the whole $\mathcal{D}$ distribution. $min_f \; L_{train}(f) = \frac{1}{N} \sum_{n=1}^{N} 1_{f(x_n) \neq y_n} = \frac{1}{N} \sum_{n=1}^{N} 1_{y_n f(x_n) \leq 0}$ with $y \in \{-1, 1\}^D$, problem is that $L_{train}(f)$ **is not convex** because $1_{y_n f(x_n) \leq 0}$ (**0-1 loss**) and $Y$ are **not continuous**

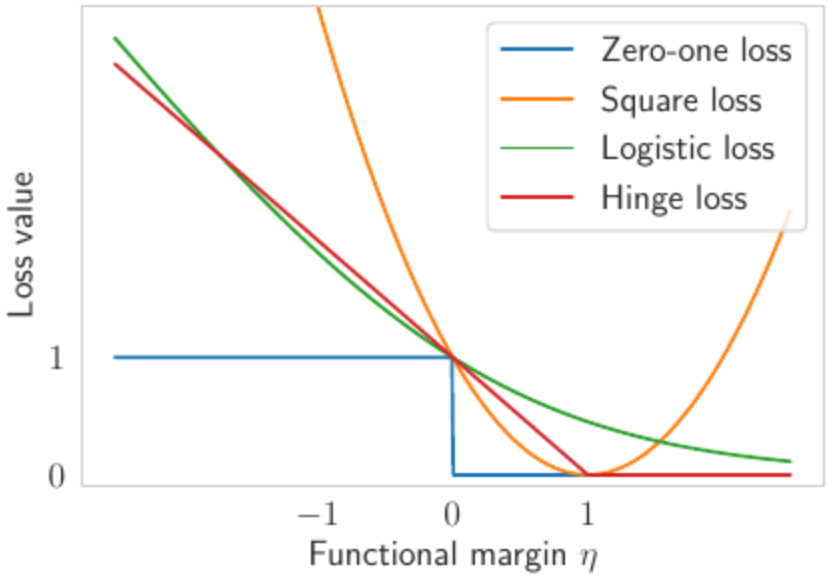
**Convex relation of the classification risk by ERM**: given different models $g : X \to \mathbb{R}$, $g \in \mathcal{G}$ **convex subset of continuous functions** (take any two functions within and you can combine them in convex functions, e.g. linear functions) you **predict $y$ labels** $f(x) = sign(g(x))$. Problem becomes

$min_{g \in \mathcal{G}} \frac{1}{N} \sum_{n=1}^{N} 1_{y_n g(x_n) \leq 0}$, which becomes $min_{g \in \mathcal{G}} \frac{1}{N} \sum_{n=1}^{N} \phi(y_n g(x_n))$ with $\phi$ a **convex function** of the functional **margin** $y_n g(x_n)$ which is **also an upper bound to the 0-1 loss** (the one with the indicator, so that $\mathbb{E}[\phi(y_n g(x_n))] \geq \mathbb{E}[1_{y_n f(x_n) \leq 0}]$)

Use a $g$ **function that is continuous** (unlike $f$ which is $0$ or $1$) and produces **outputs that can be mapped to** $y$ **labels** (e.g. sign(g)), **apply a convex function** $\phi$ **that mantains convexity when applied to** $g$ and is an **upper bound to the true loss** (this way if we minimize our loss we know we are also minimizing the true loss), we now have a **convex continuous problem**



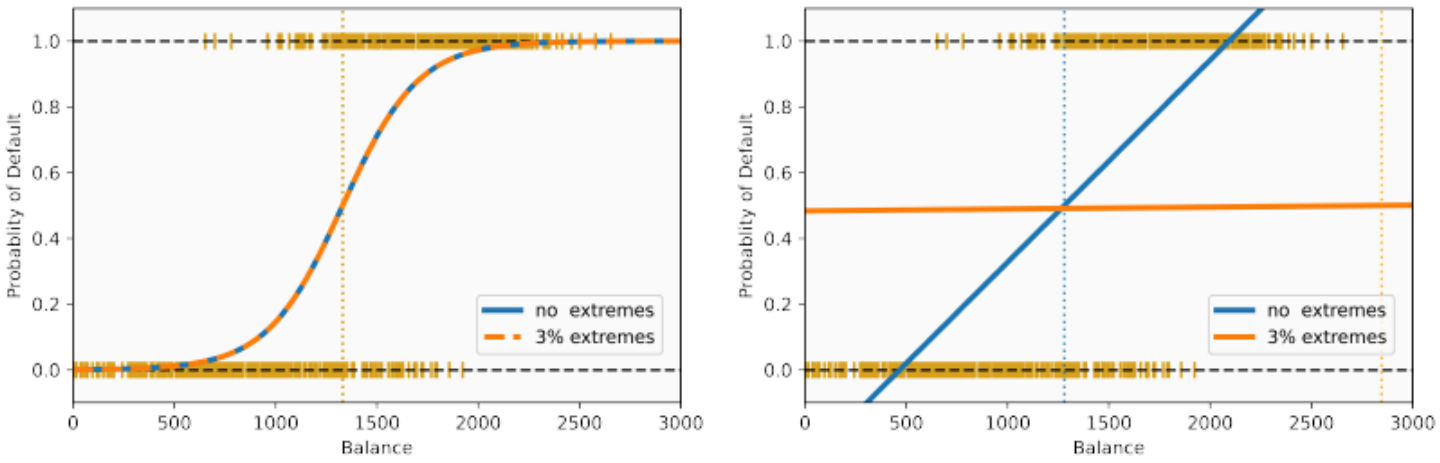

**Losses for classification**: **convex and upper bound for the 0-1 loss**, with functional margin $\eta = yx^T w$, **quadratic loss** $MSE(\eta) = (1-\eta)^2$ (penalizes any deviation from 1), **logistic loss** $Logistic(\eta) = \frac{log(1+e^{-\eta})}{log(2)}$ (asymmetric cost + always penalizes) and **hinge loss** $Hinge(\eta) = [1-\eta]_+$ (penalizes only outside range (below -1) and confidence-lacking predictions)




**Over-parametrization regime**: when $N < D$, **all training points are equally close to the separating hyperplane** with high probability, so the outcome of gradient descent is the same both for the logistic and MSE loss (so **regression and classification models are equivalent**)

# 08 - Logistic Regression

**Logistic Regression**: **maps the output to** $[0, 1]$ (**probability of belonging to a class**), $\sigma(\eta) = \frac{e^\eta}{1+e^\eta}$ $(1 - \sigma(\eta) = \frac{1}{1+e^\eta}$ and $\sigma'(\eta) = \sigma(\eta)\sigma(1-\eta))$. $p(1|x) = \mathbb{P}(Y = 1|X = x) = \sigma(x^T w + w_0)$, $p(0|x) = \mathbb{P}(Y = 0|X = x) = 1 - \sigma(x^T w + w_0)$. Very large $|x^T w + w_0|$ makes $p(1|x)$ very close to 1 or 0 (high **confidence**). You must **pick a probability threshold to determine the predicted class** from the predicted probability. Scaling $w$ makes the transition faster or slower, changing $w_0$ shifts up and down the decision region along $w$ (without it would pass through the origin every time). **Robust towards unbalanced data and outliers**




**Logistic regression** **Linear regression**

**MLE for logistic regression**: $w^* = argmax\ \mathcal{L}(w) = \prod_{n=1}^N p(z_n|w) = \prod_{n=1}^N \sigma(x_n^T w)^{z_n}(1 - \sigma(x_n^T w))^{1-z_n}$ with $\mathcal{L}$ likelihood function and $z_1, \ldots, z_n$ observations/outcomes, it's **consistent**, **if the data was generated like the model models it** then it converges to the true parameter

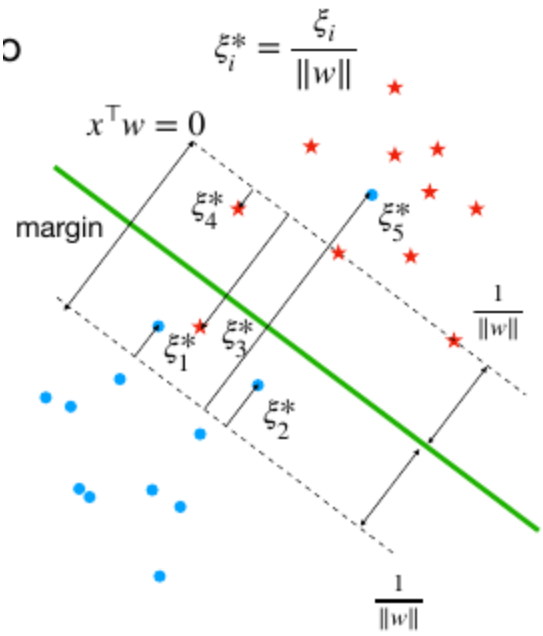
**Negative log-likelihood (NLL, logistic loss)**: $w_* = argmin\ [-log(\mathcal{L}(w))] = argmin\ \sum_{n=1}^N -log(p(z_n|w)) = \frac{1}{N}\sum_{n=1}^N -y_n x_n^T w + log(1 + e^{x_n^T w})$ **convex function** (proven under **hessian is positive semi definite** or the fact that we are doing **positive combinations and compositions of convex functions** and $log(1 + e^x)$ is convex (positive second derivative)), **more convenient to work with then MLE**. $\nabla\mathcal{L}(w) = \frac{1}{N}\sum_{n=1}^N(\sigma(x_n^T w) - y_n)x_n = \frac{1}{N}X^T(\sigma(Xw) - y)$. Perfect **convex surrogate (stand-in) for 0-1 loss** which is not differentiable in 0 (also used in **neural nets to predict classes**). **Newton's method minimizes the quadratic approximation** (using **second order** information) $L(w) \sim L(w_t) + \nabla L(w_t)^\top (w - w_t) + \frac{1}{2}(w - w_t)^\top \nabla^2 L(w_t)(w - w_t) := \phi_t(w),\ \tilde{w} = argmin\ \phi_t(w) \to \nabla L(w_t) + \nabla^2 L(w_t)(\tilde{w} - w_t) = 0,\ w_{t+1} = w_t - \gamma_t \nabla^2 L(w_t)^{-1}\nabla(w_t)$, **faster convergence but higher computational complexity**. **When the data is linearly separable** the weights will go to $\infty$ so you need to **add L2-regularization**

# 09 - Support Vector Machines

**Linear Classifier**: define hyperplane as $\{x : w^T x = 0\},\ ||w|| = 1$, predictions by $f(x) = sign(x^T w)$, distance between point $x_0$ and the hyperplane is $|w^T x_0|$

**Hard-SVM rule**: we assume that the **dataset is linearly separable** and therefore we **can compute a max-margin separating hyperplane that correctly classifies all points**. We introduce $M$ as a **threshold margin that correctly classifies all points** and have $max_{M \in \mathbb{R}, w, ||w||=1} M \ \ s.t.\ \forall n, y_n x_n^T w \geq M$, we imposed $||w|| = 1$ so that $M$ represents the true margin. We can instead make $M = \frac{1}{||w||}$ and then have $min_w \frac{1}{2}||w||^2 \ \ s.t.\ \forall n, y_n x_n^T (w/M) \geq M \rightarrow y_n x_n^T w' \geq 1$ ($w'$ is just $w$ scaled by $\frac{1}{M}$)

**Soft-SVM rule**: **allow some constraints to be violated** by introducting slack variables $\xi_1, \ldots, \xi_n$that we also want to minimize, $min_{w,\xi} \frac{\lambda}{2}||w||^2 + \frac{1}{N}\sum_{n=1}^N \xi_n \ \ s.t.\ \forall n, y_n x_n^T w \geq 1 - \xi_n,\ \xi_n \geq 0$. Can be **simplified to** $min_w \frac{\lambda}{2}||w||^2 + \frac{1}{N}\sum_{n=1}^N [1 - y_n x_n^T w]_+$ (**hinge loss** because if $y_n x_n^T w \geq 1 \rightarrow \xi_n = 0$ (no slack needed, the point is far enough) else $y_n x_n^T w < 1 \rightarrow \xi_n = 1 - y_n x_n^T w$). **First term maximizes the margin** while trying to find the best angle to separate points, **second term measures accuracy** of the predictions, $\lambda$ **manages the tradeoff between overall accuracy and margin size**. $w^*$ obtained through (stochastic) **subgradient method**




**Convex duality**: assume you can define an auxiliary function $G(w, \alpha)\ s.t.\ min_w L(w) = min_w max_\alpha G(w, \alpha)$, **primal problem** $min_w max_\alpha G(w, \alpha)$ and **dual problem** $max_\alpha min_w G(w, \alpha)$ (**sometimes easier to solve**). **Primal and dual problems are equal if** $G$ **is convex in** $w$ **and in** $\alpha$ , otherwise dual $\leq$ primal. $\alpha$ **is typically sparse** but **non-zero for the training samples crucial in determining the decision boundary**

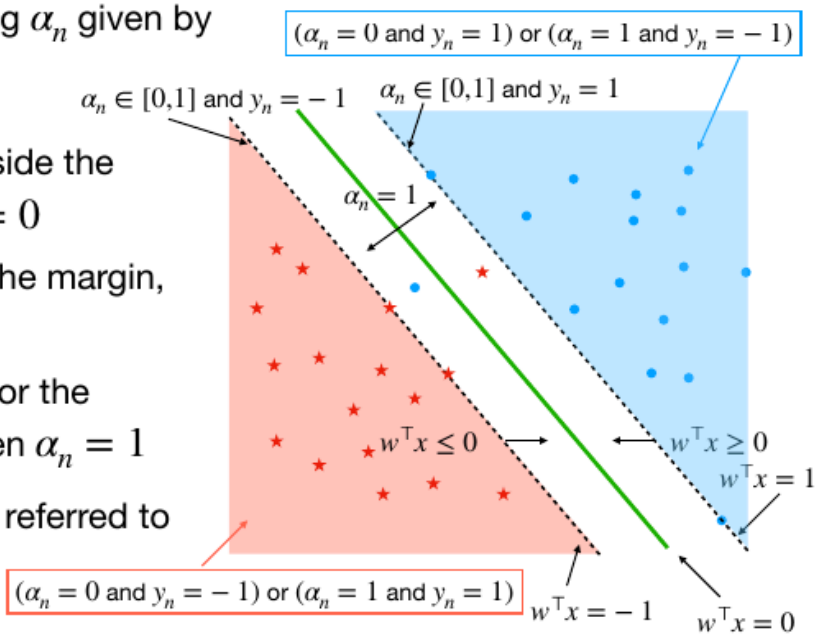
**SVM with convex duality**: Hinge loss $[z]_+ = max(0, z) = max_{\alpha \in [0,1]} \alpha z$, therefore $min_w L(w) = min_w max_{\alpha \in [0,1]^n} \frac{1}{N} \sum_{n=1}^{N} \alpha_n (1 - y_n x_n^T w) + \frac{\lambda}{2} ||w||_2^2$ with $G(w, \alpha) = \frac{1}{N} \sum_{n=1}^{N} \alpha_n (1 - y_n x_n^T w) + \frac{\lambda}{2} ||w||_2^2$ **convex in $w$ and concave in $\alpha$**. $\nabla_w G(w, \alpha) = -\frac{1}{N} \sum_{n=1}^{N} \alpha_n y_n x_n + \lambda w = 0 \rightarrow w(\alpha) = \frac{1}{\lambda N} \sum_{n=1}^{N} \alpha_n y_n x_n = \frac{1}{\lambda N} X^T Y \alpha$, so using this last part the problem becomes $min_w L(w) = max_{\alpha \in [0,1]^n} \frac{1^T \alpha}{N} - \frac{1}{2\lambda N^2} \alpha^T Y X X^T Y \alpha$. Now the **problem is differentiable and convex** so **efficient solutions** can be achieved with **coordinate ascent and quadratic programming solvers**. **Cost function depends on the data via the Kernel/Gram matrix $K = XX^T \in \mathbb{R}^{N \times N}$, completely independent of the number of features $D$** (**kernel trick**). Thanks to the dual problem you are now optimizing the loss by finding the Lagrange multipliers $\alpha_i$ to assign to each data point $x_i$ (not a vector of dimension $D$), **training cost is a function of the number of samples, not features**

For any $(x_n, y_n)$, there is a corresponding $\alpha_n$ given by

$$\max_{\alpha_n \in [0,1]} \alpha_n (1 - y_n x_n^\top w)$$

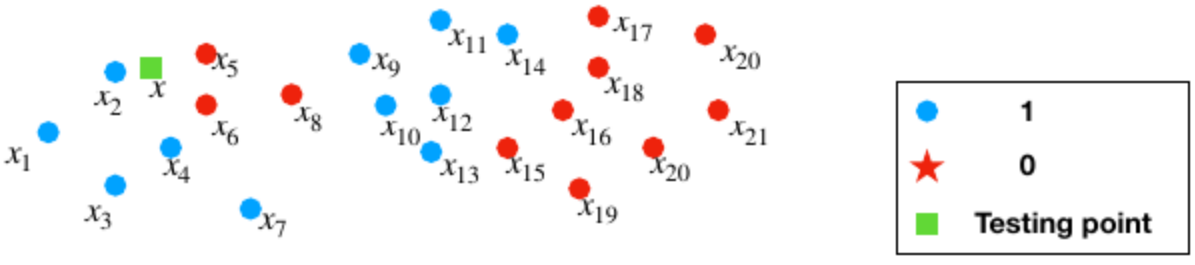
- If $x_n$ is on the correct side and outside the margin, $1 - y_n x_n^\top w < 0$, then $\alpha_n = 0$
- If $x_n$ is on the correct side and on the margin, $1 - y_n x_n^\top w = 0$, then $\alpha_n \in [0,1]$
- If $x_n$ is strictly inside the margin or or the incorrect side, $1 - y_n x_n^\top w > 0$, then $\alpha_n = 1$

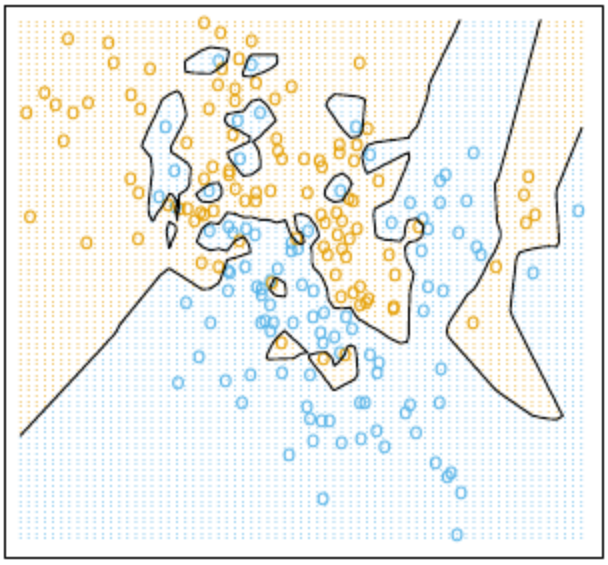
➡The points for which $\alpha_n > 0$ are referred to as support vectors




# 10 - kNN & Curse of Dimensionality

**Nearest neighbor function**: $nbh_{S_{train},k}(x)$ returns the $k$ elements of $S_{train}$ closest to $x$, **ties are often broken randomly**, different metrics can be used. **High computational complexity for large $N$**. **Relevant if spatial correlation**, models **intricate decision boundaries implicitly**

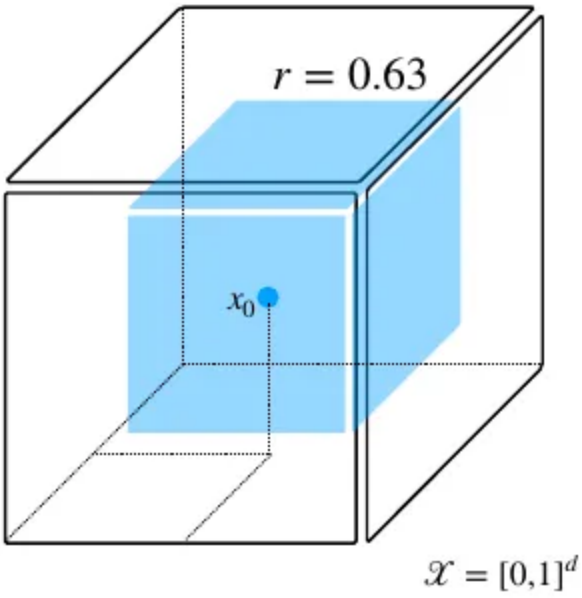


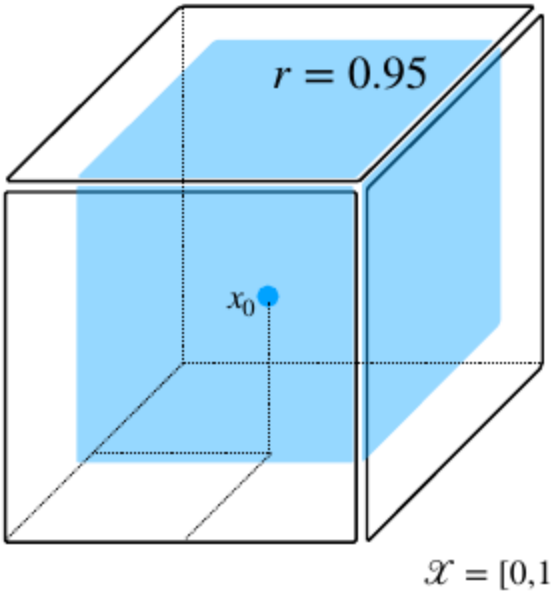


**kNN for regression**: $f_{S_{train},k}(x) = \frac{1}{k} \sum_{n:x_n \in nbh_{S_{train},k}(x)} y_n$

**kNN for classification**: $f_{S_{train},k}(x) = majority\{y_n : x_n \in nbh_{S_{train},k}(x)\}$

**Curse of dimensionality**: **as dimensionality grows the data gets more and more sparse** (each time you **add a dimension you add a degree of freedom** for points (one more reason to not be near other points)), as dimensionality grows the **distance between all points becomes the same high value** and algorithms like **kNN become useless**. Also, say you want to know the probability of a point being in a subset of the feature space $P(x \in S) = r^d = \alpha, x \in X, X = [0,1]^d, S \subset [0,1]^d$, to have the same probability $\alpha = 0.01$ of $x \in S$ with $d = 10 \rightarrow r = 0.63 \rightarrow 0.63^{10} = 0.01 = \alpha$, with $d = 100 \rightarrow r = 0.95$, to find the same 10 points as dimensionality grows you need to explore exponentially bigger areas. $P(\exists x_i \in S) \geq \frac{1}{2} \rightarrow r \geq (1 - \frac{1}{2^{\frac{1}{N}}})^{\frac{1}{d}}, \quad S \in [0,1]^d$






**Generalization bound for 1-NN**: **probability of getting a wrong prediction for new data using 1-NN** ($L(f) = P_{(X,Y)\sim\mathcal{D}}(Y \neq f(X))$). Compare the 1-NN model against **baselines**: **Bayes classifier** (**theoretical best performance** possible, predict 1 if the probability of the label being 1 given $x$ is above $1/2$, $f^*(x) = 1_{\eta(x)\geq 1/2},\ \ \eta(x) = P(Y = 1|X = x)$) and **Bayes risk** (**minimum possible probability of misclassification**, since you predict 1 when $\eta(X) \geq 1/2$ the probability you will be wrong is $1 - \eta(X)$, $L(f^*) = P(f^*(X) \neq Y) = \mathbb{E}_{X\sim\mathcal{D}_X}[min(\eta(X), 1 - \eta(X))]$, if $\eta(x) = 0.6$ for a $x$, the Bayes classifier predicts 1, but 40% of the time, the label will actually be 0). Assuming that nearby points have the same label ($\exists c \geq 0, \forall x, x' \in X\ |\eta(x) - \eta(x')| \leq c||x - x'||_2$), then the **Generalization bound for 1-NN is** $\mathbb{E}_{S_{train}}[L(f_{S_{train}})] \leq 2L(f^*) + c\mathbb{E}_{S_{train},X\sim\mathcal{D}_X}[||X - nbh_{S_{train},1}(X)||] \leq 2L(f^*) + 4c\sqrt{d}N^{\frac{-1}{d+1}}$, for constant $d$ and $N \to \infty$ : $\mathbb{E}_{S_{train}}[L(f_{S_{train}})] \leq 2L(f^*)$ (with low dimensionality $d$ and infinite data 1-NN will be at best twice as error-prone as the bayes classifier, which is good!). **Increasing $d$ requires increasing $N$ exponentially** if you want to keep the same error ($N \propto d^{(d+1)/2}$, **curse of dimensionality**), **interpolation can help generalize**

**Geometric term of Generalization bound for 1-NN**: $\mathbb{E}_{S_{train},X\sim\mathcal{D}_X}[||X - nbh_{S_{train},1}(X)||] \leq 4\sqrt{d}N^{\frac{-1}{d+1}}$, **average euclidean distance between a point and it's closest neighbor**, it's the main thing the error depends on. **High dimensions with non exponentially higher number of data points means that the distances between data points will be so big that it will be hard to define local neighborhoods ⇒ kNN breaks down**

**Local averaging method**: tries to **approximate directly the Bayes predictor**/classifier/regressor **without any optimization** algorithms, **approximates the true conditional distribution** $p(y|x)$ with some $\hat{p}(y|x)$ (for **kNN** $\hat{p}(y|x) = \sum_{n=1}^{N} \hat{w}_n(x)1_{y=y_n},\ \ \hat{w}(x) = \frac{1}{k}$ for the $k$ nearest neighbors (0 otherwise)). Classification with 0-1 loss: $f(x) \in argmax_{y\in Y}\hat{P}(Y = y|x)$. Regression with square loss: $f(x) = \hat{\mathbb{E}}[Y|x] = \int_Y y\hat{p}(y|x)\,dy$

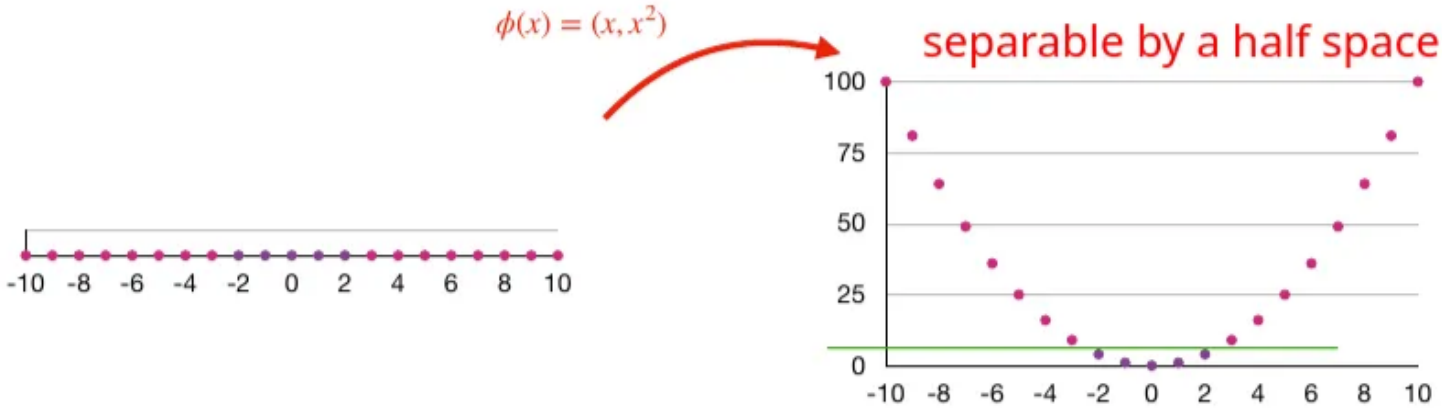
# 11 - Kernel Trick

**Ridge Regression Alternative form**: $w_* = \frac{1}{N}X^T(\frac{1}{N}XX^T + \lambda I_N)^{-1}y$, $O(N^3 + dN^2)$I instead of $O(d^3 + Nd^2)$. Also, you could have $w_* = X^T\alpha_*$ with $\alpha_* = \frac{1}{N}(\frac{1}{N}XX^T + \lambda I_N)^{-1}y$

**Representer Theorem**: for any loss function $l$ and any minimizer $w_* \in argmin_w \frac{1}{N}\sum_{n=1}^{N} l(x_n^T w, y_n) + \frac{\lambda}{2}||w||^2$ there exists $\alpha_* \in \mathbb{R}^N \ \ s.t.\ w_* = X^T\alpha_*$, meaning **there exists an optimal solution within the feature space** (span $\{x_1, \dots, x_N\}$). **Enables the Kernel Trick to various problems**

**Kernelized Ridge Regression**: $a_* = argmin_\alpha \frac{1}{2}\alpha^T(\frac{1}{N}XX^T + \lambda I_N)\alpha - \frac{1}{N}\alpha^T y$, **uses $X$ only through the kernel matrix** $K = XX^T$ (as opposed to having $Xw$)

**Feature Extractor**: **embeds data into a new features space** by applying a **trasformation to your data $\mathbb{R}^d$ in order to map it to a new feature space $\mathbb{R}^{\hat{d}}$**. Usually because your **data is not linearly separable**, so you **embed/map it into a feature space of higher dimension** where it's linearly separable. **If $d <<< \hat{d}$, calculating the Kernel matrix becomes too expensive** ($O(\hat{d})$). Also used to **extract relevant features from data** (like in **Neural Networks**)



When a feature map $\phi : \mathbb{R}^d \to \mathbb{R}^{\tilde{d}}$ is used,

$$(x_n)_{n=1}^N \hookrightarrow \left(\phi(x_n)\right)_{n=1}^N$$

The associated kernel matrix is

$$\mathbf{K} = \mathbf{\Phi}\mathbf{\Phi}^\top = \begin{pmatrix} \phi(x_1)^\top\phi(x_1) & \phi(x_1)^\top\phi(x_2) & \cdots & \phi(x_1)^\top\phi(x_N) \\ \phi(x_2)^\top\phi(x_1) & \phi(x_2)^\top\phi(x_2) & \cdots & \phi(x_2)^\top\phi(x_N) \\ \vdots & \vdots & \ddots & \vdots \\ \phi(x_N)^\top\phi(x_1) & \phi(x_N)^\top\phi(x_2) & \cdots & \phi(x_N)^\top\phi(x_N) \end{pmatrix} \in \mathbb{R}^{N\times N}$$

**Kernel Trick**: to **build out the Kernel Matrix uses a kernel function** $\kappa(x, x')\ s.t\ \kappa(x, x') = \phi(x)^T\phi(x')$ that **achieves that same dot product we want but in the lower original dimensional space** ($\mathbb{R}^d$), since in the new feature space ($\mathbb{R}^{\hat{d}}$) it's expensive to compute that dot product. The kernel function is also a **measure of similarity between points**. Allows us to **never have to compute** $\phi(x)$ **but still get the results we would have had if we were in that higher dimensional space**. **Predictions require computing the kernel function with the new point** $x$ **and every point in the training set**, calculated this way: $y = \phi(x)^T w_* = \phi(x)^T\phi(x)^T\alpha_* = \sum_{n=1}^N \kappa(x, x_n)\alpha_{*n}$, first equivalence is a linear prediction in the (new) feature space, last equivalence is a non linear prediction in the original one

1. Linear kernel: $\kappa(x, x') = x^\top x'$
   - ➡ Feature map is $\phi(x) = x$
2. Quadratic kernel: $\kappa(x, x') = (xx')^2$ for $x, x' \in \mathbb{R}$
   - ➡ Feature map is $\phi(x) = x^2$

Let $x, x' \in \mathbb{R}^3$

Polynomial: $\kappa(x, x') = (x_1x_1' + x_2x_2' + x_3x_3')^2$

Feature map:

$$\phi(x) = [x_1^2, x_2^2, x_3^2, \sqrt{2}x_1x_2, \sqrt{2}x_1x_3, \sqrt{2}x_2x_3] \in \mathbb{R}^6$$

**Radial basis function (RBF) kernel**: $x, x' \in \mathbb{R}^d,\ \kappa(x, x') = e^{-(x-x')^T(x-x')}$, the feature map was $\phi(x) = e^{-x^2}(\dots, \frac{2^{k/2}x^k}{\sqrt{k!}}, \dots)_{k=0}^{\infty}$ (**infinite dimensional vector**), feature map **cannot be represented in a finite-dimensional space as an inner product**

**Building new kernels from existing ones**: given $\kappa_1, \kappa_2, \phi_1, \phi_2$, **positive finite linear combinations of kernels are kernels** $\kappa(x, x') = \alpha\kappa_1(x, x') + \beta\kappa_2(x, x')\ \forall \alpha, \beta \geq 0$ and **products of kernels are kernels** $\kappa(x, x') = \kappa_1(x, x')\kappa_2(x, x')$

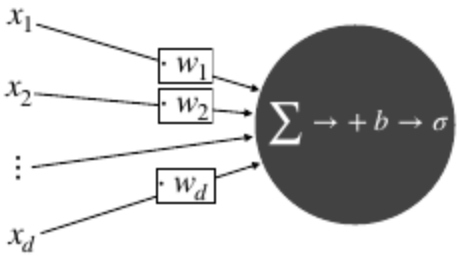
**Mercer's condition**: if a **kernel function $\kappa$ is symmetric** ($\forall x, x'\ \kappa(x, x') = \kappa(x', x)$) and the **kernel matrix is positive semidefinite for all possible input sets** ($\forall N \geq 0, \forall (x_n)_{n=1}^N,\ \mathrm{K} = (\kappa(x_i, x_j))_{i,j=1}^N \succcurlyeq 0$), then the **associated feature map $\phi$ exists**

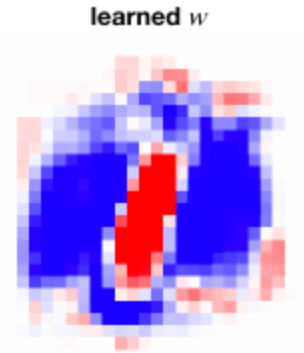
# 12 - Neural Networks

**Prediction with a neural network (NN)**: $y = f_{NN}(x) = f(x)^T w$ with $f_{NN}$ the **function implemented by the NN parameters**, $f$ **transforms the input in a good efficient representation** (**first** layer, **learned by the model**) and $w$ **performs a linear prediction** (**last** layer)

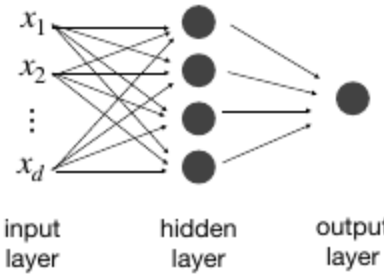
***Classical machine learning*** *is based on relatively* ***simple models on top of features handcrafted*** *by domain experts (works well if the feature are good).* ***Deep learning*** *is based on* ***large neural networks that learn features directly from data*** *(can be viewed as a* ***complicated feature extractor + linear prediction****), requires* ***large amounts of data and compute to train*** *(****quality scales*** *with that)*

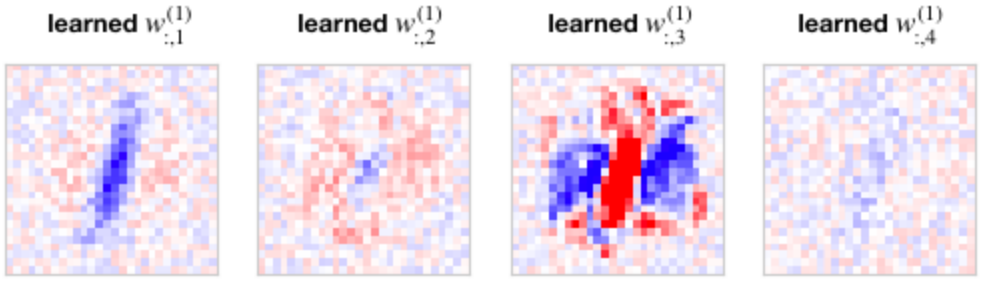
**Logistic Regression for Neural Networks**: $f_{LR}(x) = p(1|x) = \sigma(x^T w + b) = \sigma(\sum_{i=1}^d x_i w_i + b),\ \ \sigma(z) = (1 + e^{-z})^{-1}$ sigmoid, **learning $w$ and applying elementwise** to $x$ (very restrictive). We can now classify points with $1\ if\ \sum_{i=1}^d x_i w_i \geq -b$





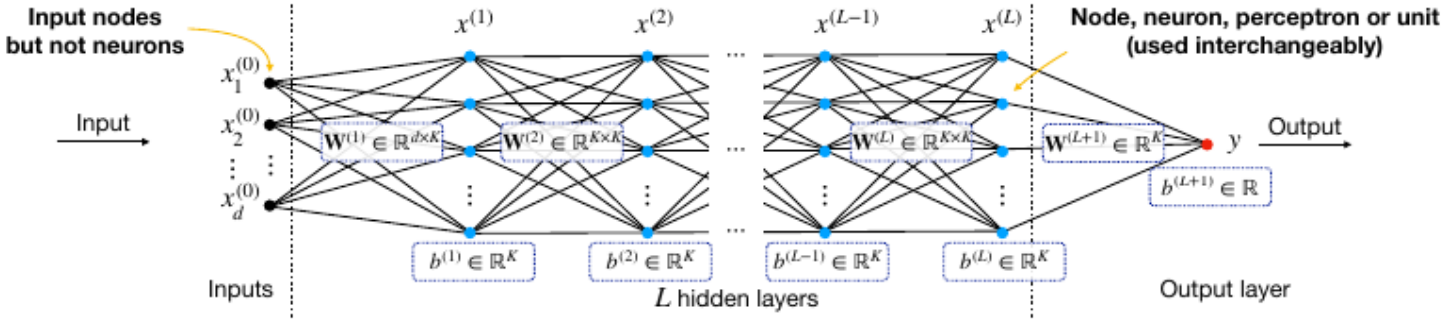

**Two-layer neural network**: aka **two-layer perceptron**, **stack logistic regression neurons/units** $\phi(x_j) = \sigma(\sum_{i=1}^{d} x_i w_{i,j}^{(1)} + b_j)$ where $j$ is a **hidden neuron/unit** in the hidden layer (**each learns a different pattern** that **might not be interpretable**), $w_{i,j}^{(l)}$ represents the weight of the edge going from node $i$ in layer $l-1$ to node $j$ in layer $l$ and $b_j^{(l)}$ is the bias term of node $j$ in layer $l$. $\phi$ is called **activation function** and it's **non-linear so that we can represent non-linear functions** with the model. **Aggregated with a linear function** $\phi(x)^T w^{(2)}$ (**very flexible**)




**Multi-layer neural network**: $L$ hidden layers with $K$ neurons each + output layer with single node. It **learns the weight matrices** $W^{(l)}$ **and bias vectors** $b^{(l)}$ for $1 \leq l \leq L+1$ ($O(K^2 L)$ params). Since you start with $x^{(0)} \in \mathbb{R}^d$ and end up with $W^{(L+1)} \in \mathbb{R}^K$ a **Neural Network as a feature extractor**. Outputs of hidden layer $l$ given by $x^{(l)} = f^{(l)}(x^{(l-1)}) = \phi((W^{(l)})^T x^{(l-1)} + b^{(l)})$. The **final linear predictions calculated during training time and inference** are given by $h(x) = f(x)^T w^{(L+1)} + b^{(L+1)}$, the **final model loss and output depends on the problem**, for **regression** it's $l(y, h(x)) = (h(x) - y)^2$ and $output = h(x)$, for **binary classification** it's $l(y, h(x)) = log(1 + e^{-yh(x)})$ and $output = sign(h(x))$ and for **multi-class classification** it's $l(y, h(x)) = -log \frac{e^{h(x)_y}}{\sum_{i=1}^{K} e^{h(x)_i}}$ (**Cross-Entropy Loss**, forces the model to make the **score for the right class** $h(x)_y$

**as big as possible to compensate for the "penalty" introduced by the denominator** (sum of the scores for the wrong classes)) and $output = argmax_{c \in [1,\ldots,K]}\ h(x)_c$

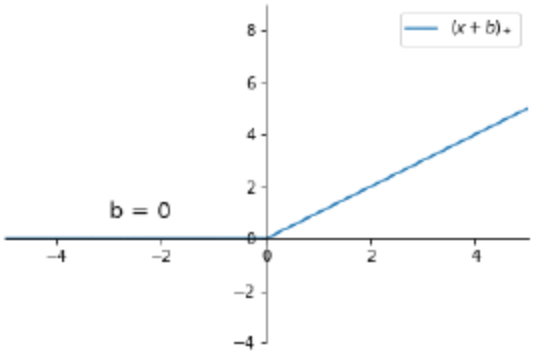



**Fully connected Neural Network**: **each node in one layer is connected to every node in the previous layer**

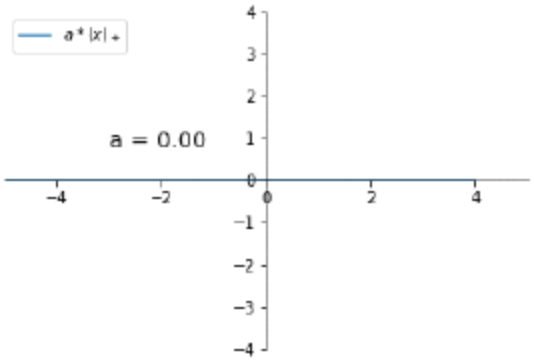
**Popular activation functions**:

- **Sigmoid**: $\phi(x) = \sigma(x) = \frac{1}{1+e^{-x}}$
- **ReLU**: $\phi(x) = (x)_+ = max\{0, x\}$, **popular**

  Role of the bias and weight in ReLU function $(ax+b)_+ = \max\{0, ax+b\}$:

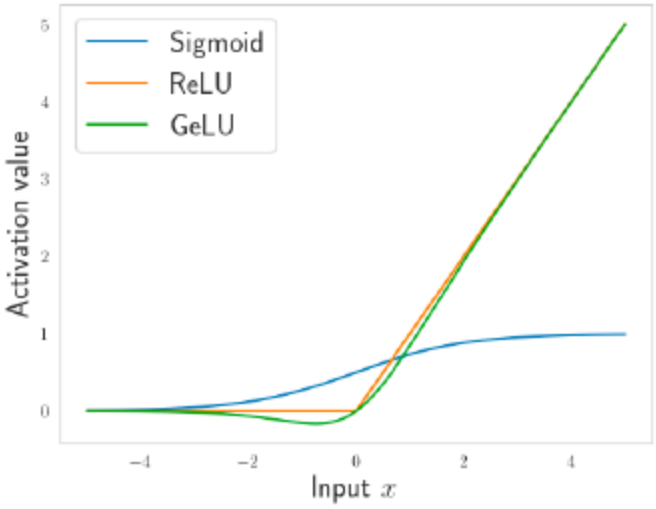



**The bias $b$ determines the position of the kink**



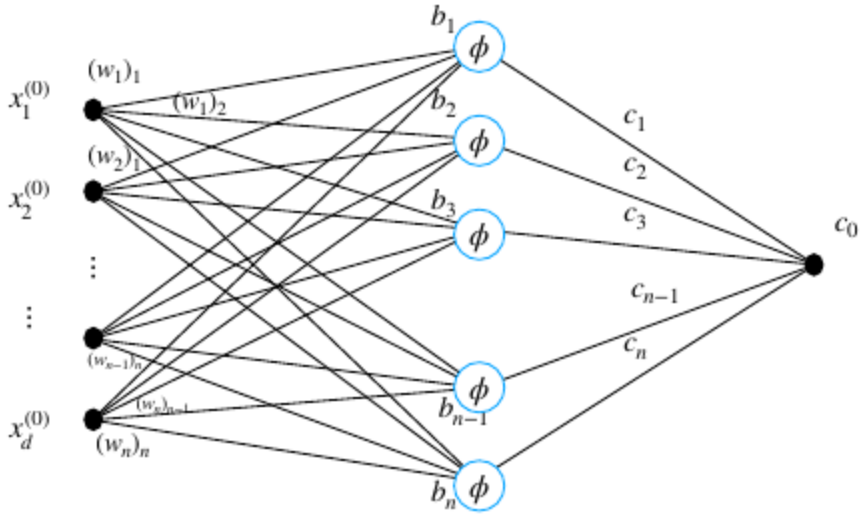

**The weight $a$ determines the slope**

- **GELU**: $\phi(x) = x\Phi(x) \approx x\sigma(1.702x)$, **popular**

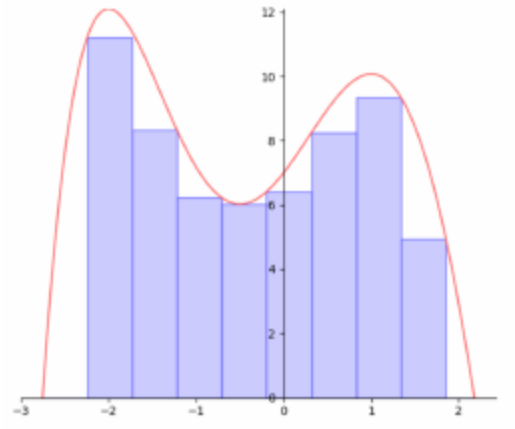


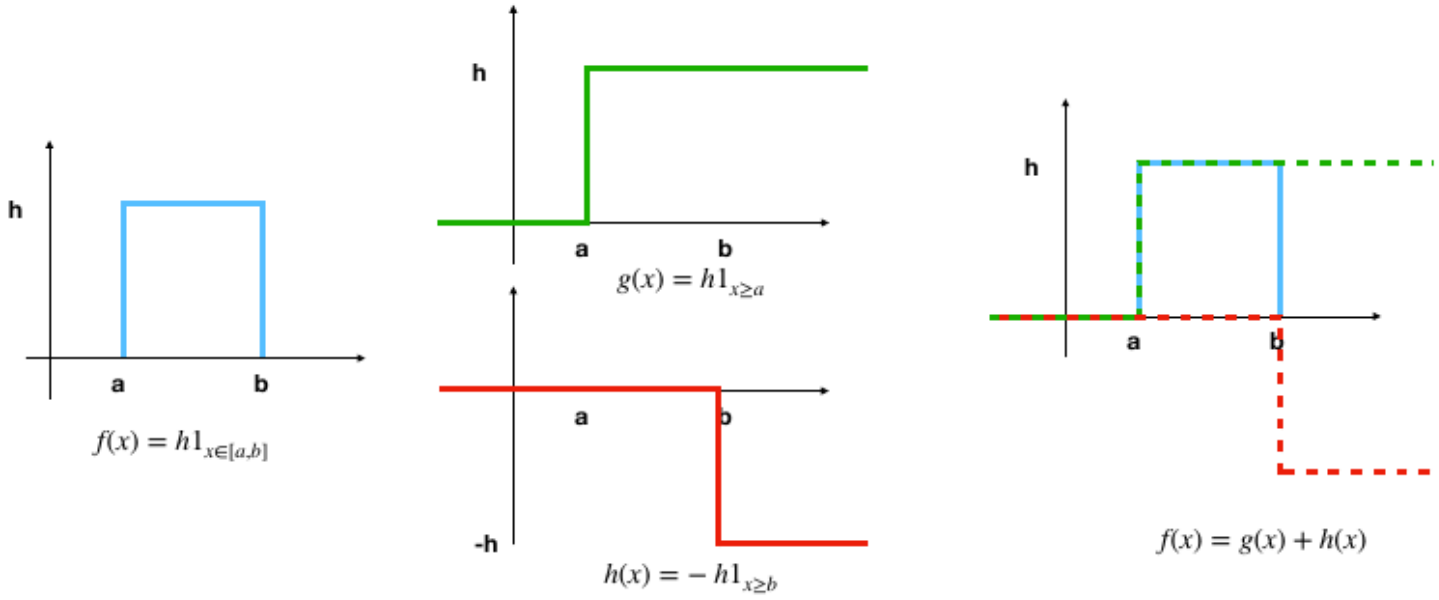
**Barron's universal approximation theorem**: $f : \mathbb{R}^d \to \mathbb{R},\ \hat{f}(\omega) = \int_{\mathbb{R}^d} f(x) e^{-i\omega^T x}$ its **Fourier transform and smoothness assumption** $\int_{\mathbb{R}^d} |\omega||\hat{f}(\omega)|\, d\omega \leq C$, we have $f_n(x) = \sum_{j=1}^{n} c_j \phi(x^T w_j + b_j) + c_0 \ \ s.t.\ \int_{|x|\leq r} (f(x) - f_n(x))^2\, dx \leq \frac{(2Cr)^2}{n}$. All sufficiently **smooth functions can be approximated by a one-hidden-layer Neural Network**, the more the neurons, the smoother the function or the smaller the domain ($r$) the smaller the error, **works with any "sigmoid-like" activation function**. Thanks to this **Neural Networks have very high representational power**

$$f_n(x) = \sum_{j=1}^{n} c_j \phi(x^\top w_j + b_j) + c_0 = c^\top \phi(W^\top x + b) + c_0$$

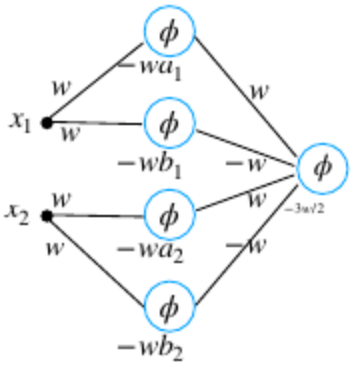


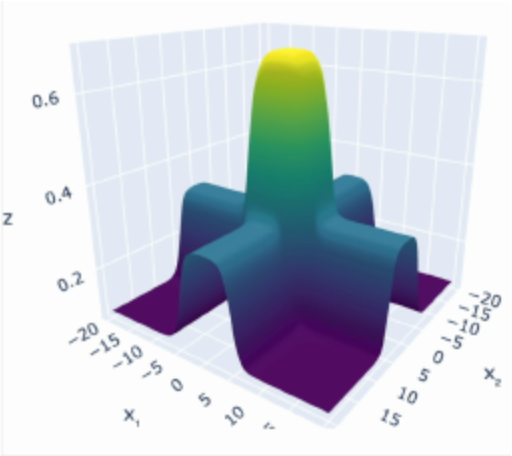

**L1-approximation in Barron's universal approximation theorem**: $\int_{|x|\leq r} |f(x) - f_n(x)|\, dx \leq$ something small. **Explains how the sum of many simple step functions** (**represented by nodes in a hidden layer**) **can approximate a complex function we are learning. Sum step functions in rectangle** functions, sum **rectangle functions to approximate a smooth function like Riemann Integrals do**. To approximate with $k$ Riemann rectangles you need $2k$ nodes in hidden layers

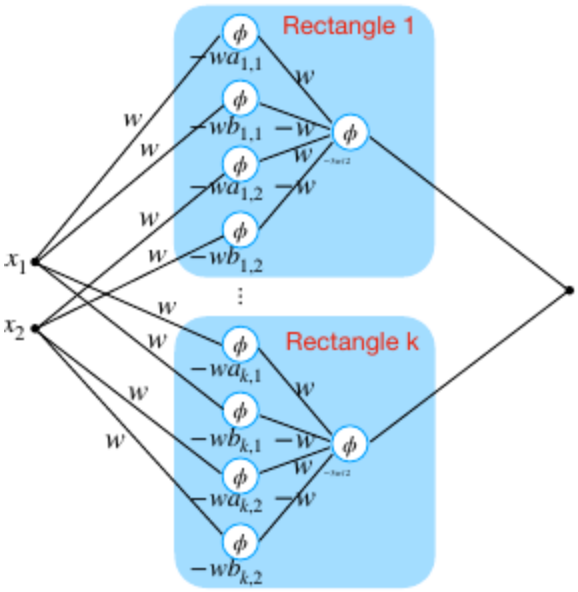





**Approximation in 2D**: make **2D rectangle functions with 4 sigmoids** $(x_1, x_2) \to \phi(w(x_1 - a_1)) - \phi(w(x_1 - b_1)) + \phi(w(x_2 - a_2)) - \phi(w(x_2 - b_2))$ + **one last sigmoid with high** $w$ **and appropriate** $b$ (e.g. $3w/2$) **that approximates** $1_{y\geq c},\ c \in (1, 2]$ scaling what would have been a function in $[0, 2]$ to $[0, 1]$



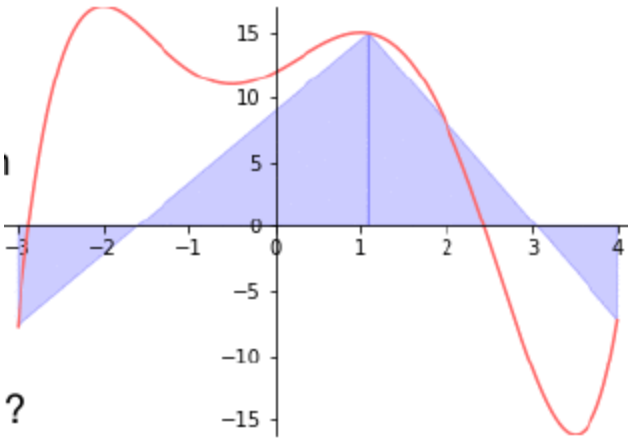






***L2-norm approximation measures the average deviation from the true function and the approximate**, **low deviation points compensate for higher ones**, not representing properly what's happening. **L∞-norm measures the maximum error at any point** and ensures **small error** accross the restricted domain **with precise control***

**Piecewise linear (PWL) function**: $q(x) = \sum_{i=1}^{m}(a_i x + b_i)1_{r_{i-1} \leq x < r_i}, \;\; a_i r_i + b_i = a_{i+1} r_i + b_{i+1}$ . A **linear combination of ReLUs $\sum_{i=1}^{m} \tilde{a}_i (x - \tilde{b}_i)_+$ is a PWL function so any PWL function can be rewritten as $q(x) = \tilde{a}_1 x + \tilde{b}_1 + \sum_{i=1}^{m} \tilde{a}_i (x - \tilde{b}_i)_+, \;\; \tilde{a}_1 = a_1, \tilde{b}_1 = b_1, a_i = \sum_{j=1}^{i} \tilde{a}_i, \tilde{b}_i = r_{i-1}$, thanks to this any PWL function can be implemented as a one hidden layer Neural Network with ReLU activation functions** ($\tilde{a}_1 x + \tilde{b}_1$ represents the output node)

**L∞ approximation with PWL functions**: $f$ continuous in $[c, d]$, $\forall \epsilon > 0, \; \exists \, q \;\; s.t \;\; sup_{x \in [c,d]} |f(x) - q(x)| \leq \epsilon$




**Training loss for a Neural Network**: $\mathcal{L}(f) = \frac{1}{2N} \sum_{n=1}^{N} (y_n - f(x_n))^2$ where $f$ is represented by a Neural Network with weights $(w_{i,j}^{(l)})$ and biases $(b_i^{(l)})$, so the task is to $min_{w_{i,j}^{(l)}, b_i^{(l)}} \mathcal{L}(f)$,

**regularization can be added** to avoid overfitting, it's a **non-convex problem** because you are **optimizing the weights and biases that make a complex non-linear function**

*Neural Networks can be **trained on the whole dataset** (vanilla GD), **on batches of the dataset** (batch SGD) or **on a sample at a time** (SGD)*

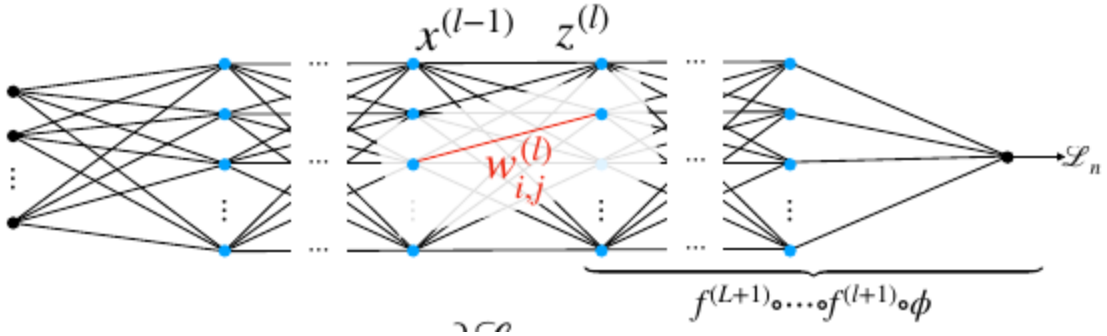
**Training Neural Networks with SGD**: uniformly sample $n$, compute the grandient of $\mathcal{L}_n = \frac{1}{2}(y_n - f(x_n))^2$ to update $(w_{i,j}^{(l)})_{t+1} = (w_{i,j}^{(l)})_t - \gamma \frac{\partial \mathcal{L}_n}{\partial w_{i,j}^{(l)}}$ and $(b_i^{(l)})_{t+1} = (b_i^{(l)})_t - \gamma \frac{\partial \mathcal{L}_n}{\partial b_i^{(l)}}$, to **prevent overfitting** you apply **step size schedule**, **mini-batch**, **momentum/adam**

**Backpropagation algorithm**: once you trained your model and calculated the final loss you want to update the weight and biases of each node in the Neural Network, and that would mean calculating the gradient at each weight and bias ****($O(K^2L)$ **gradients**, **too much**). With weight matrices $W^{(l)}\ s.t.\ W_{i,j}^{(l)} = w_{i,j}^{(l)}$ of size $W^{(1)} \in \mathbb{R}^{d \times K}, W^{(l)} \in \mathbb{R}^{K \times K}\ \forall 2 \leq l \leq L, W^{(L+1)} \in \mathbb{R}^K$ and bias vectors $b^{(l)}$ such that the i-th component is $b_i^{(l)}$ of size $b^{(l)} \in \mathbb{R}^k\ \forall 1 \leq l \leq L, b^{(L+1)} \in \mathbb{R}$. At each layer we have $x^{(1)} = f^{(1)}(x^{(0)}) = \phi((W^{(1)})^T x^{(0)} + b^{(1)}), \dots, x^{(l)} = f^{(l)}(x^{(l-1)}) = \phi((W^{(l)})^T x^{(l-1)} + b^{(l)}), \dots, y = f^{(L+1)}(x^{(L)}) = (W^{(L+1)})^T x^{(L)} + b^{(L+1)}$, so the **overall function** is $y = f(x^{(0)}),\ f = f^{(L+1)} \circ f^{(L)} \circ \dots \circ f^{(l)} \circ \dots \circ f^{(2)} \circ f^{(1)}$

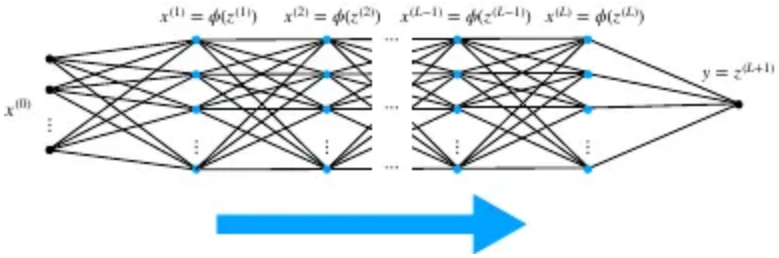
**Cost function**: $\mathcal{L} = \frac{1}{2N} \sum_{n=1}^{N} (y_n - f^{(L+1)} \circ \dots \circ f^{(2)} \circ f^{(1)}(x_n))^2$ **function of all weight matrices and bias vectors**. The **goal is to compute the individual loss of nodes over all** $(i, j, l)$ and use it to **update weights and biases**, we need the gradient $\frac{\partial \mathcal{L}_n}{\partial w_{i,j}^{(l)}} = \frac{\partial \mathcal{L}_n}{\partial w_{i,j}^{(l)}} \frac{\partial z_j^{(l)}}{\partial z_j^{(l)}} = \frac{\partial \mathcal{L}_n}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial w_{i,j}^{(l)}} = \frac{\partial \mathcal{L}_n}{\partial z_j^{(l)}} x_i^{(l-1)}$ where $z^{(l)} = (W^{(l)})^T x^{(l-1)} + b^{(l)}$ pre-activation value, so we just need to compute $x^{(l)}, z^{(l)}, \frac{\partial \mathcal{L}_n}{\partial z_j^{(l)}}$ (**error signal that says how much the loss changes based the pre-activation value of node** $j$) and **reuse them for calculating the gradient from any node to** $j$ ($\frac{\partial \mathcal{L}_n}{\partial w_{k,j}^{(l)}}\ \forall k \in [0, K]$).

So we are basically adjusting by how much the loss changes based on the pre-activation value of the node multiplied by the input received by the node

$$\mathscr{L}_n = \frac{1}{2}\Big(y_n - f^{(L+1)} \circ \dots \circ f^{(l+1)} \circ \phi\Big(\underbrace{(\mathbf{W}^{(l)})^\top x^{(l-1)} + b^{(l)}}_{z^{(l)}}\Big)\Big)^2$$



**Forward pass**: during it we can compute both $x^{(l)}$, $z^{(l)}$ and save them to memory, $O(K^2L)$



**Backward pass**: define $\delta_j^{(l)} = \frac{\partial \mathcal{L}_n}{\partial z_j^{(l)}} = \sum_k \frac{\partial \mathcal{L}_n}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = \sum_k \delta_k^{(l+1)} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}}$, this last fraction is how much does the pre-activation value of the next layer changes based on the pre-activation value of the node we are updating. Using $z_k^{(l+1)} = \sum_{i=1}^{K} w_{i,k}^{(l+1)} x_i^{(l)} + b_k^{(l+1)} = \sum_{i=1}^{K} w_{i,k}^{(l+1)} \phi(z_i^{(l)}) + b_k^{(l+1)}$ we obtain $\frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = \phi'(z_j^{(l)}) w_{j,k}^{(l+1)}$, thus $\delta_j^{(l)} = \sum_k \delta_k^{(l+1)} \phi'(z_j^{(l)}) w_{j,k}^{(l+1)}$ which is **fully dependant on the weight and gradient of the following layer**. Can also be written in vector form $\delta^{(l)} = (W^{(l+1)} \delta^{(l+1)}) \odot \phi'(z^{(l)})$ (⊙: Hadamard product, pointwise multiplication of vector). At the **last layer we initialize the error signal** to $\delta^{(L+1)} = \frac{\partial}{\partial z^{(L+1)}} \frac{1}{2}(y_n - z^{(L+1)})^2 = z^{(L+1)} - y_n$. Do a backward pass and compute all error signals $O(K^2L)$ to update all weights ($\frac{\partial \mathcal{L}_n}{\partial w_{i,j}^{(l)}} = \delta_j^{(l)} x_i^{(l-1)}$) and biases (same logic as weights, we end up with $\frac{\partial \mathcal{L}_n}{\partial b_j^{(l)}} = \delta_j^{(l)}$ )

# Backpropagation algorithm

Forward pass:

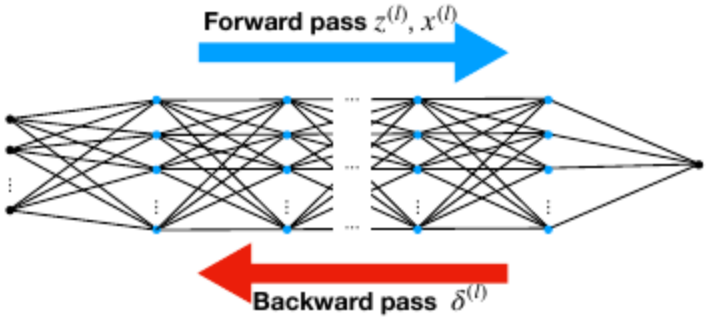
$$x^{(0)} = x_n \in \mathbb{R}^d$$
$$z^{(l)} = (\mathbf{W}^{(l)})^\top x^{(l-1)} + b^{(l)}$$
$$x^{(l)} = \phi(z^{(l)})$$

Backward pass:

$$\delta^{(L+1)} = z^{(L+1)} - y_n$$
$$\delta^{(l)} = (\mathbf{W}^{(l+1)} \delta^{(l+1)}) \odot \phi'(z^{(l)})$$

Compute the derivatives:

$$\frac{\partial \mathcal{L}_n}{\partial w_{i,j}^{(l)}} = \delta_j^{(l)} x_i^{(l-1)}$$
$$\frac{\partial \mathcal{L}_n}{\partial b_j^{(l)}} = \delta_j^{(l)}$$



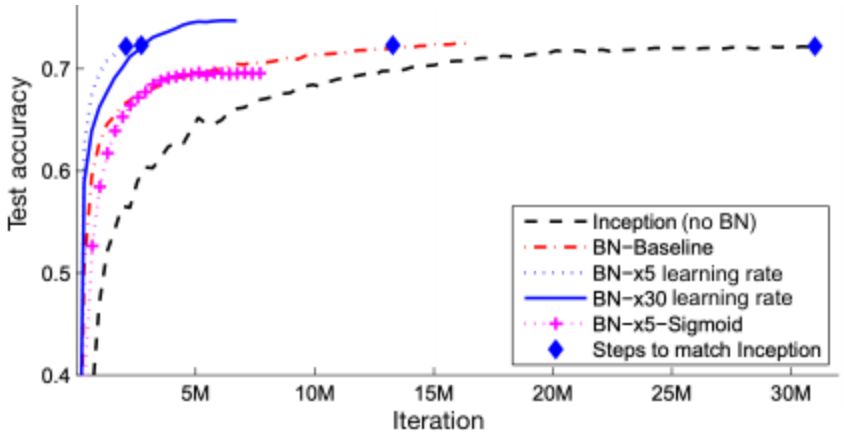

Overall Complexity: $O(K^2L)$

**Parameter Initialization**: since **gradient descent** uses the **gradient of a function made of the composition of many functions**, we are applying the **chain rule** which means that it's a **bunch of**

**terms being multiplied with one another**, for this reason the **gradient can grow or shrink incontrollably**. The remedy is choosing **suitable activation functions**, **weight normalization**, **initializing weights properly**, **implementing skip connections** and **controlling the layerwise variance of neurons** (**He initialization**)

**Variance-Preserving Initialization for ReLU**: $z^{(l)} \sim N(0, I_K)$ pre-activations at layer $l$ with variance 1, $w_i^{(l+1)} \sim N(0, \sigma^2 I_K)$ i-th weight vector at layer $l+1$, $z_i^{(l+1)} = ReLU(z^{(l)})^\top w_i^{(l+1)}$ i-th pre-activation at layer $l+1$. If we set $\sigma = \sqrt{2/K} \rightarrow Var[z_i^{(l+1)}] = 1$ (like $z^{(l)}$ )

**Batch normalization**: **applicable to batch SGD**, consider a batch $B = (x_1, \dots, x_M)$ and denote $z_n^{(l)}$ the layer's pre-activation input corresponding to observation $x_n$, **first normalize each layer's input** $\bar{z}_n^{(l)} = \frac{z_n^{(l)} - \mu_B^{(l)}}{\sqrt{(\sigma_B^{(l)})^2 + \epsilon}}$ where $\mu_B^{(l)} = \frac{1}{M}\sum_{n=1}^{M} z_n^{(l)}, (\sigma_B^{(l)})^2 = \frac{1}{M}\sum_{n=1}^{M}(z_n^{(l)} - \mu_B^{(l)})^2, \epsilon \in \mathbb{R}_{\geq 0}$, **then introduce learnable parameters** $\gamma^{(l)} \in \mathbb{R}^K$ (**scale**) and $\beta^{(l)} \in \mathbb{R}^K$ (**shift**) so we **don't end up with a linear model** and we **don't lose all negative data** after applying the activation function (e.g. with $\mathcal{N}(0,1)$ + ReLU, half of the data would be gone because negative and ReLU is basically linear in [-1,1] where most data would be), also the **neural network during training can potentially recover the original activations** ($\hat{z}_n^{(l)} = \gamma^{(l)} \odot \bar{z}_n^{(l)} + \beta^{(l)}$). There is **no need to include the bias before Batch Normalization** (**scale-invariance**, output invariant to activation-wise affine scaling of $z_n^{(l)}$). Since in **inference samples arrive one at a time use fixed mean and variance** (estimated during training, or exponential moving average). Requires sufficiently large datasets to get good estimates of mean and variance in a batch




**Source**: Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift (ICML 2015)

BatchNorm leads to **much faster convergence**

BatchNorm allows to use **much larger learning rates** (up to $30\times$)

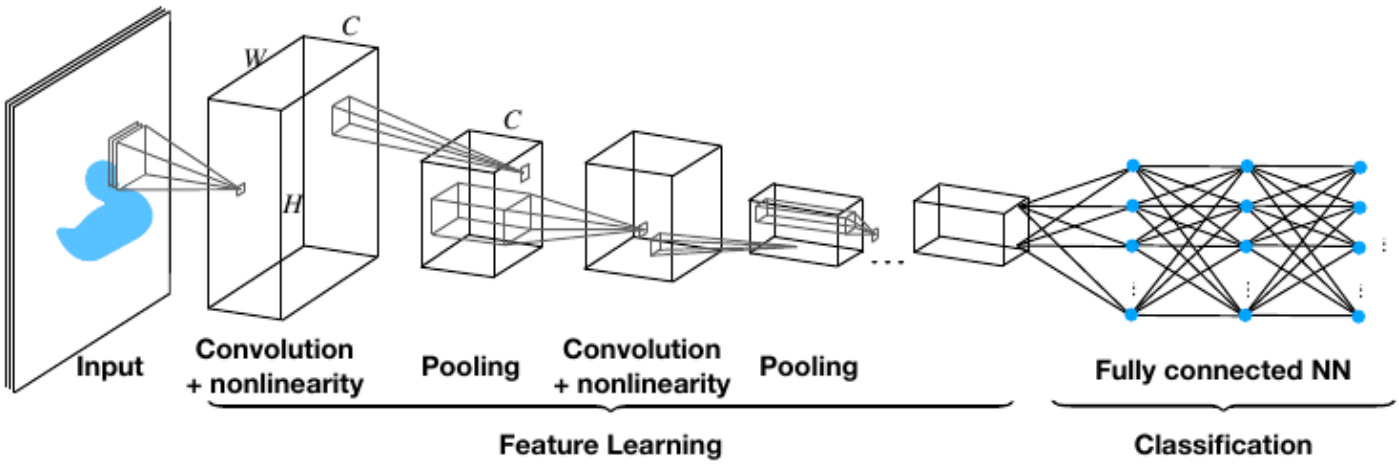
**Layer normalization**: first **normalize each layer's input using its mean and variance over the features** of a specific input $\bar{z}_n^{(l)} = \frac{z_n^{(l)} - \mu_n^{(l)} 1_K}{\sqrt{(\sigma_n^{(l)})^2 + \epsilon}}$, where $\mu_n^{(l)} = \frac{1}{K}\sum_{k=1}^{K} z_n^{(l)}(k), (\sigma_n^{(l)})^2 =$

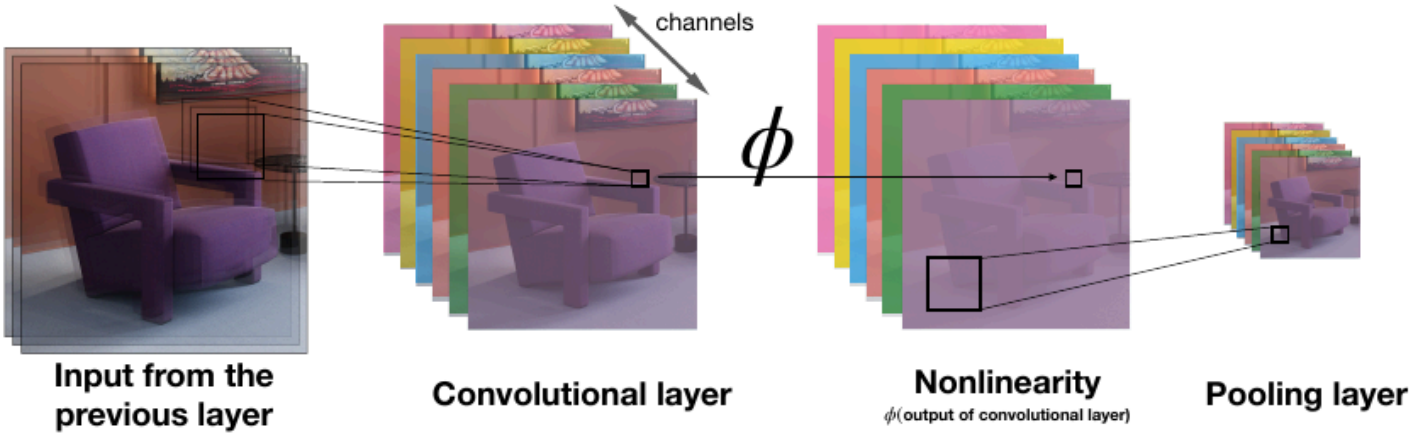
$\frac{1}{K}\sum_{k=1}^{K}(z_n^{(l)}(k)-\mu_n^{(l)})^2, \epsilon \in \mathbb{R}_{\geq 0}$, then **introduce learnable parameters** $\gamma^{(l)}, \beta^{(l)} \in \mathbb{R}^K$. It's **independent of each sample** (same for training and inference), **widely used in transformers**.

***Normalization layers*** *are* ***used almost always****, often after every convolutional layer, they* ***stabilize activation magnitudes reducing initialization impact****, stabilizes and* ***speeds up training*** *(by allowing larger learning rates),* ***regularization effect*** *in Batch Normalization*

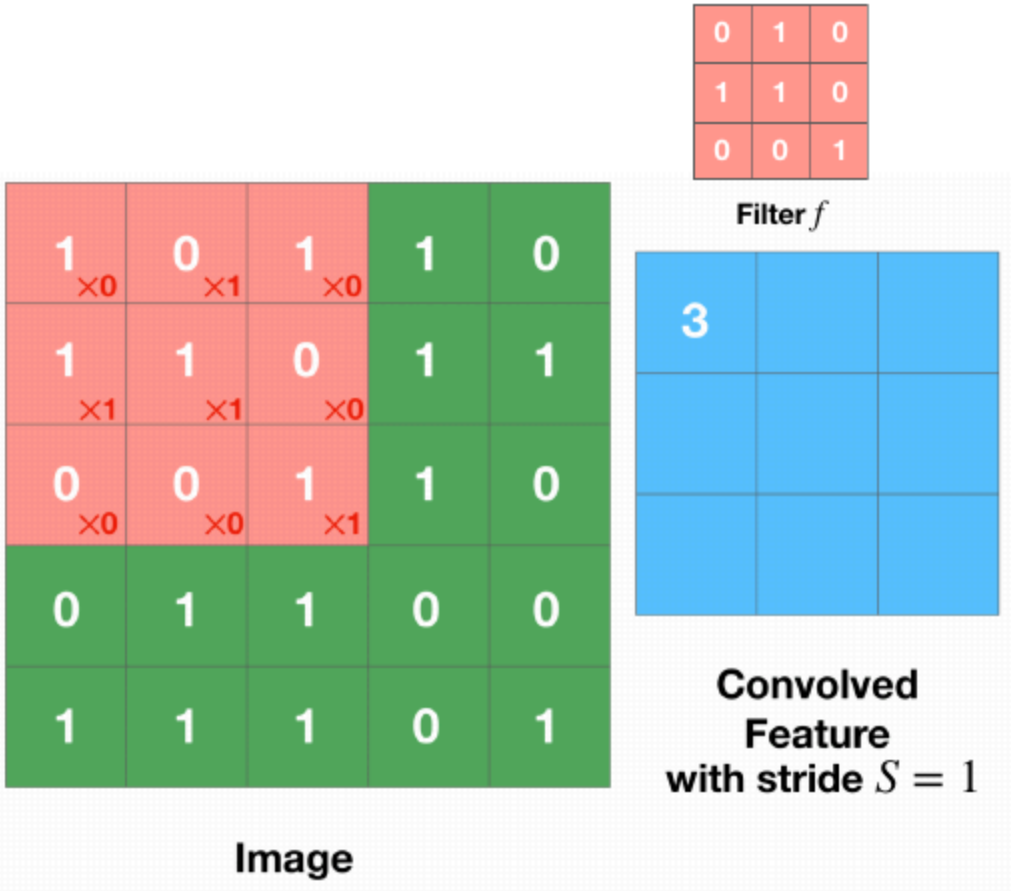
# 13 - Convolutional Neural Networks

**Convolutional Neural Network (CNN)**: solves the problem that **fully connected neural networks have a lot of parameters** ($O(K^2L)$) and **do not capture spatial dependencies**, uses **pooling layers to perform spacial downsampling** (usually **halves the dimension** of the input) **after** which a **convolution layer** is placed to **apply weights in the form of filters used in the convolutions** (**many different filters are used**). **At the end** there is a **fully connected neural network that does classification** based on the features extracted during convolutions. **After each convolutional layer we add non-linearity** (e.g. ReLU)

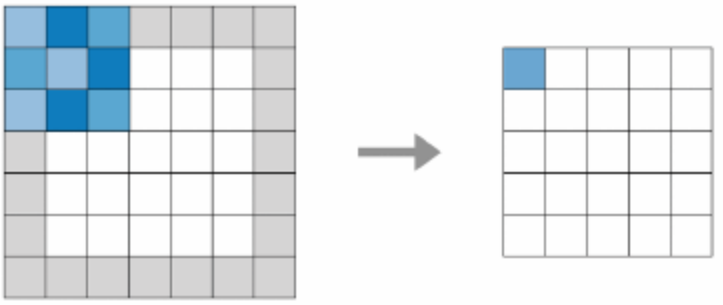


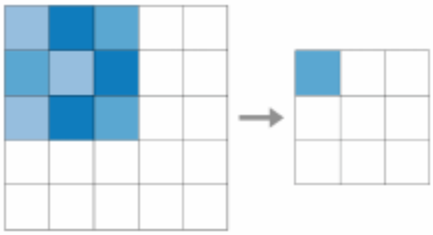
**Convolution**: for a **filter $f$ representing the weights** of size $K$ and stride $S$, $x^{(1)}_{n,m} = \sum_{k,l=0}^{K-1} f_{k,l} x^{(0)}_{nS+k,mS+l}$, we use the **same weights at every position** (**weight sharing**) which are also **smaller than a fully connected layer's weights**. **Local connectivity** since $x^{(1)}_{n,m}$ depends on the values of $x^{(0)}$ close to $(nS, mS)$ for a small $K$, **translation equivariance** (shifted input ⇒ shifted output). To **learn better about the borders** of the image you **add zero padding** around it (enough to cover the filter $K \times K$), this also **makes it so the output of the convolution has the same size as the original image** (otherwise it would be smaller). In case of **Multi-Channel Convolution** (using colors) you have a **filter for each channel**, you **sum the output of each channel and add a bias to get a single output value**. The **hyperparameters are size $K$, stride $S$ and the padding**. **All Convolution Layers could have Fully Connected Layers equivalents** that use as weight a large matrix that is mostly zero (**inefficient**) for local connectivity and there are many replicated blocks in the weights (weight sharing), the **opposite is also true** (you would have a **filter as big as the input**, therefore make a single pass and get a single output)



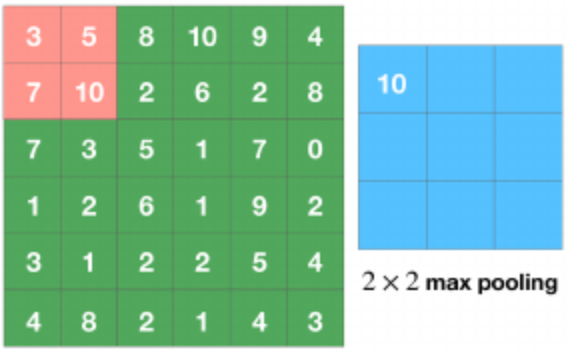
## Zero padding:

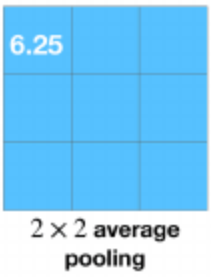


## Valid padding:



**Pooling**: **downsampling** that **reduces the spatial dimentions** of the convolved feature, uses a **kernel of size $K$, with stride $S$ which are hyperparameters together with the type of pooling** (**max** or **average**). No learnable parameters



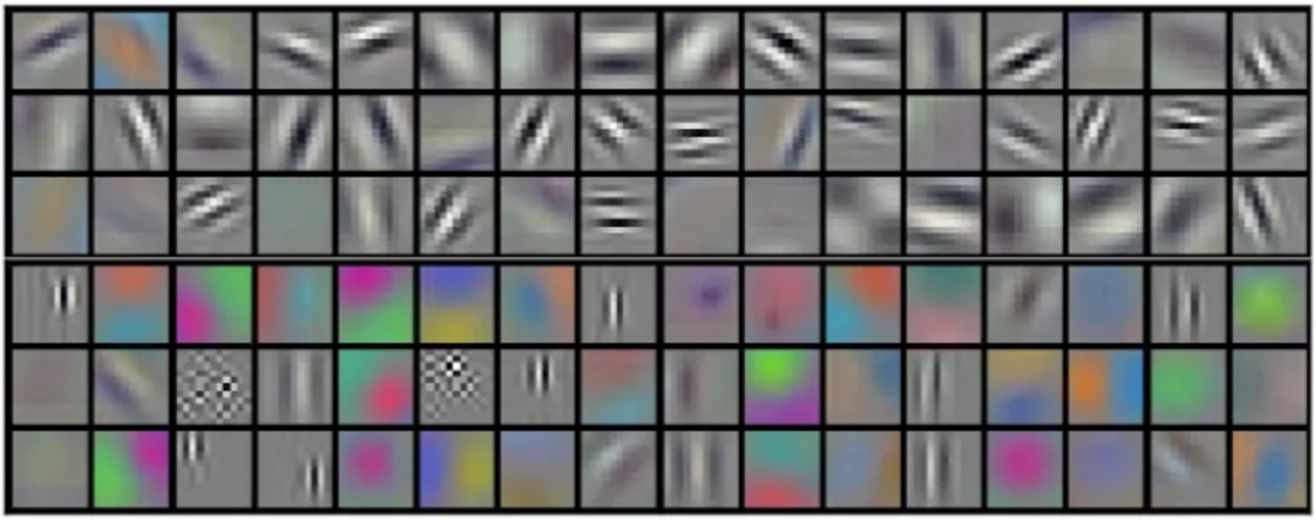




**Receptive field**: **area of input that affects a given neuron increases with depth**, **first layers extract low-level local features** (e.g. edges, colors), **subsequent layers extract high-level features**

(e.g. objects)

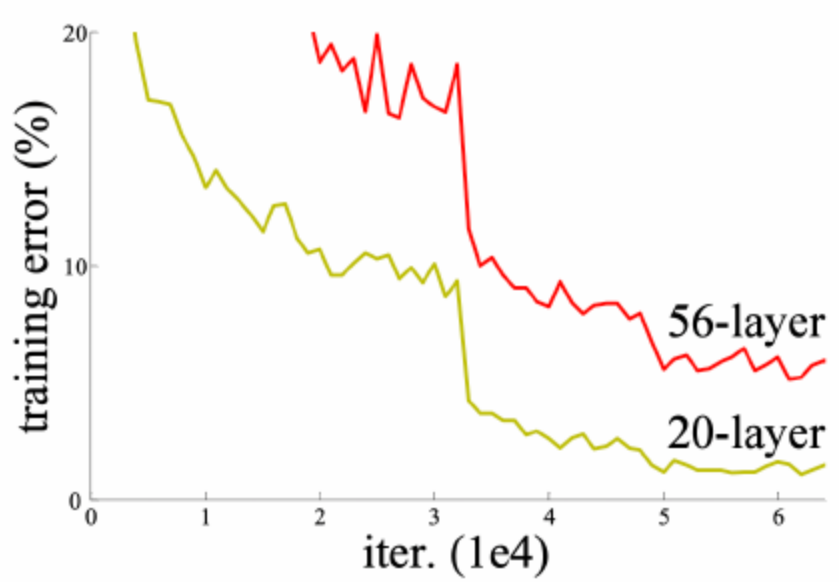
**Backpropagation with weight sharing**: do **normal backpropagation**, **sum gradients of all edges that share the same weight** (each gradient will apply a different update) and **use that**

**Filters in the first layer can be interpretable, edge and color detectors typically emerge when using large datasets**:

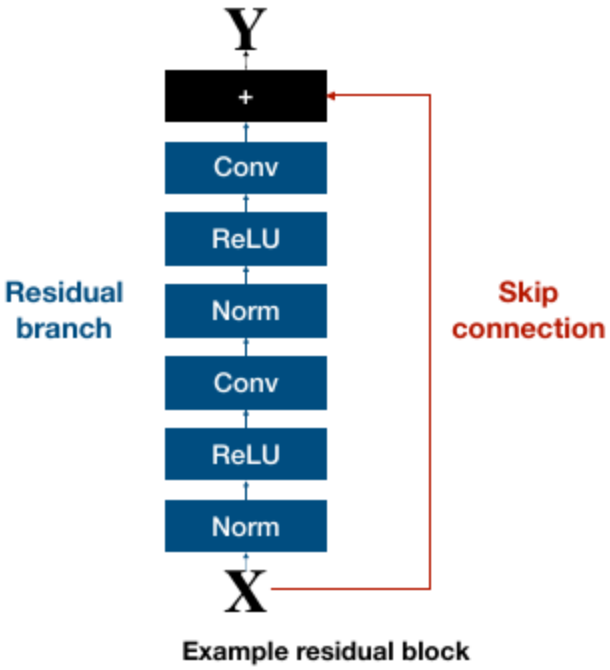


**Individual activations can be interpretable**:


**Gradient vanishing**: **gradients tend to go to 0 as deep neural networks grow in depth**, **training error increases as more layers are added** (**not overfitting**). This is because during backpropagation, you are multiplying the error signal over and over, example: **Layer 10 Gradient** = $(Error) \times 0.25$; **Layer 9 Gradient** = $(Layer10Gradient) \times 0.25 = (Error) \times 0.0625$; finally **Layer 1 Gradient** = $(Error) \times 0.25^{10} = (Error) \times 0.00000095$



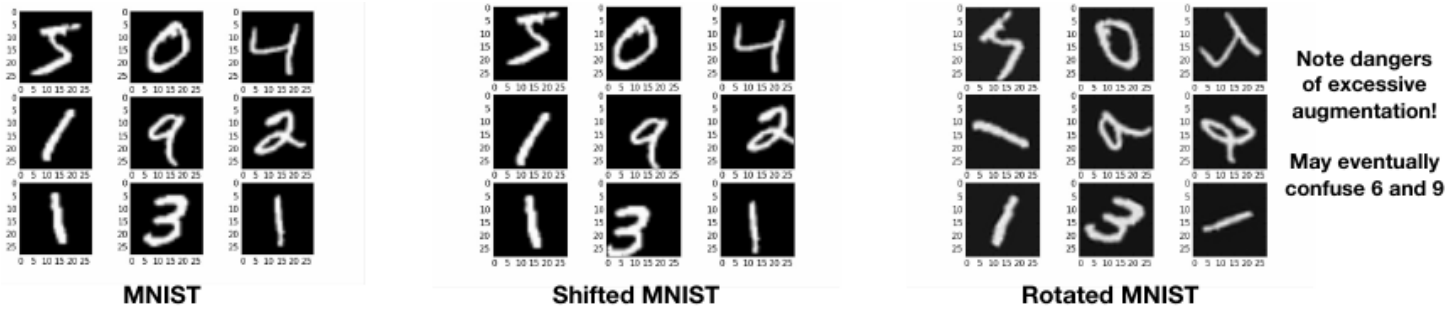

**Residual Networks**: used in **CNNs and Transformers**, **fixes gradient vanishing** by adding a **skip connection around some layers so have** $Y = R(X) + X$ ($R(X)$ **residual branch**) **instead of** $Y = F(X)$, now the **error signal/gradient can directly update the weights of the first layers** (without going through all those multiplications). This gives the possibility to **easily skip modifying the input** $X$ **in a specific layer by setting the weights of** $R(X)$ **to** $0$

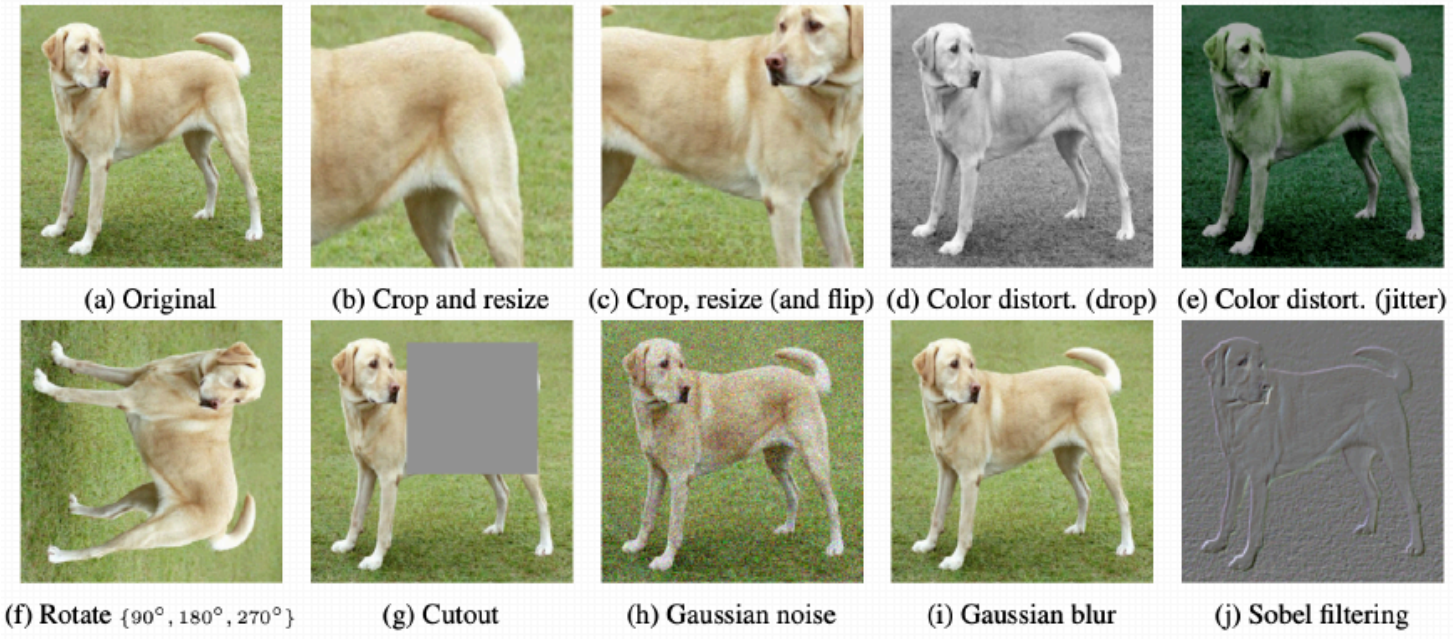



**Weight decay**: **L2-regularization** that favours **small weights which generalize** better (thus **not applied to bias terms because it does not impact model complexity**), $min\ \mathcal{L} + \frac{\lambda}{2} \sum_l \|W^{(l)}\|_F^2$, a **weight update** becomes $(w_{i,j}^{(l)})_{t+1} = (w_{i,j}^{(l)})_t - \eta \nabla \mathcal{L} - \eta \lambda (w_{i,j}^{(l)})_t = (1 - \eta\lambda)(w_{i,j}^{(l)})_t - \eta \nabla \mathcal{L}$ where **weight decay** = $(1 - \eta\lambda)$, **when the gradient vanishes the weights also go to** $0$ **very quickly**. When using **Batch Normalization there is no direct regularization effect from Weight**

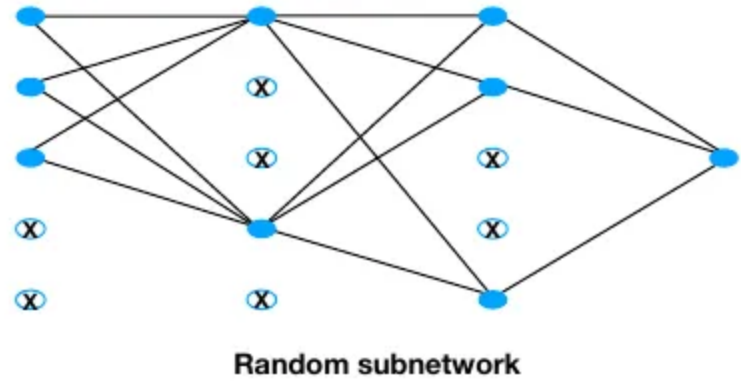
**Decay** because it's **scale-invariant** ($BN(WX) = BN(\alpha WX)\ \forall \alpha \in \mathbb{R}_{>0}$), so **we need other regularization techniques**

**Data Augmentation**: generate **new data from your data while preserving labels** to make the neural network **generalize better** and **have more data to train on** (**regularization**), transformation $\tau$ : $\mathbb{R}^d \to \mathbb{R}^d,\ \ S = S_{train} \cup \{(\tau(x_i), y_i)\}_{i=1}^n$ with $y_x = y_{\tau(x)}$

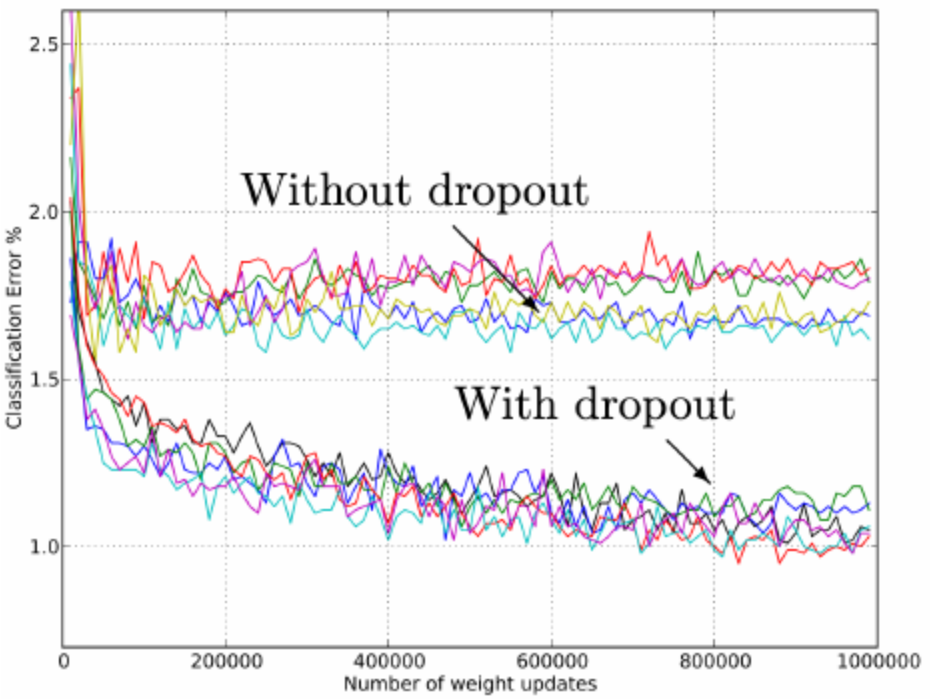



**Dropout**: at each **training** step, **retain with probability $p^{(l)}$ each node in layer $(l)$** and **run SGD on the new random subnetwork**, **helps the model generalize** (**regularization**, same idea as RandomForests being trained on subsamples of data). During **testing use all nodes but scale them by $p^{(l)}$** so that the expected output is the same as the training set (**or do weight rescaling during training by $1/p^{(l)}$**), Requires **more iterations to converge since it increases stochastic noise**



Random subnetwork




# 14 - Transformers

***Instead of the classical single-datapoint inputs and 1-dimensional output*** *we may want to inject and/or produce richer objects*

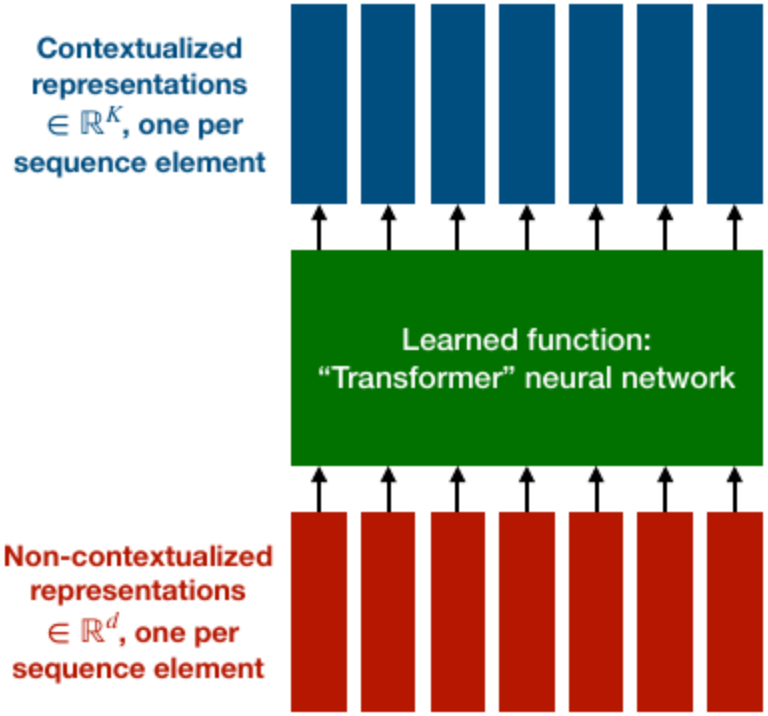
**Tagging**: for **each element in an input sequence/set, make one prediction** (not independent of predictions for other elements, e.g. **part-of-speech tagging**)

**Sequence-to-sequence transduction**: machine translation, question answering, summarization, image-to-text, etc…

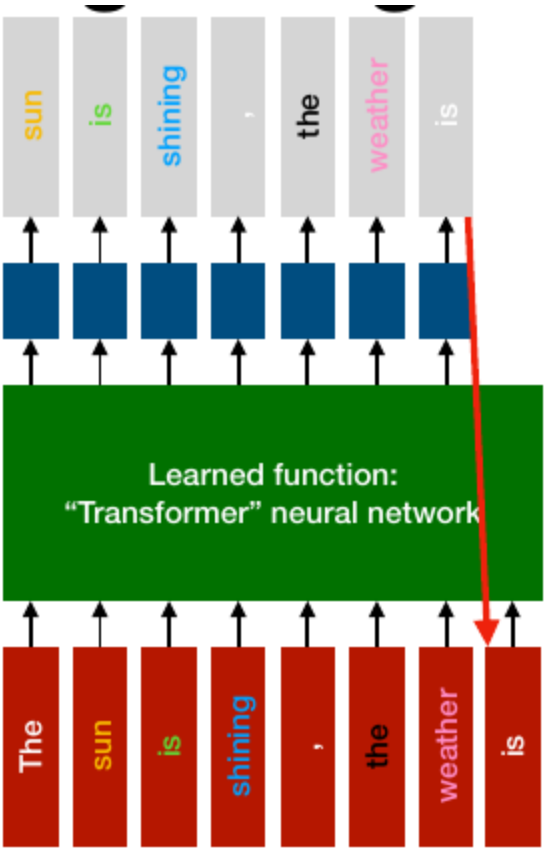
**Contextualized representations**:

- for **tagging**, a **linear classifier on top of each contextualized representation** (learned together with other params, one logistic loss per element)

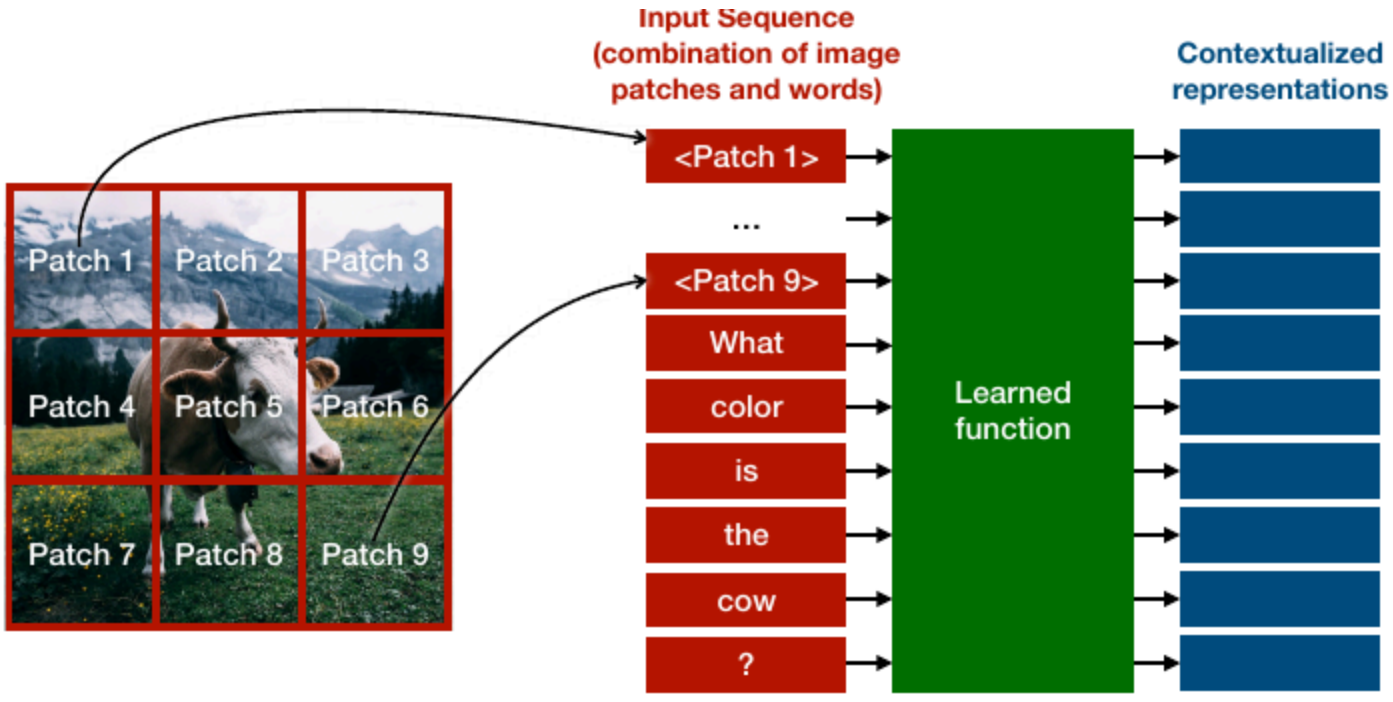
- for **sequence-to-sequence transduction**, a **learned function that maps sequences of contextualized representations to sequences of outputs** (e.g. via next-token prediction)
- for **classification of sequences**, the **final element is interpreted as feature vector for the entire sequence** and fed to a **logistic regression layer** (trained together with other params, logistic loss on the final sequence element). On some tasks, **also feed the contextualized representations to loss functions** and do backpropagation




**Large Language Models (LLM)**: train the neural network to **tag every element with the word that comes next** (**next-word prediction**), **contextualized representation of word $i$ is used to predict word $i+1$** via logistic regression (#classes = #words in vocab), use **masking** to avoid cheating (**next-word prediction can only see words up to $i$**). During **inference** you would **obtain the contextualized representation of last input element and predict the next-word**, add it to the input and repeat

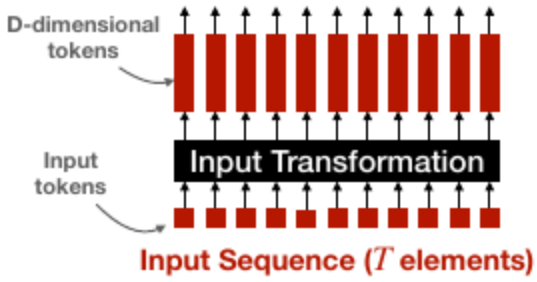



**Images can also be represented as sequences** and you could **also mix in some text** (**multimodality**):

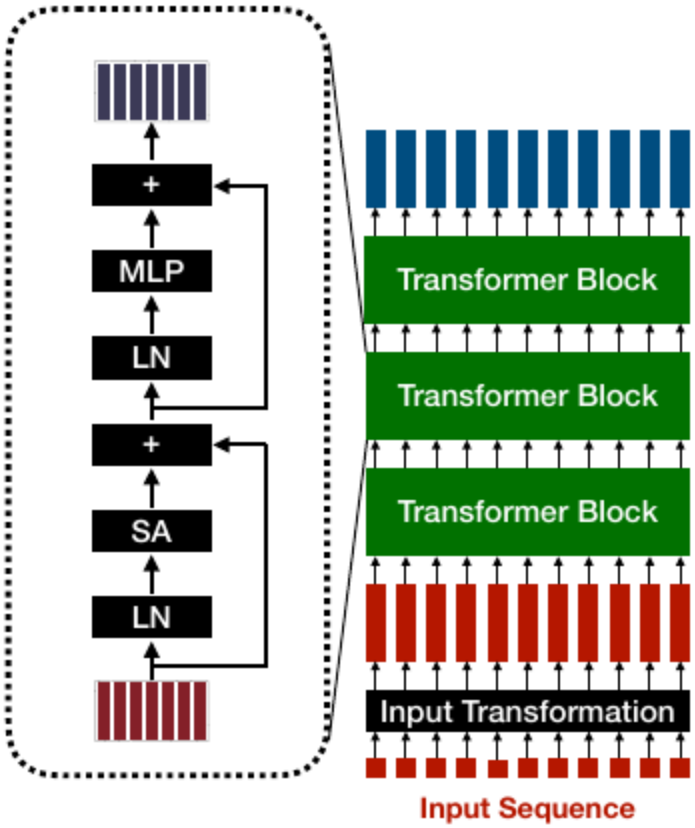



**Transformers**: the **learned function that iteratively transforms a sequence of non-contextualized representations into a sequence of contextualized representations**, **mixes information between sequence elements** via **self-attention**

**Transformer Input transformation**: **converts raw input** sequence elements (**tokens**) **into real-value vector representations**, for texts **every word is replaced with a** fixed non-contextualized **vector** representing the word, for images each image patch is flattened into a vector




**Transformer block**: **transforms a sequence of $T$ vectors of dimension $D$ into a new sequence** of $T$ vectors of dimension $D$ using **self-attention and MLP sub-blocks** (mixes information within each token). **Skip connections and Layer normalization** (LN, at the start of a residual branch) are also often used




**Tokenization**: **split input text into a sequence of input tokens** (represented by IDs) according to some **predefined tokenizer procedure**, **more general than** words and it's the **tradeoff between having an infinite vocabulary with all possible words or having just characters** (that hold no semantic information). An example is **Byte Pair Encoding** (**BPE**), train by using a large corpus of text (**pretraining data**), **start with one token per character**, **merge common pairs of tokens into a token**, **repeat** until desired vocabulary size or all merged

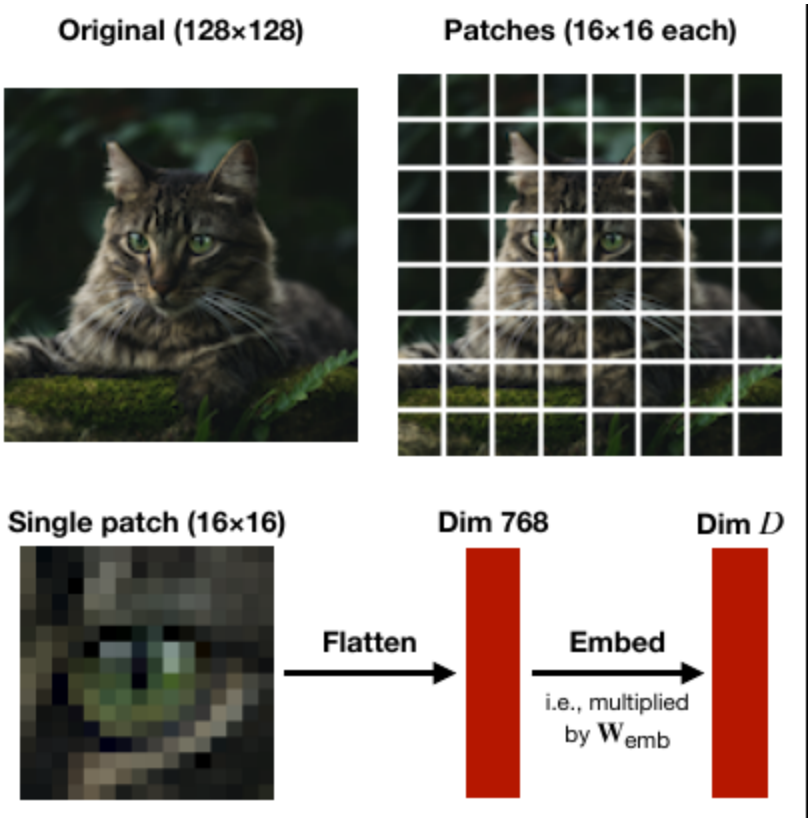
**Token embedding**: **maps each token** ID $i \in \{1, \dots, N_{vocab}\}$ **into a real-valued vector** $w_i \in \mathbb{R}^D$, the whole **input sequence of** $T$ **tokens becomes an input matrix** $X \in \mathbb{R}^{T \times D}$

**Token embeddings matrix**: the mapping of tokens to real-valued vectors is initialized with random numbers and **learned later via backpropagation as part of transformer's training** $W_{emb} = [W_1, \dots, W_{N_{vocab}}]^T \in \mathbb{R}^{N_{vocab} \times D}$ (**contextualized representations**). The actual input matrix is calculated $X = [e_{i_1}, \dots, e_{i_T}]^T W_{emb}$ where $i_1$ in $e_{i_1}$ is the token ID of the first word in the input and the only index with value 1 in the **one-hot encoded vector** $e_{i_1}$ (so **looking up the embedding for a word is a differentiable mathematical operation** $e_i W_{emb} = (W_{emb})_i = w_i$)

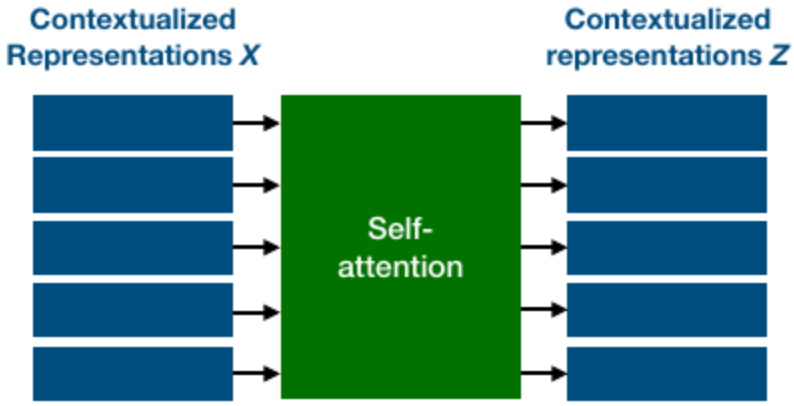
**Softmax**: **converts values stored in a vector** $z \in \mathbb{R}^D$ **into probabilities**, ****pick a dimension $i$ you are interested, the softmax is $\sigma(z)_i = \frac{exp(z_i)}{\sum_j^D exp(z_j)}$, great because it's a **valid probability distribution** ($\sum_i^D \sigma(z)_i = 1$), **differentiable**. Used for **multi-class classification and sensible to outliers** (but **shift-invariant**, so subtracting the dimension with the highest value $z_k$ both above and below can help), **alternate to the sigmoid used in binary classification**

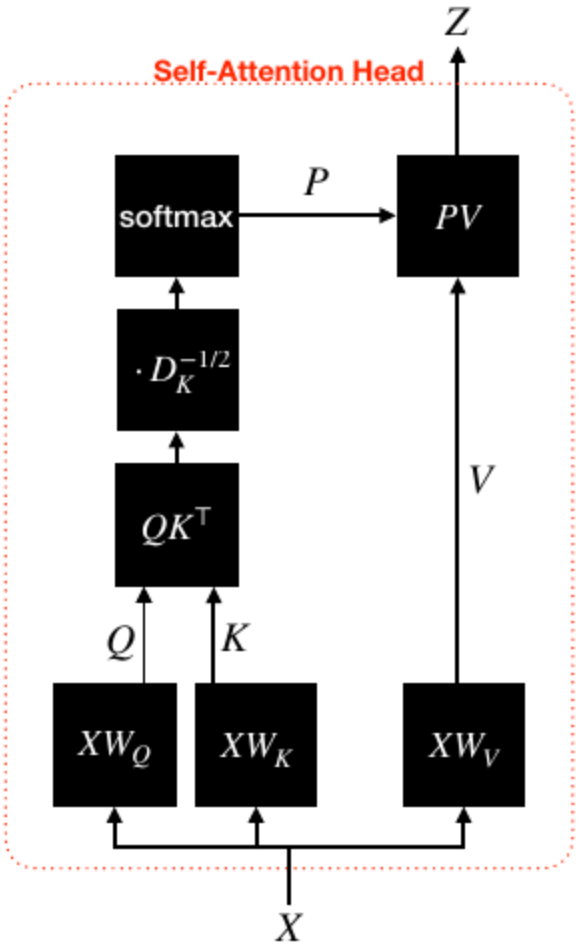
**Label Smoothing**: when using softmax and minimizing the **cross-entropy loss** we are asking our **predictions** $z_k$ **to be exactly 1 when the real outcome** $y_k$ **is 1**, this requires the probability for all other values in our softmax $\sum_{i \neq k} exp(x_i) = 0$, but $exp$ is strictly very positive making it impossible. So to be able to predict exactly 1 with the softmax the **network learns to increase the weights a lot** to reach $exp(x_k) = \infty$, this makes it **numerically unstable**. Solved by **replacing the hard outcome targets with softer ones** $\hat{y} = (1 - \epsilon)y + \frac{\epsilon}{d}1$

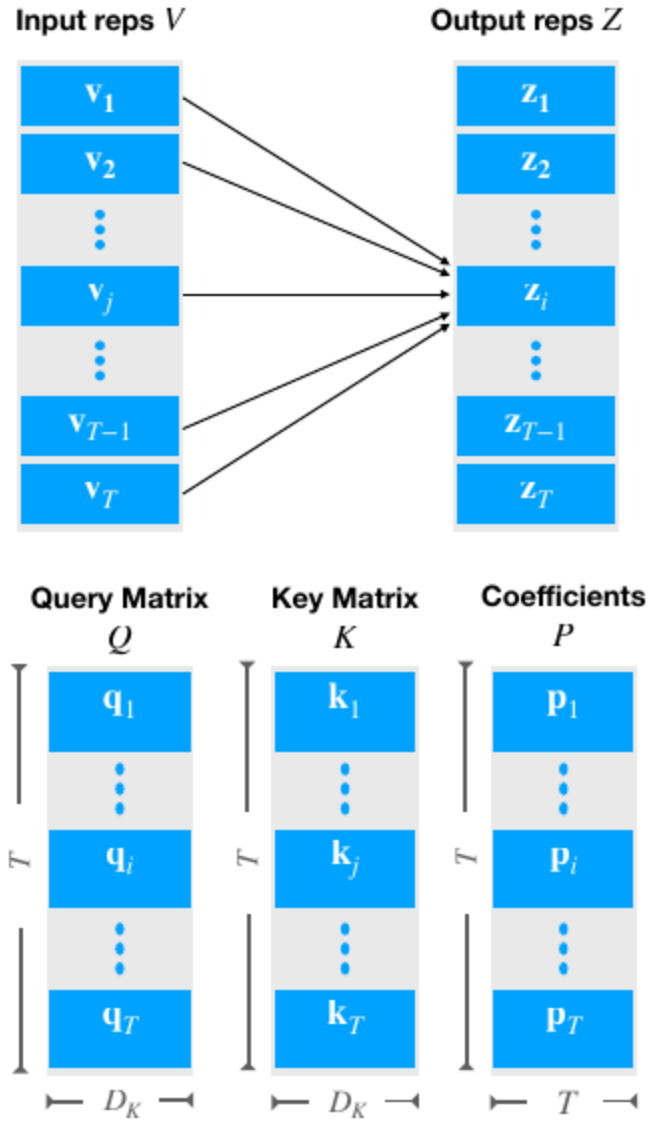
**Image path embeddings**: **divide image into patches of a given size** (e.g. 16x16 pixels), **flatten each patch into a vector** of size $16 \cdot 16 \cdot 3 = 768$, **multiply each resulting vector by an embedding matrix** $W_{emb} \in \mathbb{R}^{768 \times D}$, everything is the same ($N_{vocab} = 768$)



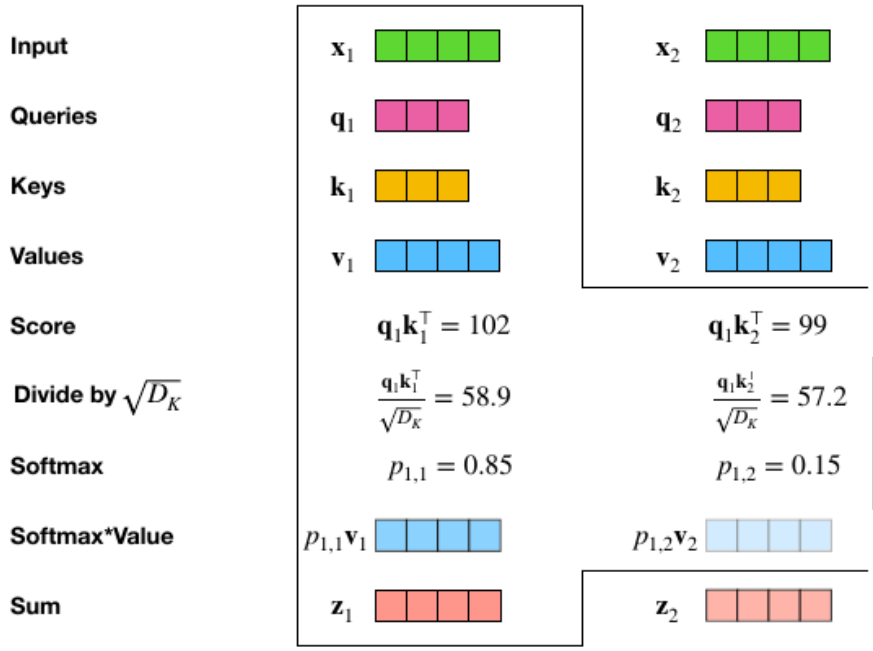

**Self-attention**: function that **transforms a sequence of contextualized representations** (one per input token) **into a new sequence of contextualized representations using learned input-dependent weighted averages**. $T$ input and output tokens $V \in \mathbb{R}^{T \times D_v}$, $Z \in \mathbb{R}^{T \times D_v}$, **each output is a weighted average of the inputs** $z_i = \sum_{j=1}^{T} p_{i,j} v_j$ (or $Z = PV$) where $P \in [0,1]^{T \times T}$ are the **weighting coefficients** (**valid probability distribution over the input tokens**, $\sum_{j=1}^{T} p_{i,j} = 1$). Computing the input sequence $X \in \mathbb{R}^{T \times D}$, keys $K = XW_K \in \mathbb{R}^{T \times D_K}$, queries $Q = XW_Q \in \mathbb{R}^{T \times D_K}$, values $V = XW_V \in \mathbb{R}^{T \times D_V}$ with $W_K, W_Q \in \mathbb{R}^{D \times D_K}$, $W_V \in \mathbb{R}^{D \times D_V}$ as parameters. The final output of self-attention is $Z = PV$. **One query token** $q_i$ **per output** token and **one key token** $k_i$ **per input** token. Determine **weight** $p_{i,j}$ **based on how similar** $q_i$ **and** $k_j$ are (use **inner product to obtain raw similarity scores**, **normalize with softmax** scaled with "**temperature**" $\sqrt{D_K}$ so that we **dont have a sharp distribution of** $p$ **weights at random initialization** (which is an issue because it takes a lot more time to adjust from the initial peak due to vanishing gradients), **uniformity at initialization and faster convergence**) to **obtain a probability distribution**: $p_{i,j} = \frac{exp(\frac{q_i k_j^T}{\sqrt{D_K}})}{\sum_{i=1}^{T} exp(\frac{q_i k_t^T}{\sqrt{D_K}})}$ (in matrix form $P = softmax(\frac{QK^T}{\sqrt{(D_K)}})$ (applied on each row independently))


Contextualized
Representations X
Contextualized
representations Z
Self-
attention


Z
Self-Attention Head
softmax
P
PV
$\cdot D_K^{-1/2}$
$QK^\top$
V
Q
K
$XW_Q$
$XW_K$
$XW_V$
X

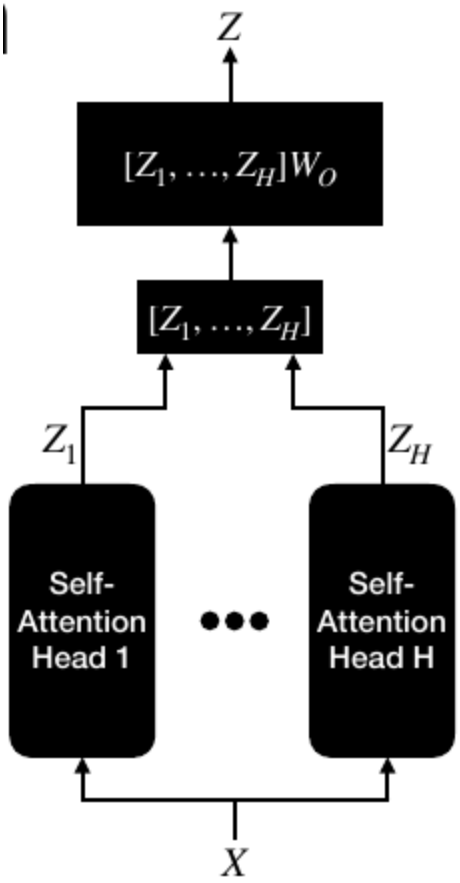



$O(T \times T)$ so **it scales quadratically** (not nice) but **it's parallelizable**. In some applications **causal masking** is used, so you sum until position $i$ instead of $T$ in the denominator and you have $p_{i,j} = 0, \ \ j > i$, or in matrix form you add before the softmax $M \in \mathbb{R}^{T \times T}, \ M_{ij} = -\infty, j > i, \ 0$ otherwise. **Attention does not account for the order of input**, but **in practice it should matter** (e.g. "She prefers cats to dogs" ≠ "She prefers dogs to cats") so **positional embeddings** $W_{pos}$ ****(learned via backpropagation, map position of a token in a sequence to a feature vector just for that, $\{1, \ldots, T\} \to \mathbb{R}^D$) **are added to the input token's embeddings**, $X = [e_{i_1}, \ldots, e_{i_T}]^T W_{emb} + [e_1, \ldots, e_T]^T W_{pos}$

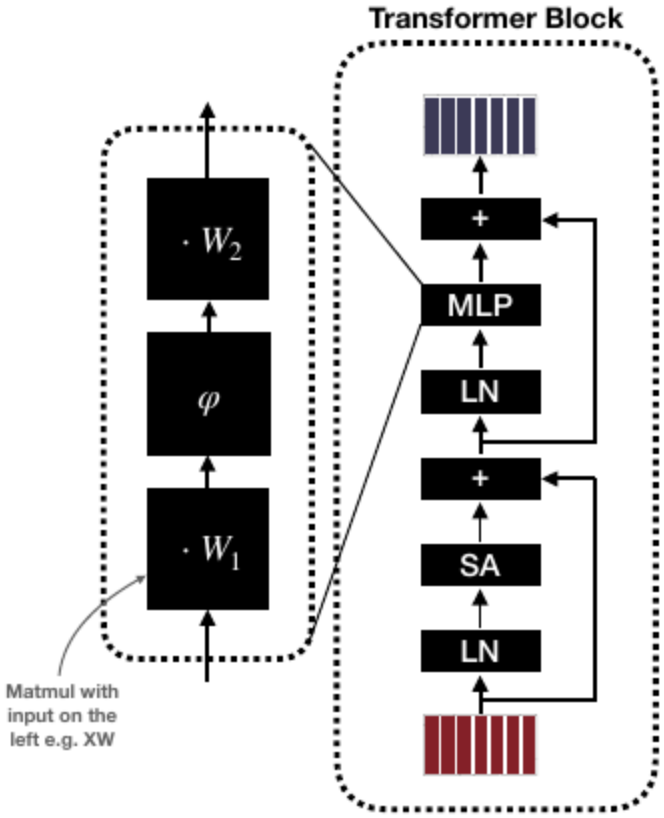



$$Z = \text{softmax}\left(\frac{XW_Q W_K^\top X^\top}{\sqrt{D_K}}\right) XW_V$$

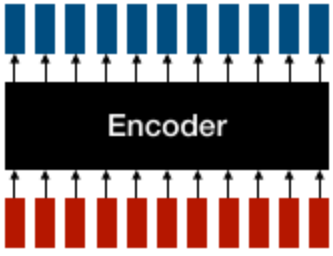
**Multi-Head Self-Attention**: **multiple attention patterns per layer in parallel** (like having multiple output channels in a convolution layer), each attention pattern has "head" h and output $Z_h = softmax(\frac{XW_{Q,h}W_{K,h}^T X^T}{\sqrt{D_K}})XW_{V,h}$ with $W_{K,h}, W_{Q,h} \in \mathbb{R}^{D \times D_K}, W_{V,h} \in \mathbb{R}^{D \times D_V}$. The final output is obtained through concatenation $Z = [Z_1, \ldots, Z_H]W_O$ where $W_O \in \mathbb{R}^{HD_V \times D}$ is learned via backpropagation

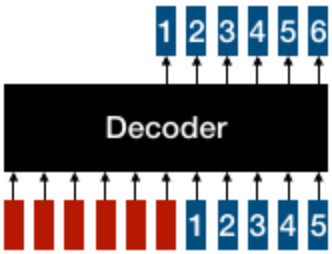


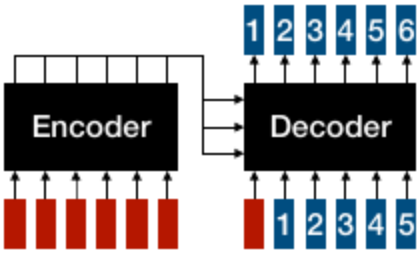
**Multi-Layer Perceptron (MLP) in Transformers**: **mixes information within each token** (from each **tokens we study and think on its meaning**, but it's **independent of all other tokens**), applies the **same transformation to each token independently** $MLP(X) = \varphi(XW_1)W_2$ with $W_1, W_2 \in \mathbb{R}^{D \times D}$ learned via backprop, $\varphi$ introduce non-linearity (e.g. ReLU), may also have bias terms




**Uses for Transformers**: **encoders** (produce fixed output size and process all inputs at the same time, e.g. element-level tagging, sequence-level classification), **decoders** (**auto-regressively sample the next token** as $x_{t+1} \sim softmax(f(x_1, \ldots, x_t))$ and append it as a new token, repeat, capable of **generating responses of arbitrary length**, e.g. **ChatGPT**) and **encoder-decoder** (first encode the whole input (in one language) then decode token by token (in another language), e.g. translation). Since **everything can be tokenized**, **transformers can always be applied**. CNNs can also be used for text processing, but **transformers excel at capturing long-range depedencies** (large **context-windows**). When in big scale **transformers can do few and zero shot learning** (no prior training on the task)






# 15 - Adversarial Machine Learning

**Adversarial examples**: **small perturbations that cause misclassification with high confidence**, neural networks struggle with them (security issue) because we **don't understand how these models generalize and react to shifts in the distribution** of data

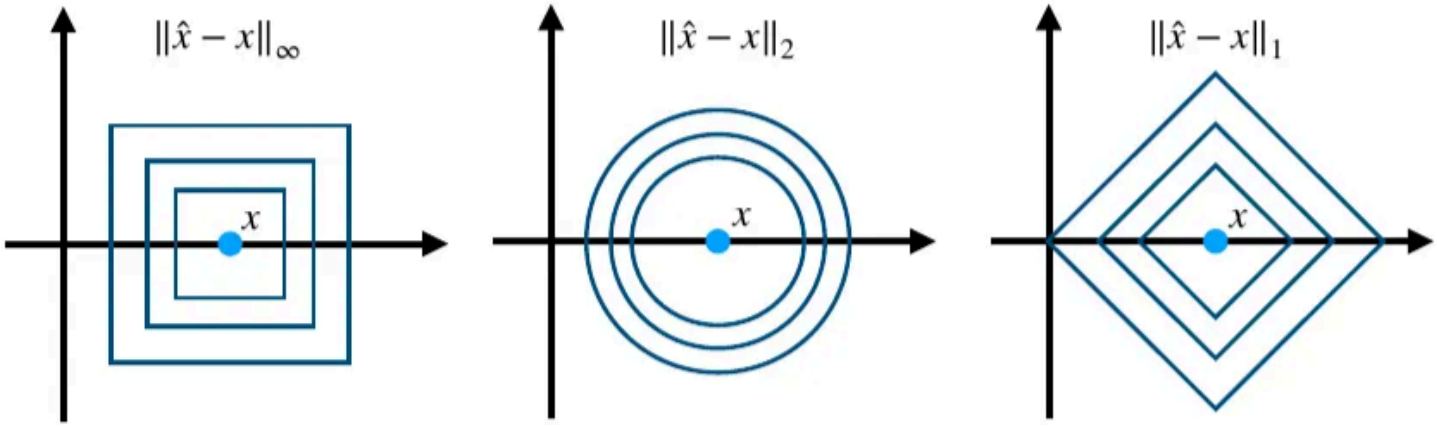
**Adversial risk**: **average zero-one loss over small, worst-case perturbations** of $X$, $R_\epsilon(f) = \mathbb{E}_{\mathcal{D}}[max_{\hat{x},||\hat{x}-X||\leq\epsilon}1_{f(\hat{x})\neq Y}]$, **defining the adversary's power, picking the norm and the set of perturbations can be tricky**, also depends on what access we have to the model

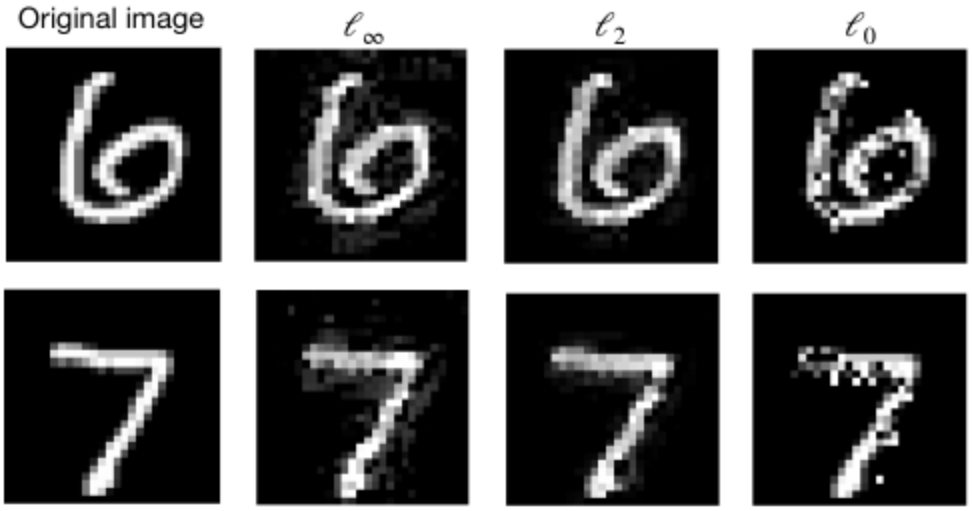
**Generating adversarial examples**: given an input $(x, y)$ and a model $f : X \to \{-1, 1\}$, **find an input $\hat{x}$ such that $||\hat{x} - x|| \leq \epsilon$ and the model makes a mistake on it**, only if $x$ is correctly classified by the model. **Solve the optimization problem** $max_{\hat{x},||\hat{x}-X||\leq\epsilon}1_{f(\hat{x})\neq Y}$, difficult because $1$ **is not continuous** and $f$ **outputs discrete class values** $\{-1, 1\}$. To fix that we **replace it with a smooth problem** $max_{\hat{x},||\hat{x}-X||\leq\epsilon}l(yg(\hat{x}))$ using a **smooth classification loss** $l$ and **applying it on the output** $g$ **of the neural network before classification**

**White-box adversarial attacks**: when we know the model **we can generate adversarial examples by computing its gradient and applying gradient descent** to find a point that maximizes the loss. We can linearize the problem, $max_{\hat{x},||\hat{x}-X||\leq\epsilon}l(yg(\hat{x})) \to max_{\hat{x},||\hat{x}-X||\leq\epsilon}l(yg(x)) + \nabla_x l(yg(x))^T(\hat{x} - x) = l(yg(x)) + max_{\hat{x},||\hat{x}-X||\leq\epsilon}\nabla_x l(yg(x))^T(\hat{x} - x) = l(yg(x)) + max_{||\delta||\leq\epsilon}\nabla_x l(yg(x))^T\delta$ with $\delta = \hat{x} - x$ the **perturbation size**, this is a **simple problem**. Its closed-form solution **depends on the norm used** to measure $||\delta||$ (**one single step/attack**), if the $L2$ norm is used then the adverserial point will be in the circle with radius $\delta$ ($\delta_2^\star = \varepsilon \cdot \frac{\nabla_x l(yg(x))}{||\nabla_x l(yg(x))||_2} = -\varepsilon y \cdot \frac{\nabla_x g(x)}{||\nabla_x g(x)||_2} \to \hat{x} = -\varepsilon y \cdot \frac{\nabla_x g(x)}{||\nabla_x g(x)||_2}$), if $L\infty$ is used it will be on the edges of the box with distance $\delta$ ($\delta_\infty^\star = \epsilon \cdot sign(\nabla_x l(yg(x))) = -\epsilon y \cdot sign(\nabla_x g(x)) \to \hat{x} = x - \epsilon y \cdot$

$sign(\nabla_x g(x))$, **fast gradient sign method**). The optimal $\hat{x}$ can also be calculated in **multiple iterative steps (attack)** using **projected gradient descent**, if $L2$ norm is used $\delta^{t+1} = \prod_{B_2(\epsilon)}(\delta^t + \alpha \cdot \frac{\nabla l(yg(x+\delta^t)}{||\nabla l(yg(x+\delta^t)||_2})$ where $\prod_{B_2(\epsilon)}(\delta) = \{\epsilon \cdot \frac{\delta}{||\delta||_2}, \ \ ||\delta||_2 \geq \epsilon$ otherwise $\delta$, if $L\infty$ norm is used $\delta^{t+1} = \prod_{B_\infty(\epsilon)}(\delta^t + \alpha \cdot sign(\nabla l(yg(x+\delta^t)))$ where $\prod_{B_\infty(\epsilon)}(\delta_i) = \{\epsilon \cdot sign(\delta_i), \ \ |\delta_i| \geq \epsilon$ otherwise $\delta_i$. $\nabla_x g(x)$ can be **computed by backpropagation** since they are **taken w.r.t inputs**

***Different norms have different geometry*** *and* ***properties*** *(e.g. $l\infty$ are dense, $l0$ sparse). We need to pick which perturbations we should aim to be robust against*



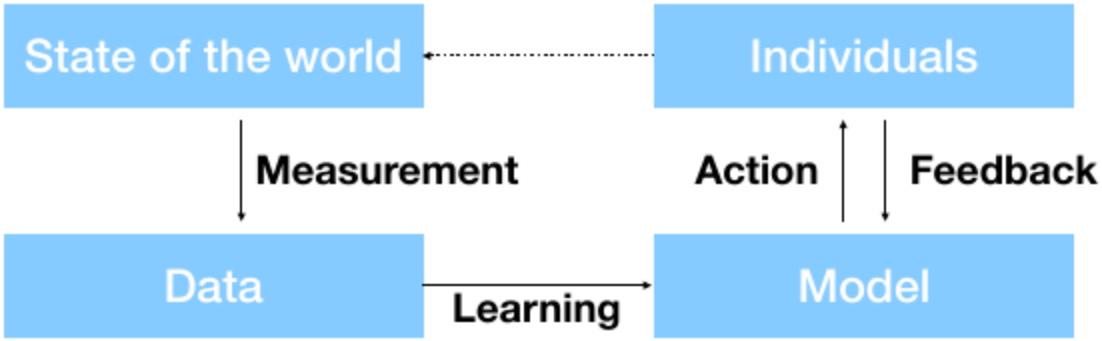

**Black-box adversarial attacks**: solved with **query-based gradient estimation**, two possible cases that **both require a lot of queries** (10k-100k, **access can be restricted**): we **can query the continuous model scores** $g(x) \in \mathbb{R}$ (**score-based**) or we **can query only the predicted class** $f(x) \in \{-1, 1\}$ (**decision-based**). In **score-based** we can **approximate the gradient using the finite difference formula** $\nabla_x g(x) \approx \sum_{i=1}^{d} \frac{g(x+\alpha e_i) - g(x)}{\alpha} e_i$, similar techniques are used in decision-based ($x$ is near the decision boundary). **Also solved via transfer attacks**, **train a similar surrogate model** $\hat{f} \approx f$ **on similar data** (**costly**), **transfer the resulting white-box adversarial perturbation** from $\hat{f}$ to $f$, success depends on how similar the models are. If we are allowed to query $f$ given some unlabeled inputs $\{x_n\}_{n=1}^N$ we can make $\hat{f}$ more fitting (**model stealing**)

**Physically realizable attacks**: **adversarial examples must meet additional requirements**, like the **perturbation not going away after JPEG compression**, the **perturbations should be bigger than simple photographic distortions** (for real-world examples) **or different camera angles of the same object** because during training the **model probably became naturally robust to those perturbations already**

**Adversarial training**: goal is to **minimize the adversarial risk**, the real data distribution $\mathcal{D}$ is unknown but we **approximate it with a sample average**, the classification loss is non-continuous so we apply a **smooth loss**, so $min_\theta \frac{1}{N} \sum_{n=1}^{N} max_{\hat{x},||\hat{x}-X||\leq\epsilon} l(y_n g_\theta(\hat{x}_n))$ (**minimizing the risk on adversarial examples**). At each iteration $t$, for each $x_n$ approximate $\hat{x}_n^* \approx arg\ max_{\hat{x},||\hat{x}-X||\leq\epsilon} l(y_n g_\theta(\hat{x}_n))$ via **PGD attack**, **perform a gradient descent step** w.r.t $\theta$ using $\frac{1}{N} \sum_{n=1}^{N} \nabla_\theta l(y_n g_\theta(\hat{x}_n^*))$. **State-of-the-art approach for robust classification**, leads to **more interpretable gradients** $\nabla_x l(y g_\theta(x))$, **fully compatible with SGD**. Bad news is the **increased computational time** (proportional to the number of PGD steps) and the **robustness-accuracy tradeoff** (using too large $\epsilon$ leads to worse accuracy but more robustness)

# 16 - Ethics

**The machine learning loop**: **people and their behaviour are at the center of most ML applications** (14 out of top 30 in Kaggle are about decisions made by individuals). Training **data often encodes existing demographic disparities**. Since **ML applications have an impact on the world** we have to be aware of that, **it's not only about the training**




*We **don't want to discriminate against social categories** that have served as the basis for unjustified and systematically adverse treatment in the past. We **don't care about differentiating by factors that are practically or morally irrelevant***

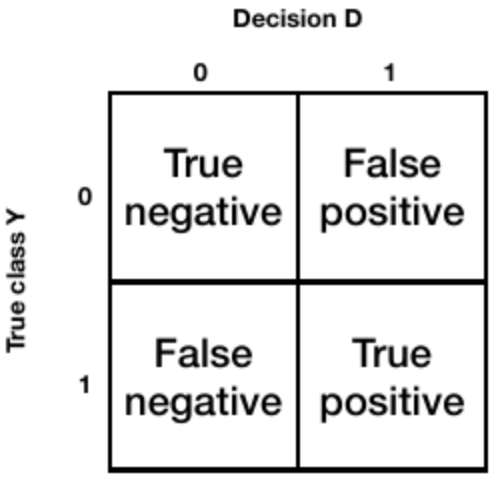
**Provenance of data**: **crucial to understand** it as a researcher, how do you **define your variable of interest** (e.g. how do you measure intelligence), the **process for interacting with the real world** (e.g. questionnaires) and **turning observations into numbers** (i.e. data collection, e.g. camera). Can be **subjective and challenging**

*Some **patterns in training data represent knowledge** (we want to learn it), **others represent stereotypes** (we want to avoid), **ML algorithms cannot distinguish** between the two, **intervention is needed***

**Sample size disparity**: **uniform subsampling from population leads to fewer data about minorities**, if the minorities are underrepresented compared to reality than we have even fewer data, and **ML works best with a lot of data**. Also, **true error is an average criterion** so it may hide high error on minorities in exchange of low error on the majority (makes sense to avoid overfitting, e.g. outliers)

*Simply **removing sensitive attributes does not give fairness** because other attributes may be correlated with the sensitive attributes*

**Fairness Criteria**: in binary classification ****a **score** $R$ is produced in function of $X$ and mapped to a label using a threshold $t$, **final prediction** is $D = 1_{R>t}$. **Introduce a random variable $A$ encoding membership status in a protected class**. Most of the **fairness criteria are properties of the $(A, Y, R)$ set**: **independence** ($R \perp A$, **equal acceptance rate**), **separation** ($R \perp A|Y$, **equal error rates**) and **sufficiency** ($Y \perp A|R$, **calibration by group**). To have a **fair treatment we want independence** so that $P(D = 1|A = a) = P(D = 1|A = b)$, but it's not enough because it's **just saying that both groups have the same probability for outcome** $1$ (maybe to get this result one of the groups was treated unfairly, they only **matched true positives with false positives**). **Separation can be applied to solve that** but it **uses the true outcome $Y$ so it can only be used in retrospect** (**post-hoc** criterion), it's saying that $\mathbb{P}(D = 1|Y = 0, A = a) = \mathbb{P}(D = 1|Y = 0, A = b)$ (same false positives) and $\mathbb{P}(D = 0|Y = 1, A = a) = \mathbb{P}(D = 0|Y = 1, A = b)$ (same false negatives). **Suffiency** instead says that we **do not need $A$ to predict $Y$ if we have** $R$ ($\mathbb{P}(Y = 1|R = r, A = a) = \mathbb{P}(Y = 1|R = r, A = b)$)




True positive rate: $\mathbb{P}(D = 1 \mid Y = 1)$

False positive rate: $\mathbb{P}(D = 1 \mid Y = 0)$

True negative rate: $\mathbb{P}(D = 0 \mid Y = 0)$

False negative rate: $\mathbb{P}(D = 0 \mid Y = 1)$

**Calibration**: a score $R$ is calibrated if $\mathbb{P}(Y = 1|R = r) = r$, the **probability of the true outcome being $1$, when our computed score is $r$, is also $r$**, $R$ is in fact a **probability**. **Calibration by group**

$\mathbb{P}(Y=1|R=r,A=a)=r$ **implies suffiency** (the opposite is also possible)

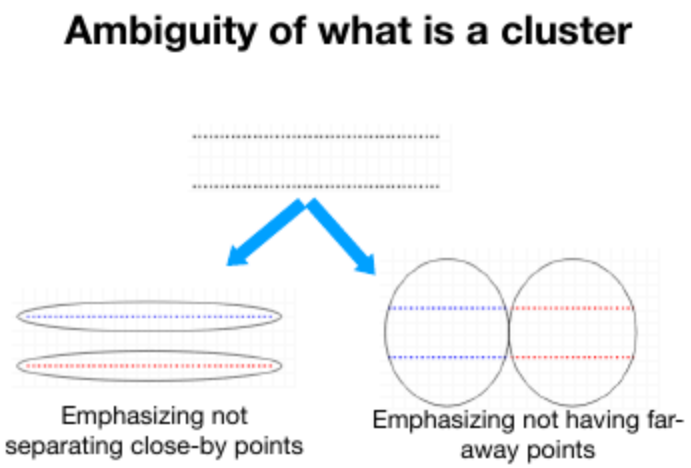
**How to achieve the Fairness Criteria**:

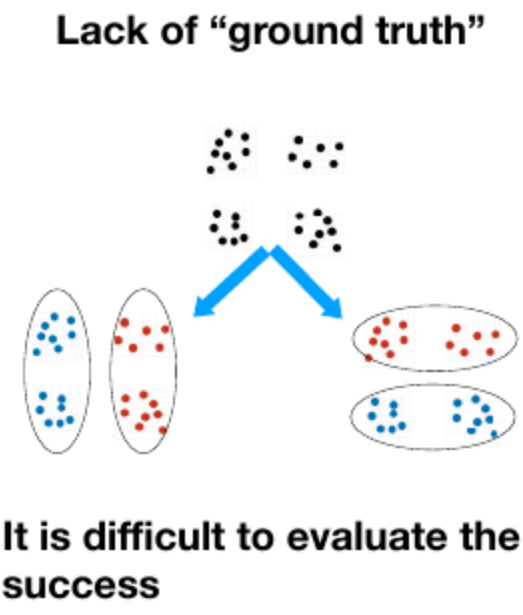
- **Pre-processing**: **adjust your features** so that they become **uncorrelated with the sensitive attribute** $A$
- **At training time**: work the **constraint into the optimization** process
- **Post-processing**: **adjust your learned classifier** so that it becomes **uncorrelated with the sensitive attribute** $A$

# 17 - Unsupervised Learning

**Unsupervised Learning**: uses **unlabeled training data** $\mathcal{D}=\{x_n\}_{n\in[1,\dots,N]}$ (common and cheap), learning goals are **learning a compact and useful representation of the data** (**PCA, matrix factorization, Word2Vec, BERT, CLIP**), density estimation and **generative modelling** (**GMMs, LLMs, GANs and Diffusion models**). Useful for **exploratory data analysis**, **anomaly detection**, **learning good features/embeddings**, generating new observations and **clustering** (**K-means, GMMs**). More occurrent in humans. Many different ideas are used to **find a valid training objective**

**Clustering**: mainly for **exploratory data analysis**, groups objects such that **similar objects end up in the same group** and dissimilar objects are separated into different groups. **Graph-based** if we **compute a graph** from the objects and then **partition it to minimize inter-cluster weight and maximize intra-cluster weights**. **Hierarchical** if we **build a tree** (bottom-up or top-down) **representing distances among data points**. **Model-based** if maintain **cluster models and infer membership based on them** (e.g. **K-Means, GMMs**)

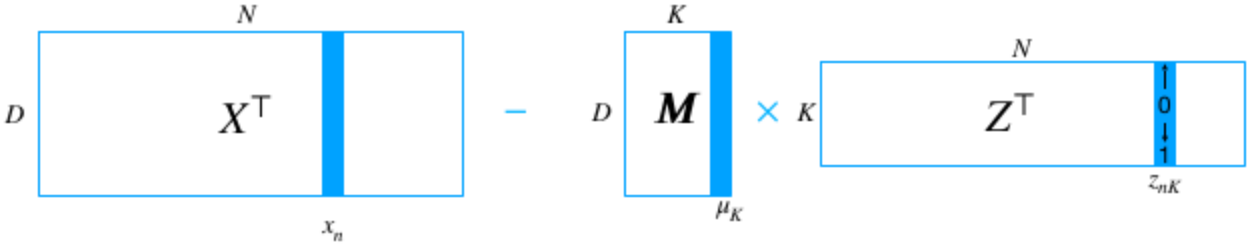


**K-Means**: **model-based** clustering, **partition data in K clusters** where **each point belongs to the cluster with the closest mean**. **Outputs K $\mu$ centroids** (**centers of each cluster**, representing the mean of the points in it) and $z_{n,k} \in \{0, 1\}$ **cluster assignments** for each point. **Objective function is the within-cluster sum of squares (WCSS)** $min_{z,\mu}\mathcal{L}(z, \mu) = \sum_{n=1}^{N}\sum_{k=1}^{K} z_{n,k}||x_n - \mu_k||_2^2$ such that $\mu_k \in \mathbb{R}^D, \sum_{k=1}^{K} z_{n,k} = 1$ where $z_n = [z_{n,1}, \ldots, z_{n,K}]^T, z = [z_1, \ldots, z_N]^T, \mu = [\mu_1, \ldots, \mu_K]^T$. **NP-hard problem**, some of the **optimization variables are discrete**, equivalent objective $min_{\mu}\mathcal{L}(\mu) = \sum_{n=1}^{N} min_{k\in\{1,\ldots,K\}}||x_n - \mu_k||_2^2$ is **non-convex**

**K-Means Algorithm**: **randomly choose initial centroids**, for each point $n$ **compute** $z_{n,k} = \{1\ if\ k = argmin_j||x_n - \mu_j||_2^2$ otherwise $0$ and **update all centroids** $k$ $\mu_k = \frac{\sum_{n=1}^{N} z_{n,k}x_n}{\sum_{n=1}^{N} z_{n,k}}$, **repeat until convergence**, $O(NKd)$. It's **monotonically decreasing the objective function** ( $\mathcal{L}(z^{(t)}, \mu^{(t)}) \leq \mathcal{L}(z^{(t-1)}, \mu^{(t-1)})$) because in the first step we optimize the assignments and in the second step we optimize the centroids, we **cannot make it worse than before**. It's a **coordinate descent algorithm** $z^{(t+1)} = argmin_z\mathcal{L}(z, \mu^{(t)}),\ \mu^{(t+1)} = argmin_{\mu}\mathcal{L}(z^{(t+1)}, \mu)$ and it **converges in a finite number of steps** (monotonically decreasing), **no guarantees on how fast** and **no nontrivial lower bound on the gap between the final objective value and global minimum** (**may not find it**)

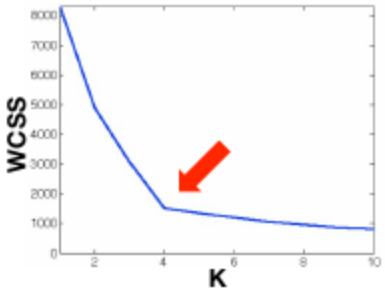
**K-Means with Matrix Factorization**: $min_{z,\mu}\mathcal{L}(z, \mu) = \sum_{n=1}^{N}\sum_{k=1}^{K} z_{n,k}||x_n - \mu_k||_2^2 = ||X^T - MZ^T||_{Frob}^2$



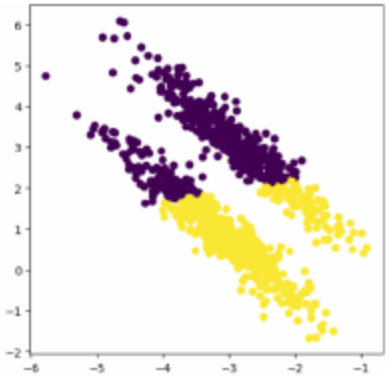

$\mathbf{x}_n$ is reconstructed by the mean of closest centroid $\mu_k$

**K-Means++**: since **initialization changes the model performance a lot in K-Means**, we want to **avoid poor initial centroid placements by spreading out the initial centroids** so that they are **more representative of the data distrubution** (rather than random), **avoids poor local minima**, **faster convergence** and **lower final objective**. Start by **selecting the first centroid** $\mu_1$ **randomly** from data points, then for each point $x_n$ calculate the distance $D(x_n) = min_{j\in\{1,\ldots,m\}}||x_n - \mu_j||_2^2$ where $\{\mu_1, \ldots, \mu_m\}$ represent the centroids that have already been chosen, select the next centroid $\mu_{m+1}$ by choosing a datapoint $x_n$ with probability $P(x_n) = \frac{D(x_n)}{\sum_{i=1}^{N} D(x_i)}$, repeat until you have $K$ centroids, run K-Means

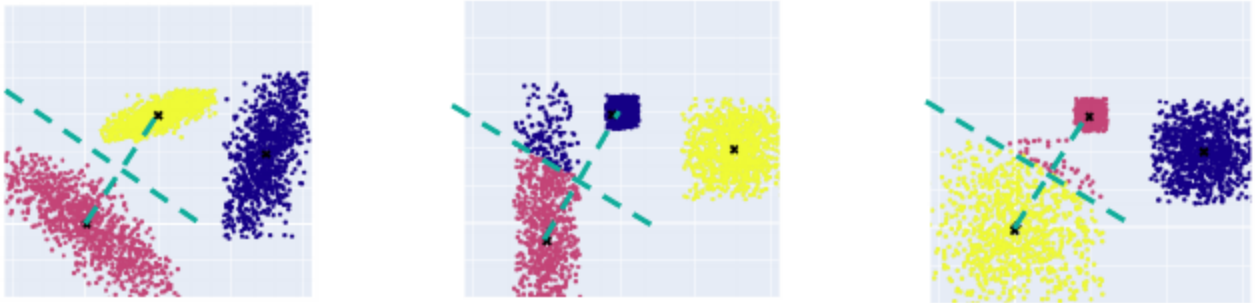
**Defining K in K-Means**: **test loss typically keeps decreasing as you increase the number of clusters** (K), so cross-validation cannot be used. We either use **domain knowledge or the elbow method** (run **K-means for different K values** and **calculate the final objective**, **plot it as K varies** and look for an **elbow point**, because a new cluster significantly reduces loss when there are too few clusters)

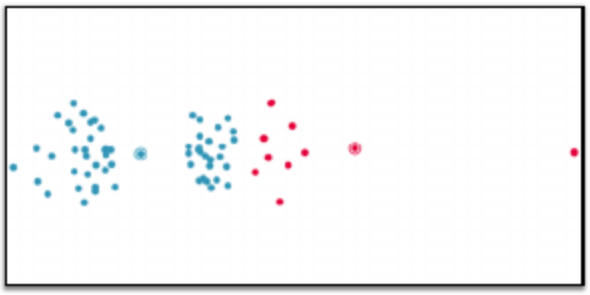



**When does K-Means fail**: when **clusters have arbitrary shape** (since K-Means forces clusters to have spherical shapes) and when **clusters are not separated by the bisectors of the lines connecting their centers**. Also **struggles with overlapping clusters** that could belong to any cluster **and with outliers as they can drag centroids away from the actually dense areas**

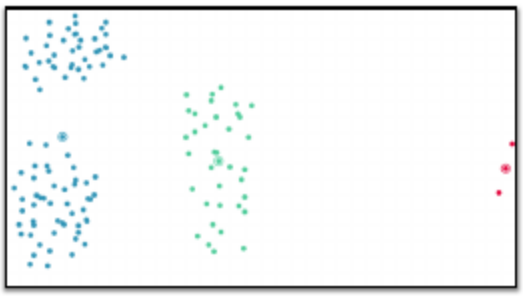


K-means struggles with elliptical clusters

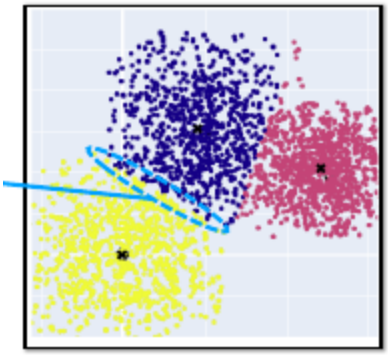




Outliers can lead to suboptimal results



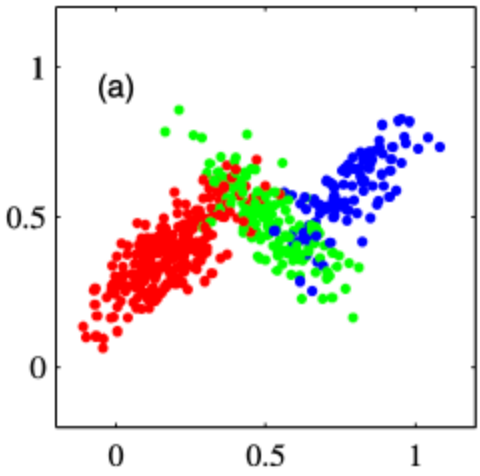
Outliers can lead to undesirable clusters



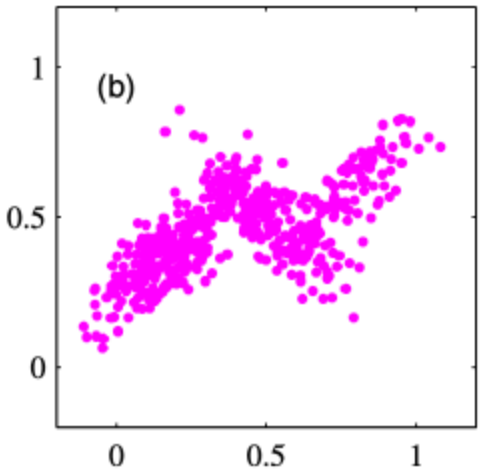
**Probabilistic view on K-Means**: consider each cluster as an isotropic Gaussian $p(x_n|\mu, z_n) = \prod_{k=1}^{K} \mathcal{N}(x_n|\mu_k, I_D)^{z_{n,k}} = \prod_{k=1}^{K} cexp(-\frac{1}{2}||x_n - \mu_k||_2^2 z_{n,k})$. The dataset likelihood will be $log \prod_{n=1}^{N} p(x_n|\mu, z_n) = log \prod_{n=1}^{N} \prod_{k=1}^{K} cexp(-\frac{1}{2}||x_n - \mu_k||_2^2 z_{n,k}) = -\sum_{n=1}^{n} \sum_{k=1}^{K} z_{n,k}||x_n - \mu_k||_2^2 + c' = -\mathcal{L}(z, \mu) + c'$, **K-Means can be interpreted as maximizing the log-likelihood of the data** with this data model

# 18 - Expectation Maximization Algorithm

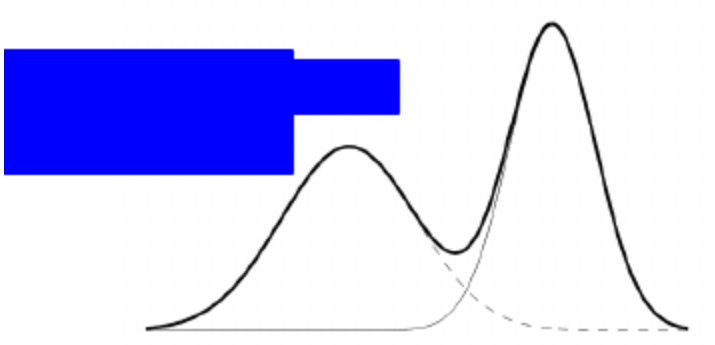
**Gaussian Mixture Models (GMMs)**: **data is modeled as a mixutre of $K$ Gaussian distributions** (clusters), sample space is $X = \mathbb{R}^D$ and each sample $x$ is generated as follows: choose a random component $k \in \{1, \dots, K\}$ (i.e. let $Z$ be a multinomial random variable with $P(Z = k) = \pi_k$ **mixture proportion**), sample $x$, given the value of $Z = k$, from a Gaussian distribution $p(x|Z = k) = \mathcal{N}(x|\mu_k, \Sigma_k) = \frac{1}{(2\pi)^{d/2}|\Sigma_k|^{1/2}} exp(-\frac{1}{2}(x - \mu_k)^T \Sigma_k^{-1}(x - \mu_k))$ (**mixture components**). We can write the distribution of $x$ as $p(x) = \sum_{k=1}^{K} P[Z = k]p(x|Z = k) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x|\mu_k, \Sigma_k) = \sum_{k=1}^{K} \frac{\pi_k}{(2\pi)^{d/2}|\Sigma_k|^{1/2}} exp(-\frac{1}{2}(x - \mu_k)^T \Sigma_k^{-1}(x - \mu_k))$. $Z$ **is a latent (hidden) variable** that is not directly observed, we introduce it as it helps describe a simple parametric form (like in kernels). You are trying to say that you are the **data was generated by $K$ different Gaussian distributions** (**each represents a cluster**), the **probability of each point of belonging to a particular distrubution/cluster $k$ is $P(z = k)$** ($Z$ is the distribution we are using for a specific point), **once you pick the distribution you are drawing the samples from you have** $p(x|Z = k) = \mathcal{N}(x|\mu_k, \Sigma_k)$, we then **apply the law of total probability to get** $p(x)$




Samples from the joint distribution $p(X, Z)$, with Z represented by colors




Samples from the marginal distribution $p(X)$



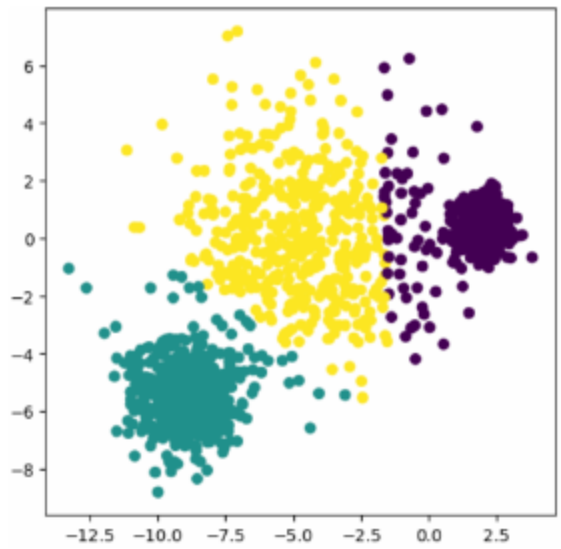
Average of two probability distributions of two Gaussian for which it is natural to introduce a mixture model

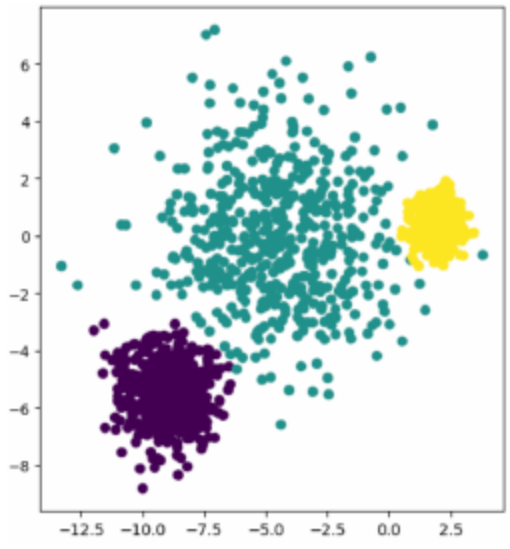
**Maximum log likelihood with Latent Variables**: given samples $\mathcal{D} = \{x_1, \ldots, x_N\}$ and a latent variable model $p(x, z|\theta)$ ($z$ is observed), we want to maximize the log-likelihood of $\mathcal{D}$, i.e. $\hat{\theta}_{MLE} = argmax_\theta log\ p(\mathcal{D}|\theta)$ with $log\ p(\mathcal{D}|\theta) = log \prod_{n=1}^{N} p(x_n|\theta) = \sum_{n=1}^{N} log\ p(x_n|\theta) = \sum_{n=1}^{N} log(\sum_{z_n=1}^{K} p(x_n, z_n|\theta)) = \sum_{n=1}^{N} log(\sum_{k=1}^{K} p(z_n|\theta)p(x_n|z_n, \theta))$, the **summation inside the log makes the optimization problem computationally hard** (Expectation-Maximization (EM) is used in cases where if we knew $z$, the MLE would have been computationally okay)

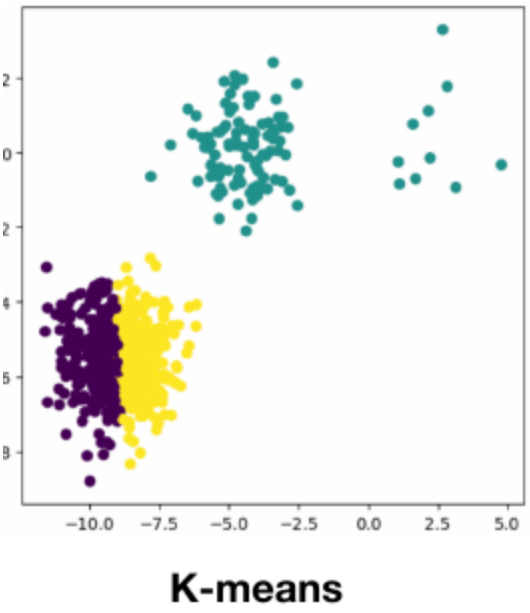
**MLE Objective for Gaussian Mixtures**: $max_\theta \sum_{n=1}^{N} log(\sum_{k=1}^{K} \pi_k \mathcal{N}(x|\mu_k, \Sigma_k))$, **non convex**, **no unique optimum** (permutation of K clusters), **unbounded**, can be solved directly using black-box optimization algorithms. We want to **solve this using the Expectation-Maximization (EM)** algo

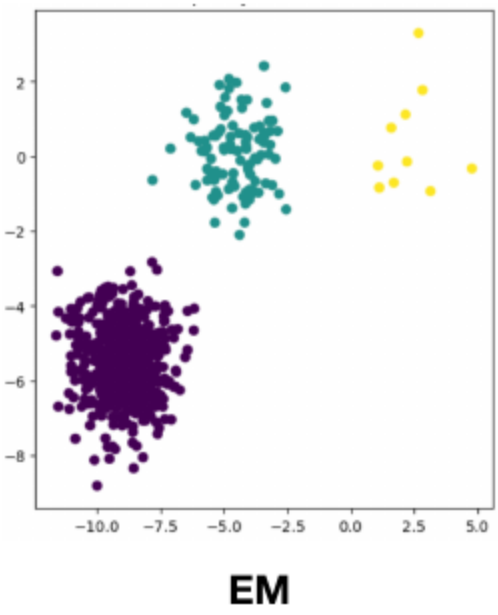
**MLE with access to Labels for GMM**: assume we have $\mathcal{D}_c = \{(x_n, z_n)\}_{n\in[1,\ldots,N]}$ the **complete log likelihood** is $\mathcal{L}_c(\theta|\mathcal{D}_c) = \sum_{n=1}^{N} log(\pi_{z_n} \mathcal{N}(x_n|\mu_{z_n}, \Sigma_{z_n}))$ which is **convex**, if we had the labels we could **easily compute MLE in closed form** (write it as a precise mathematical formula that gets you the optimal params) $\hat{\pi}_k = \frac{1}{N} \sum_n I(z_n = k)$, $\hat{\mu}_k = \frac{\sum_n I(z_n=k) x_n}{\sum_n I(z_n=k)}$ and $\hat{\Sigma}_k = \frac{1}{\sum_n I(z_n=k)} \sum_n I(z_n = k)(x_n - \hat{\mu}_k)(x_n - \hat{\mu}_k)^T$

**Hard Expectation-Maximization Algorithm**: given dataset $\mathcal{D} = \{x_1, \dots, x_N\}$, **initialize the parameters** $\theta^{(0)} = \{\pi_k, \mu_k, \Sigma_k\}_{k \in [1,\dots,K]}$, at **each iteration do E-step** (**predict the most likely class for each data point**) $z_n^{(t)} = argmax_z p(z|x_n, \theta^{(t-1)}) = argmax_z p(z|\theta^{(t-1)}) p(x_n|z, \theta^{(t-1)}) = argmax_z \pi_z \mathcal{N}(x_n|\mu_z, \Sigma_z)$, now we **have the complete labeled data for that iteration** $\mathcal{D}_c^{(t)} = \{(x_n, z_n^{(t)})\}_{n \in [1,\dots,N]}$ so we do the **M-step** (**compute MLE of** $\theta$ **using newly computed labels**) $\theta^{(t)} = argmax_\theta p(\mathcal{D}_c^{(t)}|\theta) = argmax_\theta \sum_n^N log(\pi_{z_n^{(t)}} \mathcal{N}(x_n|\mu_{z_n^{(t)}}, \Sigma_{z_n^{(t)}}))$. **K-Means is a special case of Hard-EM where optimization is over the means** $\mu_k$ **with fixed uniform weights** ($\pi_k = 1/K \ \forall k$) **and fixed identical spherical covariances** ($\Sigma_k = \sigma^2 I \ \forall k$). **Hard EM** is way more flexible (**handles clusters of different shapes and sizes**), but it **requires more iterations to converge** and **iterations are expensive** ($O(NKD^3)$). **K-Means is often used to initialize EM** for faster convergence, like K-Means it **assigns labels to points even if uncertain (HARD)**



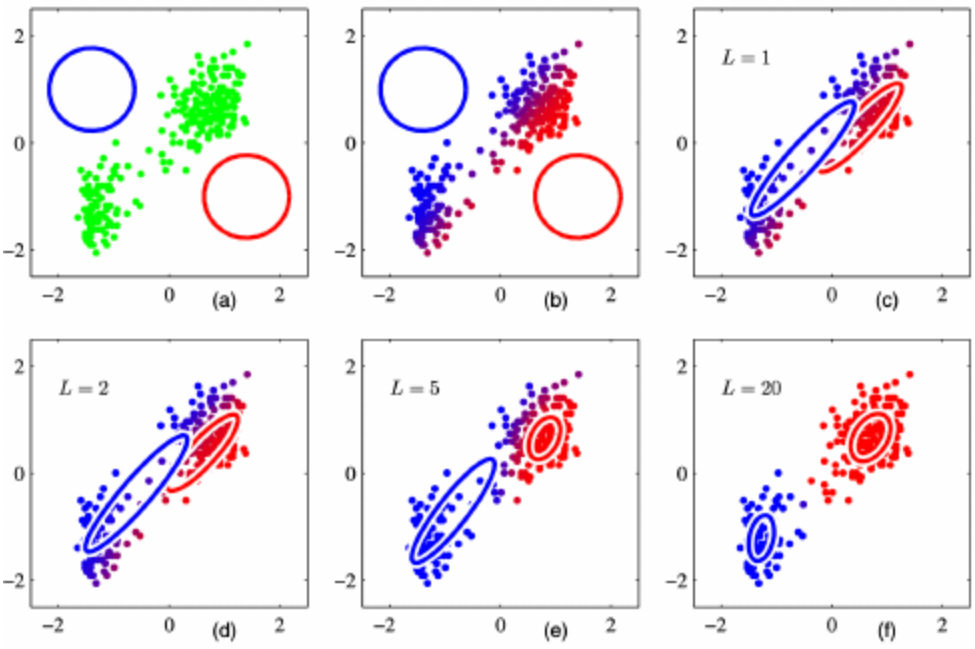



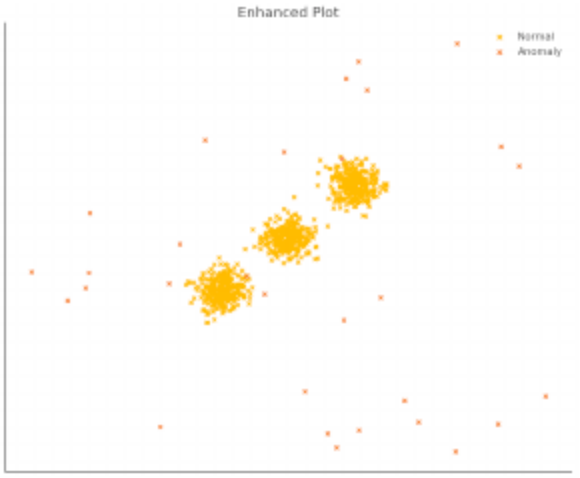



**Soft MLE with GMM**: **assigns to each point a (posterior) probability of belonging to each cluster** (soft), given parameters $\theta = \{\pi_k, \mu_k, \Sigma_k\}_{k\in[1,\dots,K]}$ we **model distributions over latent variable** $z$ (that defines the cluster we want to assing a point to, IT'S LATENT BECAUSE WE DONT KNOW IT SINCE ITS UNSUPERVISED LEARNING) $q_k(x_n) = p(z_n = k|x_n, \theta) = \frac{\pi_k \mathcal{N}(x_n|\mu_k,\Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_n|\mu_j,\Sigma_j)}$, **captures uncertainty**. Applying MLE we get that in theory $\pi_k^* = \frac{1}{N}\sum_n q_k(x_n)$, $\mu_k^* = \frac{\sum_n q_k(x_n)x_n}{\sum_n q_k(x_n)}$ and $\Sigma_k^* = \frac{1}{\sum_n q_k(x_n)} \sum_n q_k(x_n)(x_n - \mu_k^*)(x_n - \mu_k^*)^T$ but the issue is that **these closed-form equations** that calculate the optimal paramters $\theta = \{\pi_k, \mu_k, \Sigma_k\}_{k\in[1,\dots,K]}$ **use** $q_k(x_n)$ **which also relies on the optimal parameters** (**chicken and egg**), so **WE CANNOT USE THE CLOSED FORM SOLUTION FOR MLE**

**Soft EM Algorithm for GMMs**: **solves the previous chicken-and-egg problem**, **initialize the parameters** $\theta^{(0)} = \{\pi_k, \mu_k, \Sigma_k\}_{k\in[1,\dots,K]}$, at **each iteration do E-step** (**predict the most likely class for each data point**) $q_k^{(t)}(x_n) = p(z_n = k|x_n, \theta^{(t-1)}) = \frac{\pi_k \mathcal{N}(x_n|\mu_k^{(t-1)},\Sigma_k^{(t-1)})}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_n|\mu_j^{(t-1)},\Sigma_j^{(t-1)})}$, now we **have** $q_k^{(t)}(x_n)$ so we do the **M-step** (**compute MLE of** $\theta$) $\pi_k^{(t)} = \frac{1}{N}\sum_n q_k^{(t)}(x_n)$, $\mu_k^{(t)} = \frac{\sum_n q_k^{(t)}(x_n)x_n}{\sum_n q_k^{(t)}(x_n)}$ and $\Sigma_k^{(t)} = \frac{1}{\sum_n q_k^{(t)}(x_n)} \sum_n q_k^{(t)}(x_n)(x_n - \mu_k^{(t)})(x_n - \mu_k^{(t)})^T$




**Anomaly Detection with GMMs**: after training a GMM, we **compute the likelihood** $p(x)$ **for any new data point that we want to check if it's an anomaly**

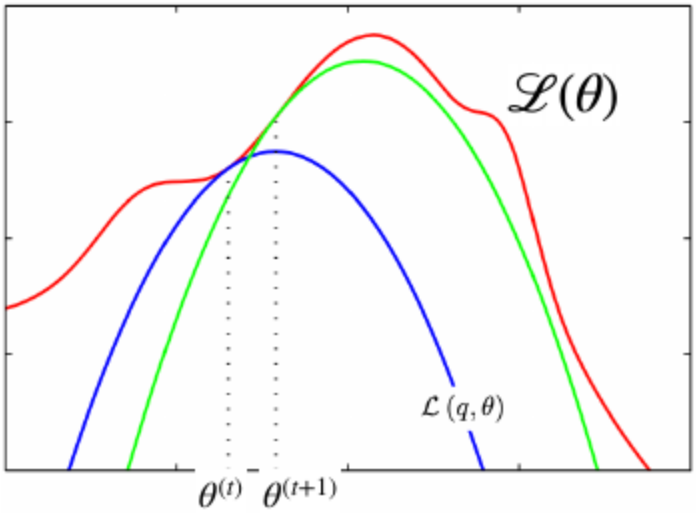



Likelihood-based anomaly detection: red points are flagged as anomalies based on the GMM threshold

$$p(\mathbf{x}) = \sum_{k=1}^{K} \frac{\pi_k \exp\left(-\frac{1}{2}(\mathbf{x}-\mu_k)^T \Sigma_k^{-1}(\mathbf{x}-\mu_k)\right)}{(2\pi)^{d/2}\,|\Sigma_k|^{1/2}}$$

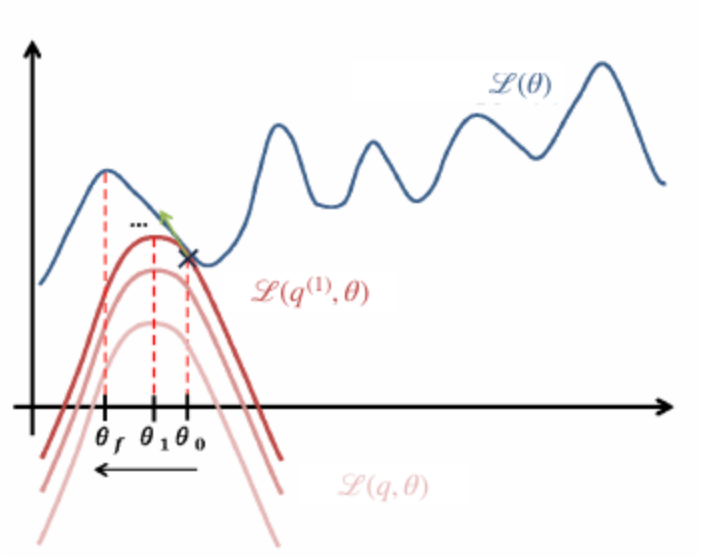
**If** $p(x) \leq \tau \implies$ **anomaly**

**EM Algorithm in General**: we assume that $(x, z)$ are random variables where $x$ **is observed** (our data) and $z$ **is not observed** (e.g. cluster membership), their **joint distribution under a parametric model is given by** $p(x, z|\theta)$. Given a dataset $X = (x_1, \ldots, x_N)$ the **goal is to maximize the log likelihood function defined in terms of the latent variables** $Z = (z_1, \ldots, z_N)$ $\mathcal{L}(\theta) = log(p(X|\theta)) = log(\sum_Z p(X, Z|\theta)) = \sum_{n=1}^{N} log \sum_{z_n} p(x_n, z_n|\theta)$. To **make it soft we introduce a distribution** $q(Z)$ **over the latent variables**, $q(Z) \geq 0$ and $\sum_Z q(Z) = 1$ for all $q(Z)$ , we get $\mathcal{L}(\theta) = log(p(X|\theta)) = \sum_Z q(Z) log(p(X|\theta)) = \sum_Z q(Z) log(\frac{p(X|\theta)p(Z|X,\theta)q(Z)}{p(Z|X,\theta)q(Z)}) = \sum_Z q(Z) log(\frac{p(X|Z,\theta)}{q(Z)}) + \sum_Z q(Z) log(\frac{q(Z)}{p(Z|X,\theta)})$ we assign $\mathcal{L}(q, \theta) = \sum_Z q(Z) log(\frac{p(X|Z,\theta)}{q(Z)})$ and $KL(q||p) = \sum_Z q(Z) log(\frac{q(Z)}{p(Z|X,\theta)})$ note that $\forall q, p\ KL(q||p) \geq 0$ (equality iff $q = p$) so $\forall q\ \mathcal{L}(\theta) \geq \mathcal{L}(q, \theta)$ (equality iff $q(Z) = p(Z|X, \theta)$) so, using $q$, we introduced an auxilary function $\mathcal{L}(q, \theta)$ that is alway below $\mathcal{L}(\theta)$ (**upper bound**). **Make the EM Algorithm just maximize** $\mathcal{L}(q, \theta)$, **coordinate ascent algorithm** with E-step $q^{(t+1)} = argmax_q \mathcal{L}(q, \theta^{(t)}) = argmax_q log(p(X|\theta^{(t)})) - KL(q||p(Z|X, \theta^{(t)}))$ (the last part ($KL$) is the only one dependent on $q$ and it's the distance between $\mathcal{L}(\theta)$ and $\mathcal{L}(q, \theta)$ and it's 0 iff $q = p(Z|X, \theta^{(t)})$ so $q^{(t+1)} = p(Z|X, \theta^{(t)})$ to have $\mathcal{L}(\theta^{(t)}) = \mathcal{L}(q^{(t+1)}, \theta^{(t)})$) and M-step $\theta^{(t+1)} = argmax_\theta \mathcal{L}(q^{(t+1)}, \theta) = \sum_Z q^{(t+1)}(Z) log \frac{p(X,Z|\theta)}{q^{(t+1)}(Z)} = \sum_Z q^{(t+1)}(Z) log\ p(X, Z|\theta) - \sum_Z q^{(t+1)}(Z) log\ q^{(t+1)}(Z) = \mathbb{E}_{Z\sim q^{(t+1)}}[log\ p(X, Z|\theta)] + H(q)$ (**first part is the only dependent on** $\theta$ **and it maximizes the expected complete log likelihood**)




The EM algorithm involves alternately computing a lower bound on the log likelihood for the current parameter values (E-step) and then maximizing this bound to obtain the new parameter values (M-step).

**Final EM Recipe**: the goal is **maximize the incomplete likelihood** $\mathcal{L}(\theta) = log(p(X|\theta))$ **by maximizing its lower bound** $\mathcal{L}(q,\theta) = log(\frac{p(X,Z|\theta)}{q(Z)})$, the **E-step consists of computing** $p(Z|X,\theta^{(t)})$ (corresponding to $q^{(t+1)} = argmax_q \mathcal{L}(q,\theta^{(t)})$), **writing the complete likelihood** $log\ p(X,Z|\theta)$ and **calculating the conditional expected value of the complete log likelihood** $\mathbb{E}_Z[log\ p(X,Z|\theta)|X,\theta^{(t)}]$, the **M-step consists of maximizing** $\theta^{(t+1)} = argmax_\theta \mathbb{E}_Z[log\ p(X,Z|\theta)|X,\theta^{(t)}]$ (corresponding to $\theta^{(t+1)} = argmax_\theta \mathcal{L}(q^{(t+1)},\theta)$). It's an **ascent algorithm** ($\mathcal{L}(\theta^{(t)}) \leq \mathcal{L}(\theta^{(t+1)})$), the **sequence of log-likelihoods converges** (under suitable conditions), **does not converge to a global maximum** because it's **not convex**




**Picking K in EM**: picking $K = N$ maximizes the log-likelihood but it's highly overfitting (a Gaussian with 0 variance for each data point), instead we use **Cross-Validation** to **select a** $K$ **that maximizes the log-likelihood on the validation dataset** (this **was not possible with K-Means**)

**Initialization in EM**: for **cluster weights** ($\pi_k$) typically use **uniform distribution**, for **means** ($\mu_k$) randomly initialize or **use K-means++**, for **covariance matrices** ($\Sigma_k$) use **empirical variances in individual feature dimensions**

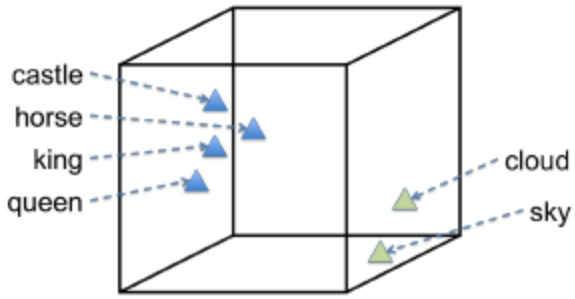
# 19 - Matrix Factorization

**Predicting movie ratings by users**: interesting for **recommendation systems**, given items (movies) $d = 1, \dots, D$ and users $n = 1, \dots, N$ we define $X$ to be the $D \times N$ **matrix containing all rating entries**, **most ratings are missing** in the matrix (users review only a few movies). We aim to find $W, Z \; s.t. \; X \approx WZ^T$ to be able to **explain each rating with a numerical representation of the corresponding item and user** (inner product of the **item feature vector** (row of $W$) **with the user feature vector** (row of $Z$)). The objective $min_{W,Z}\mathcal{L}(W, Z) = \frac{1}{2}\sum_{(d,n)\in\Omega}[x_{dn} - (WZ^T)_{dn}]^2 + \frac{\lambda_w}{2}||W||^2_{Frob} + \frac{\lambda_z}{2}||Z||^2_{Frob}$ where $W \in \mathbb{R}^{D\times K}$ and $Z \in \mathbb{R}^{N\times K}$ are tall matrices ($K < D, N$) and the **last two terms are regularizers**. The set $\Omega \subseteq [1, \dots, D] \times [1, \dots, N]$ collects the indices of the observed ratings of $X$. We need to pick a $K$ (**feature vectors length**, **number of latent features**), **large $K$ facilitates overfitting**. The **cost is not jointly convex**, and the **model is not identifiable** but **we can incorporate side-information** about users and items. The training objective is just a **sum over $|\Omega|$ terms**, so if we want to **apply SGD** we need to **derive the stochastic gradient for $W, Z$ given one observed rating** $(d, n) \in \Omega$. For $W$: $\frac{\partial}{\partial w_{d',k}} f_{d,n}(W, Z) = \{-[x_{dn} - (WZ^T)_{dn}]z_{n,k} \; if \; d' = d$ otherwise $0$. For $Z$: $\frac{\partial}{\partial z_{n',k}} f_{d,n}(W, Z) = \{-[x_{dn} - (WZ^T)_{dn}]w_{d,k} \; if \; n' = n$ otherwise $0$

**Alternating Least-Squares (ALS)**: for simplicity let's assume that there are no missing entries ($\Omega = [1, \dots, D] \times [1, \dots, N]$), then $\frac{1}{2}\sum_{d=1}^{D}\sum_{n=1}^{N}[x_{dn} - (WZ^T)_{dn}]^2 = \frac{1}{2}||X - WZ^T||^2_{Frob}$. We can use **coordinate descent to minimize the cost plus regularizer, we first minimize w.r.t $Z$ for fixed $W$ and then minimize $W$ given $Z$** (like EM), $Z^T = (W^TW + \lambda_z I_K)^{-1}W^TX$, $W^T = (Z^TZ + \lambda_w I_K)^{-1}Z^TX^T$ (doing **ridge regression**). What if we have missing entries?

# 20 - Large Language Models

**Representation learning for text**: for **each word $w_i$ find a mapping/embedding $\mathbf{w}_i \in \mathbb{R}^K$** that **captures semantics** of the word, having **good feature representations benefits all ML** applications

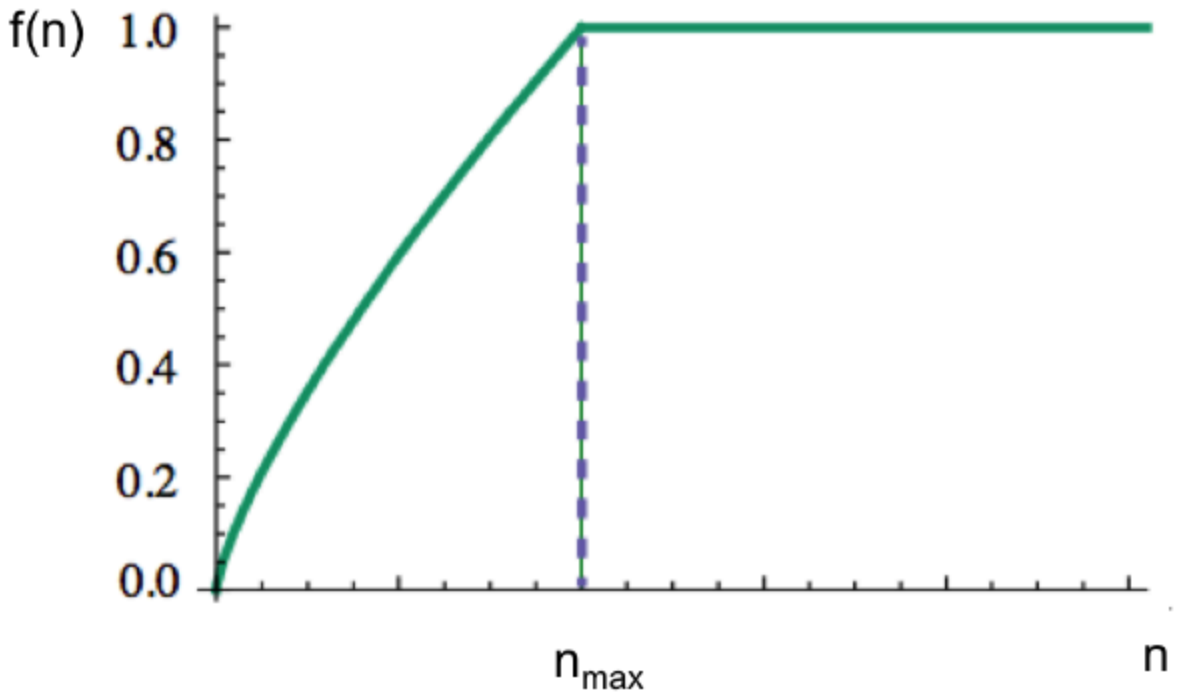


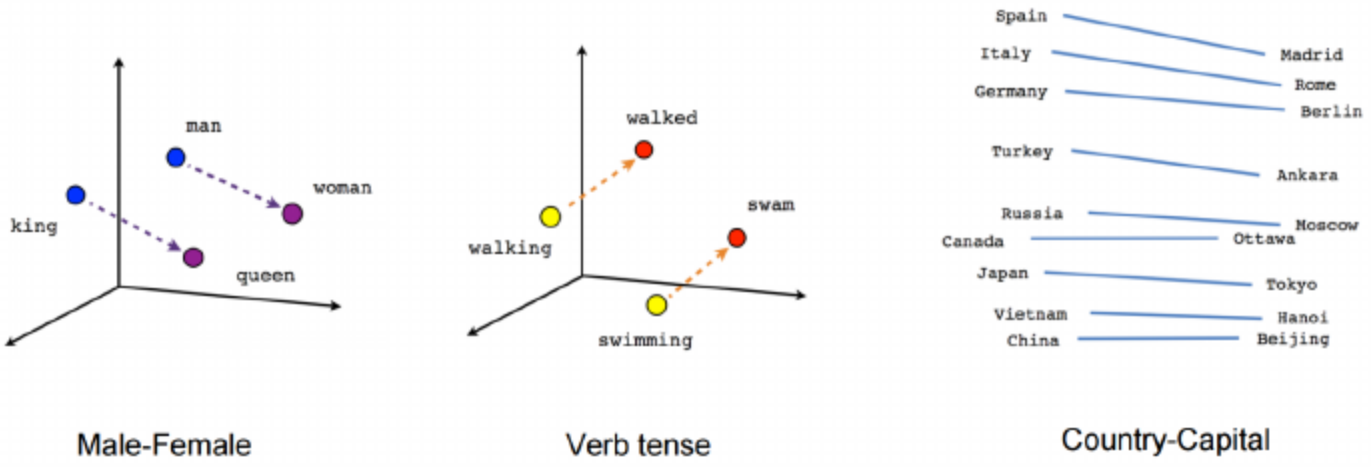

**Co-occurrence counts**: can **represent a big corpus of un-labeled text**, $n_{ij} = \#$**contexts where word $w_i$ occurs together with word $w_j$**, needs **definition of context** (e.g. document) and

**vocabulary** $\mathcal{V} = \{w_1, \ldots, w_D\}$, $D$ words $w_d$ and $N$ context words $w_n$ (usually $N = D$). **Co-occurrence counts** $n_{ij}$ **form a very sparse** $D \times N$ **matrix**

**Matrix Factorization of Co-occurrence counts**: typically **log of actual counts** are used $x_{dn} = log(n_{dn})\ or\ log(1 + n_{dn})$, so we aim to find $W, Z\ s.t.\ X \approx WZ^T$. Each pair of words $(w_d, w_n)$ we try to **explain their co-occurence count by a numerical representation of the words** (inner product of the two feature vectors $W_d, Z_n$). The objective is $min_{W,Z}\mathcal{L}(W, Z) = \frac{1}{2}\sum_{(d,n)\in\Omega}[x_{dn} - (WZ^T)_{dn}]^2 + \frac{\lambda_w}{2}||W||^2_{Frob} + \frac{\lambda_z}{2}||Z||^2_{Frob}$ where $W \in \mathbb{R}^{D\times K}$ and $Z \in \mathbb{R}^{N\times K}$ are tall matrices ($K < D, N$) and the **last two terms are regularizers**. The set $\Omega \subseteq [1, \ldots, D] \times [1, \ldots, N]$ collects the indices of non-zeros of the count matrix $X$. Each **row of those matrices forms a representation of a word** ($W$) or a context word ($Z$)

**GloVe**: **variant of word2vec**, **weights** $f_{dn}$ **give importance to each entry** (choosing 1 as weight is okay) $f_{dn} = min\{1, (n_{dn}/n_{max})^\alpha\},\ \alpha \in [0, 1]$, training is done with **SGD or ALS**



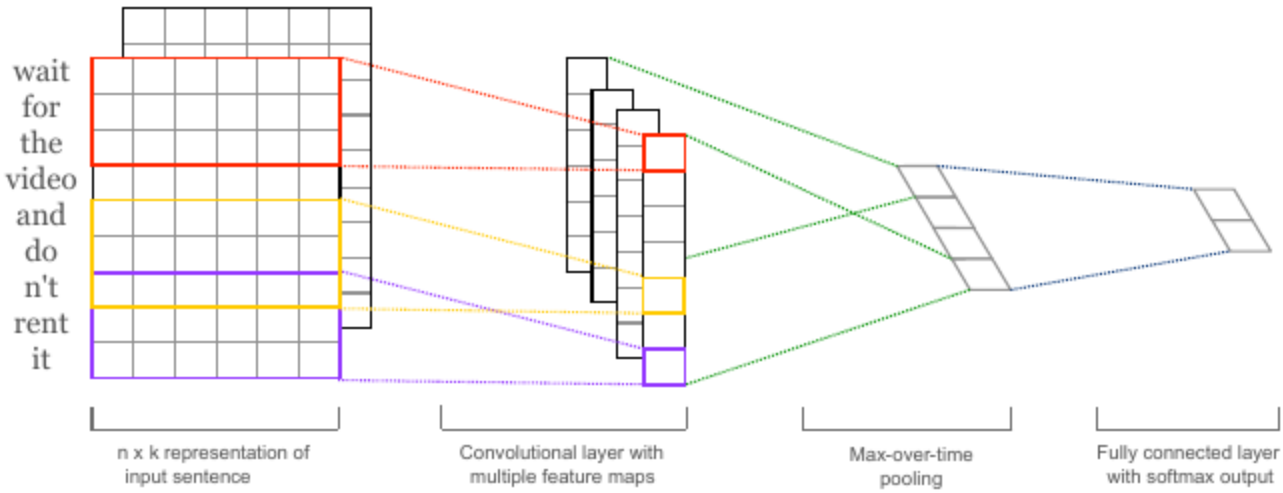



Male-Female | Verb tense | Country-Capital

**Skip-Gram Model**: part of original word2vec. uses **binary classification with logistic regression to separate real word pairs** $(w_d, w_n)$ **from fake word pairs**. Tries to **maximize the inner product for real pairs** (making their vectors point in the same direction), like how **matrix factorization** works. Given a word $w_d$, the context word $w_n$ is **real if it appears together in a context window of size 5**, or **fake if it's any word** $w_{n'}$ **sampled randomly** (**negative sampling**, creating samples that represent noise, **making the model learn how to distinguish noise** from real data)

**FastText**: **matrix factorization to learn document/sentence representations**, supervised data $(s_n, y_n)$ is given. Given a **sentence** $s_n = (w_1, \ldots, w_m)$, let $\mathbf{x}_n \in \mathbb{R}^{|\mathcal{V}|}$ be the **bag-of-words representation** of the sentence, the **objective** is $min_{W,Z}\mathcal{L}(W, Z) = \sum_{s_n} f(y_n(WZ^T)\mathbf{x}_n)$ where $W \in \mathbb{R}^{1\times K}$, $Z \in \mathbb{R}^{|\mathcal{V}|\times K}$ are the **label vector and the word embeddings matrix** and $f$ is a **linear classifier loss function**, $y_n \in \{-1, +1\}$ is the **classification label for sentence** $\mathbf{x}_n$, **not convex because we are optimizing for** $W$ **and** $Z$ **at the same time**

**Supervised representative learning of sentences/documents**: **any neural network architecture** (e.g. **CNNs**), **transformers** or **matrix factorization** (e.g. **fasttext**), e.g. **tweet sentiment classification**

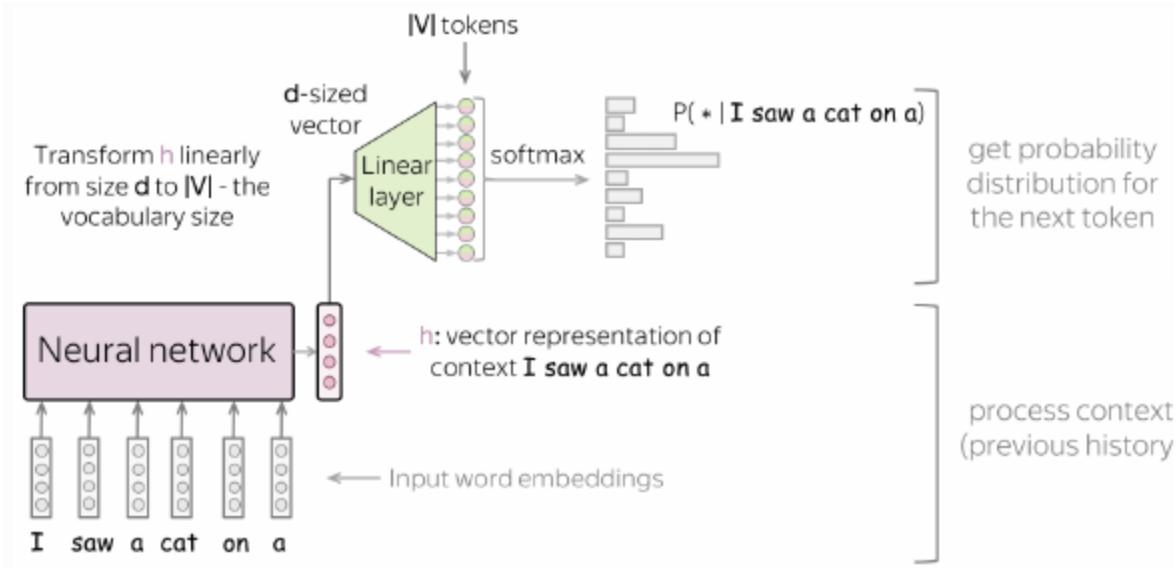


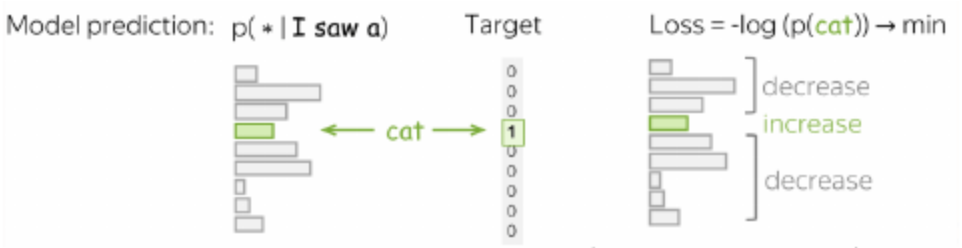
**Unsupervised representative learning of sentences/documents**: **simplest** approach is **adding or averaging** (**fixed, given**) **word vectors**, **optimized** way is to **train word vectors so adding/averaging works well**, **modern** approach is to **direct unsupervised training using sentences as primary unit instead of words** (**no bag-of-words**), this way we **can understand context**

**Language Models**: **selfsupervised training**, classifier trained to **predict the continuation** (next word) **of given text**, used with **multi-classification** (#classes = vocabulary size) or **binary classification** (predict if the **next word is real or fake** (e.g. skip-gram)). They **describe distributions over sequences of text** ($p(\text{I saw a cat}) = p(x_1, \ldots, x_n)$), **simplest factorization is predicting the next word/token** $p(x_t|x_1, \ldots, x_{t-1}) \cdot p(x_{t-1}|x_1, \ldots, x_{t-2}) \cdot \cdots \cdot p(x_2|x_1) \cdot p(x_1)$ ($p(\text{cat} \mid \text{I saw a}) = 0.0002$)

**Next-Token Prediction**: can be viewed as **massively multi-task learning** and it's **extremely general**

| Task | Example sentence in pre-training |
|---|---|
| Grammar | In my free time, I like to {run, banana} |
| Lexical Semantics | I went to the zoo to see giraffes, lions, and {zebras, spoon} |
| World Knowledge | The capital of Denmark is {Copenhagen, London} |
| Sentiment Analysis | Movie review: I was engaged and on the edge of my seat the whole time. The movie was {good, bad} |
| Translation | The word for "pretty" in Spanish is {bonita, hola} |
| Math | First grade arithmetic exam: 3 + 8 + 4 = {15, 11} |
| Coding | def fibonacci(n): [...] |






**Maximize full text likelihood:** $\max \prod_t p(x_t \mid x_{1:t-1}) = \min \left( -\sum_t \log p(x_t \mid x_{1:t-1}) \right)$

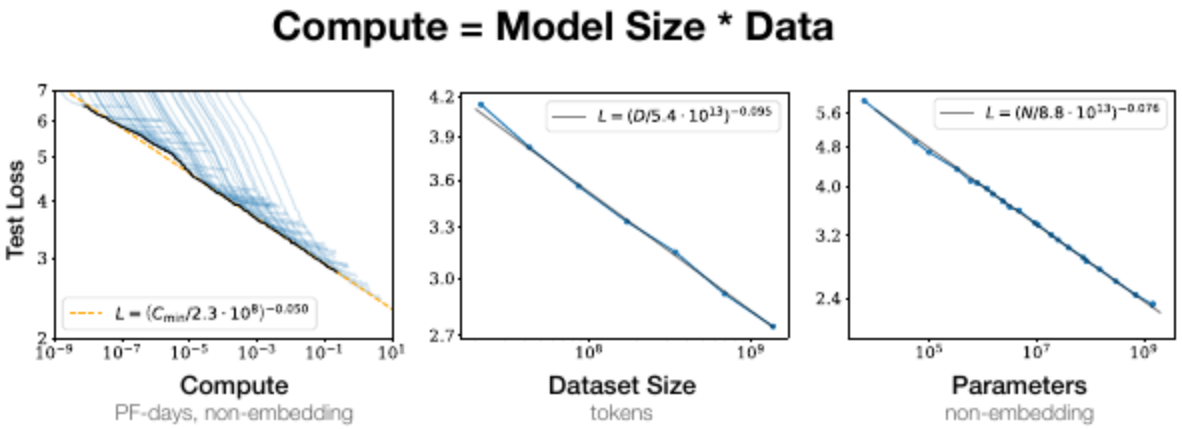
**Autoregressive inference**: given context (sequence of tokens) output a sentence, **forward pass** through the model, **obtain probabilities for the next token**, **select one** by sample or argmax, **add it to the context and repeat**

**Pretraining vs Posttraining**:

| | Pretraining | Posttraining |
|---|---|---|
| **Data Quantity** | Massive, ~the internet (Bilions-trillions of words) | Small, millions of examples |
| **Data Quality** | Low(er) | High |
| **Data Source** | Webcrawls, Papers, Textbooks, Github, … | (Often) Human |
| **Goal** | General Learning, Knowledge | Make model usable, give specific skills + character, alignment, … |

**Pretraining**: produces a **general purpose model trained on massive quantities of text**, challenges are **maximizing coverage and diversity** of the data, **maximizing quality and correctness** of the model, **finding right measures of data quality** and **efficient processing of trillions of tokens** (TBs of data). Data is from **Common Crawl** (starting point), **Open Source Code** (GitHub), **Curated** (specific websites and books) or **Synthetic** (**generated by existing models**, used in distillations)

**Power of Scale**:

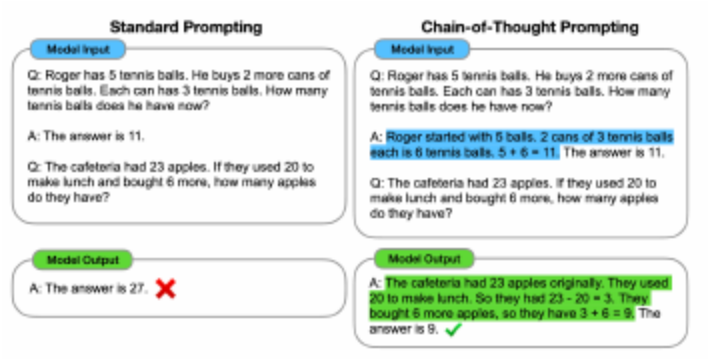


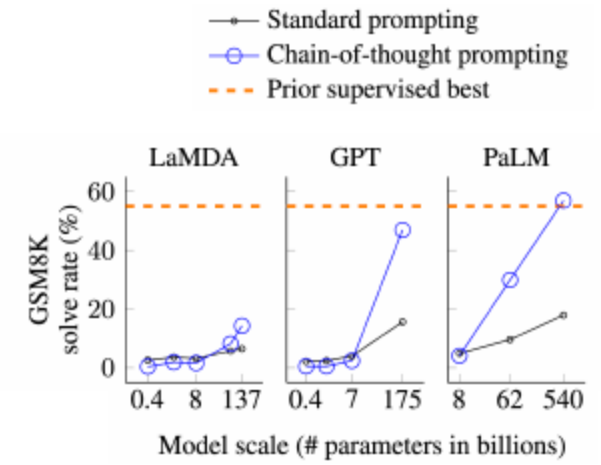

**Emergent Abilities of LLMs**:

- **GPT1** (2018), **decoder-only** transformer with **12 layers** and **117M parameters** trained on **BooksCorpus** (7k books, **4.6GB** text), for **each task a new dataset and finetuning was needed**, **no generalization to new-tasks**
- **GPT2** (2019), **decoder-only** transformer with **48 layers** and **1.5B parameters** trained on **WebText** (40GB), **fine-tuning not needed** anymore, **pre-trained alone achieves SOTA for all tasks**
- **GPT3** (2020), **decoder-only** transformer with 96 **layers** and **175B parameters** trained on **WebText** (600GB), **specify a new task by adding examples in the prompt**

**In-Context Learning**: **zero-shot** if with **only a description of the task** and no examples it **tries doing the new task**, **one-shot** if we also supply an **example**, **few-shot** if we supply **multiple examples**. The model is able to **fulfill the task that it has never seen before just from the prompt**. **Cannot solve every task** (especially the ones requiring **multi-step reasoning**)

**Chain-of-Thought (CoT) prompting**: **showing some examples of reasoning steps** can help solve more complex tasks

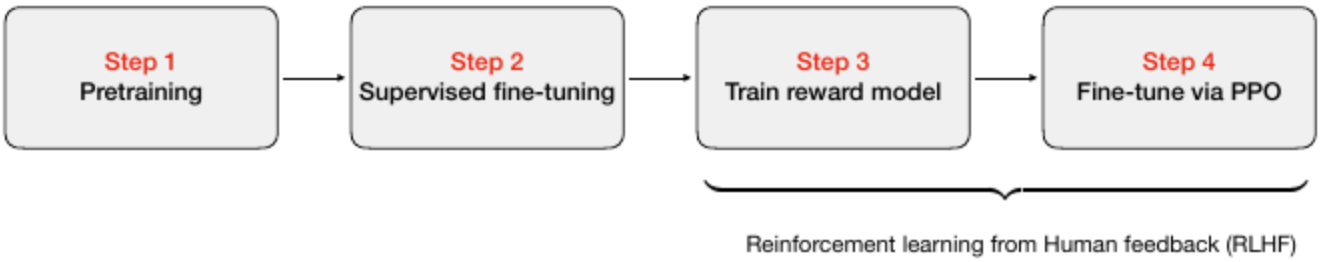



**Posttraining**: **after pretraining models are not aligned with user's intent or policies** (might not follow user instructions or generate harmful responses). Do **Supervised Finetuning** on **collected**

**demonstrations of how the model will be used/we want it to behave**, very **expensive** and **penalizes all mistakes equally**. **Instead of having humans manually label** answers (not reliable and **noisy**), we **ask for pairwise comparison** (order a set of **outputs from best to worst**, this **outputs are generated from a sampled prompt**), we **use this data to train a reward model to simulate human preferences**. **Reward model** is a function $R_\phi : X_{prompt} \times X_{response} \to \mathbb{R}$, for a given prompt $x_{prompt}$ and a pair of responses $x_a, x_b$ where human annotators prefer $x_a$ over $x_b$, loss is $L(\omega) = \mathbb{E}_{(x_{prompt}, x_a, x_b) \sim \mathcal{D}}[-log(\sigma(R_\phi(x_{prompt}, x_a) - R_\phi(x_{prompt}, x_b)))]$ where $\sigma(z)$ is the sigmoid. The **reward model learns to give scores that reflect human's pairwise comparisons**. Given a pre-trained LLM $\pi_\theta$ and trained reward model $R_\phi$, we want to **fine-tune the LLM to maximize the expected reward**: $\theta^* = argmax_\theta \mathbb{E}_{x_{prompt} \sim P_{data}, x \sim \phi_\theta}[R_\phi(x_{prompt}, x)] = argmax_\theta J(\theta)$, we optimize by **Policy Gradient** $\nabla_\theta J(\theta) = \mathbb{E}\left[R_\phi(x_{prompt}, x) \cdot \nabla_\theta \log \phi_\theta(x | x_{prompt})\right]$, since this is **Reinforcement Learning** user responses $x$ are **sampled on the fly** from $\phi_\theta$, **directly computing the gradient of the expected reward is challenging** because the **reward expectation is taken over the policy distribution** $\phi_\theta$, which **depends on the parameters** $\theta$ **that we are optimizing** with the new updated gradient (solved with **log-derivative trick**), so the distribution changes as we update it

**Proximal Policy Optimization (PPO)**: **penalty-based constraints applied to the Policy Optimization/Gradient so that updates are not too large** (and pretraining is not forgotten)




**Evaluations**: **loss/accuracy is meaningless across models** (as they use different tokenizers and data), so **challenges** (**open-ended diverse tasks** to solve, how do you automate **assigning a score**?, sensitive to prompts) and **benchmarks** (**assessing knowledge and abilities**, e.g. Measuring Massive Multitask Language Understanding (**MMLU, multiple choice**), AI2 Reasoning Challenge (**ARC, question answering**)) are used. **Arenas of LLMs evaluating each other**'s answers is also used (also humans blindly evaluating models)

# 21 - Self-supervised Learning

**Transfer Learning**: **fine-tune an existing model trained on a related task** to give it a good internal representation of the input data. Achieves **high performance on the downstream task with**

**significantly few labelled data** (which is **expensive and limited**), you **still need labelled data for the base task** on which the existing model is trained

**Self-Supervised Learning**: **use input itself to generate supervisory signals**, the **base task is an artifical pretext task** that requires **learning useful features and structure in the input** data. In the **pretext task we want to learn a model** $f : x_{in} \rightarrow x_{out}$ **where we use a transformation** $g : x \rightarrow (x_{in}, x_{out})$ **to create an input and a target/label** from an unlabelled datapoint $x \in X$ (often involves some randomness). Some examples of pretext tasks for text are **Masked Language Modelling** and **Next Token Prediction**, for images it's **Predicting Image Rotation**, **Image Colorization** and **Predicting Relative Patch Placement**

**Masked Language Modelling (MLM)**: learn to **predict hidden words** (e.g. `She [MASK] her coffee` → `She drank her coffee`), forces the model to **learn a comprehensive representation of the language** that is very **useful for many downstream tasks**

**BERT**: Bidirectional Encoder Representations from Transformers, family of language **models based on the encoder-only transformer architecture and trained on masked language modelling**. Training **inputs usually are two sentences and special tokens** (e.g. `[CLS] The cat is sleeping. [SEP] It's on the sofa. [SEP]`) like `[CLS]` (special classification token) and `[SEP]` (separates sentences). Around **15% of the standard tokens are selected to be replaced by** `[MASK]`, selected tokens are occasionally replaced by a random word or not replaced at all to **reduce distribution shift for downstream tasks that do not have** `[MASK]`. **Positional and segment encodings are added** to the token embeddings. The training **objective** is to **predict the original token for each replaced or masked token**, uses **softmax cross-entropy loss** over all possible tokens. **For the** `[CLS]` **token predict whether the second sentence immediately follow the first sentence or not in the original context** (first and second sentence could be put together from any source), this works because through Self-Attention **the** `[CLS]` **token by the end of the iteration has a representation of the relationship between the two sentences** (can also be used for **sentiment analysis**). The **structure allows fine-tuning for a variety of downstream tasks**. Can be used for **word level classification** (e.g. **named entity recognition**) using the **output of each token** or **extracting a relevant passage** (e.g. **search**), **TOKENS ARE PREDICTED IN PARALLEL** (SO WHEN MASKED TOKENS ARE IN A SENTENCE THERE IS NO ORDER TO SOLVING THEM)

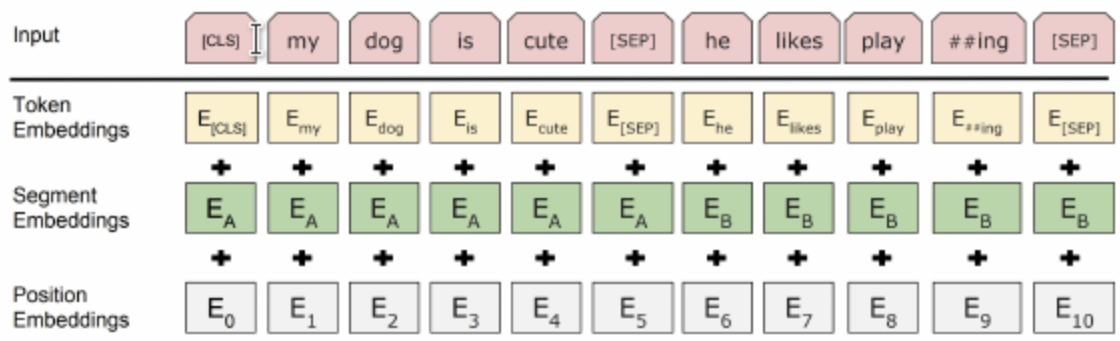
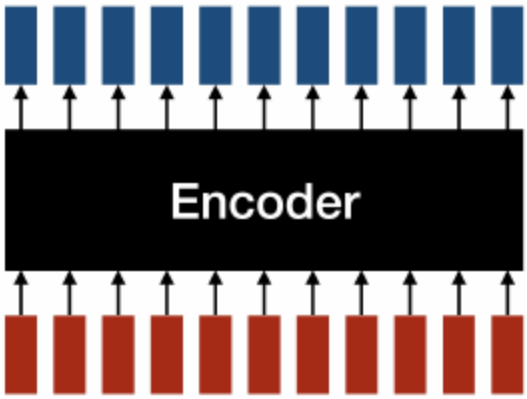



Figure from the original BERT paper: https://arxiv.org/abs/1810.04805

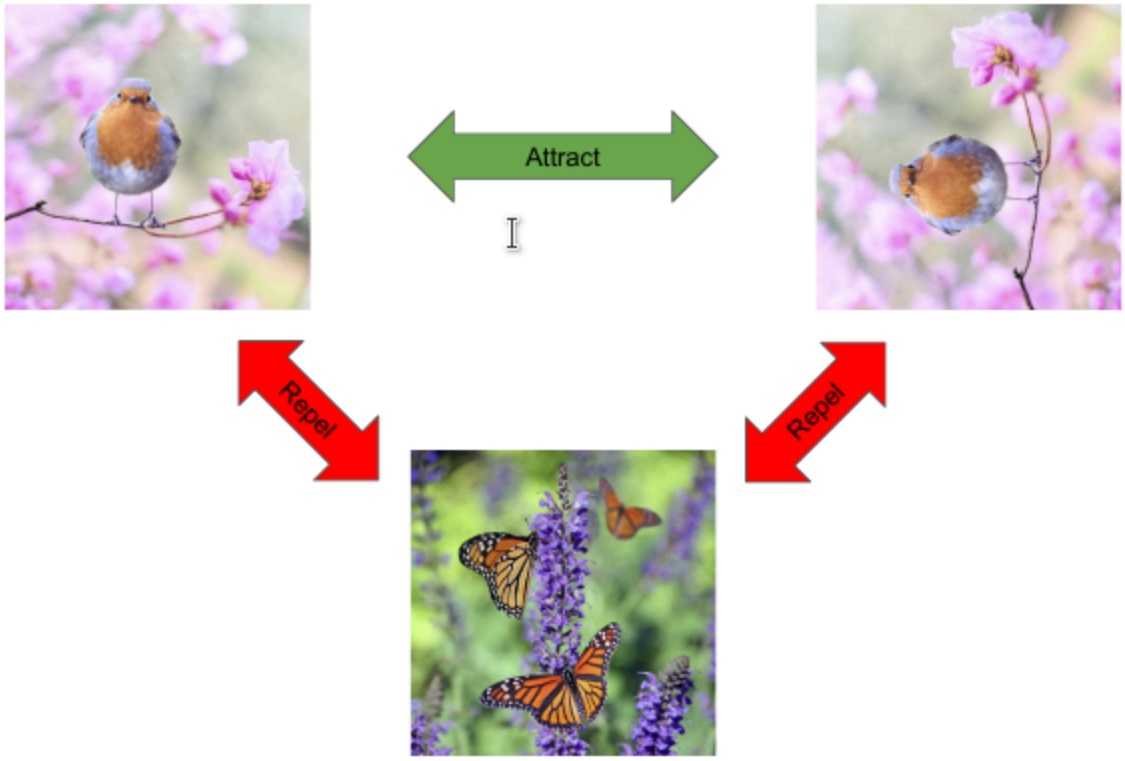
**Encoder**: **sees the whole sequence at once, every token can generally attend to every other token** (contribute to his **context specific meaning**, Self-Attention), generates a fixed sized output (one token per input token)



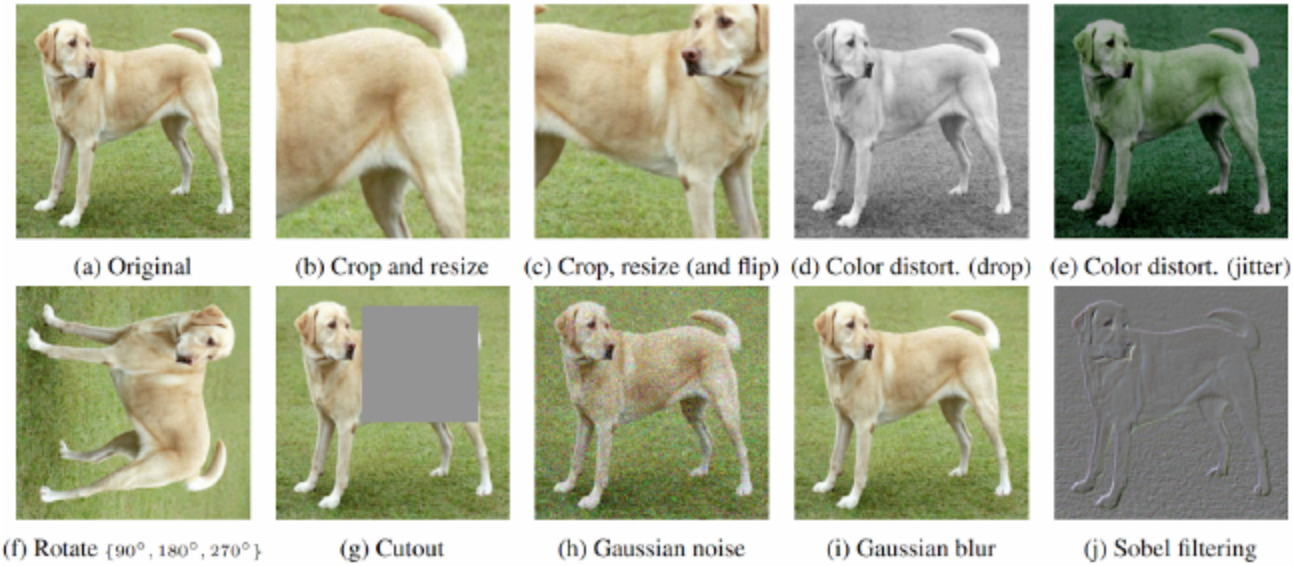

**Next Token Prediction**: what **LLMs** are based on, similar to MLM but it's **causal** (auto-regressive), we **can only see previous words** (makes it better for **generating arbitrary length responses**)

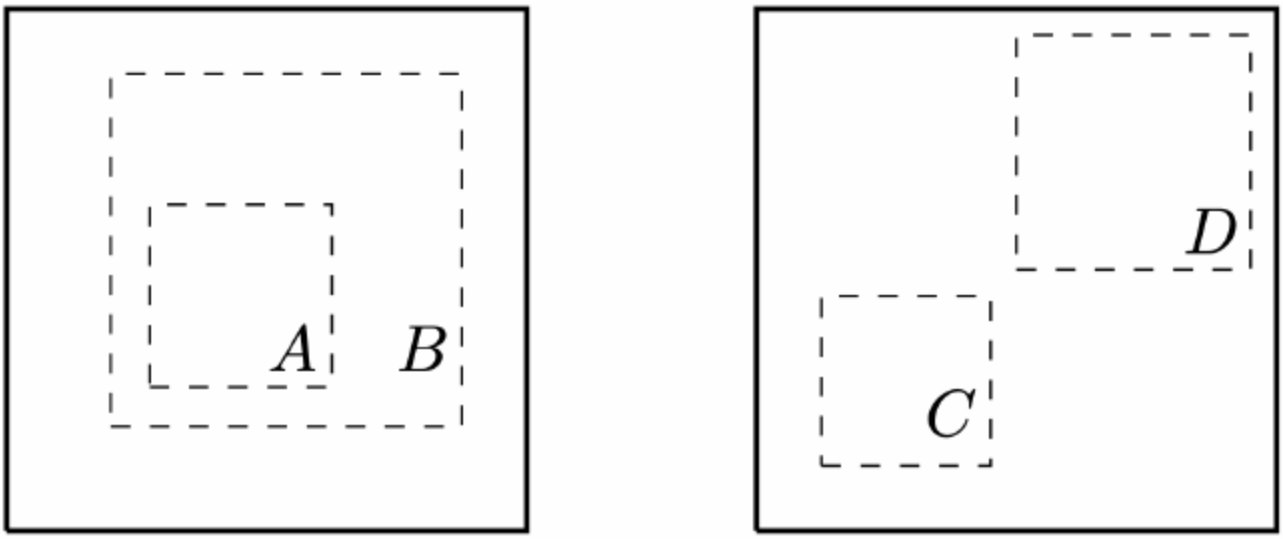
**Joint Embedding Methods**: learn an **encoder that is invariant to certain transformations of the data**, rather than learning a model $f : x_{in} \rightarrow x_{out}$, learn $f : X \rightarrow \mathbb{R}^d \; s.t \; f(x_1) \approx f(x_2)$ where $g : x \rightarrow (x_1, x_2)$ is a **transformation that creates two views from the same input** datapoint $x \in X$ ($g$ usually **random function that applies multiple data augmentations**, each with certain probability). The ****problem is that the **algorithm could learn that if $f$ is constant then it's also invariant** (which is something we dont want), **solved with Contrastive Learning or non-contrastive methods with regularization** (e.g. **BYOL** that uses **two encoders** $f_1, f_2$ where $f_2$ is the **exponential moving average** and force $f_1(x_1) \approx f_2(x_2)$)

**Contrastive learning**: **push away representations of unrelated views from different data points**, given a **data point $x$, a different view of it $x^+$ and a negative view $x^-$ from another datapoint** $x^{'} \neq x$ we want $s(f(x), f(x^+)) > s(f(x), f(x^-))$ where $s$ **quantifies the similarity of two embeddings**

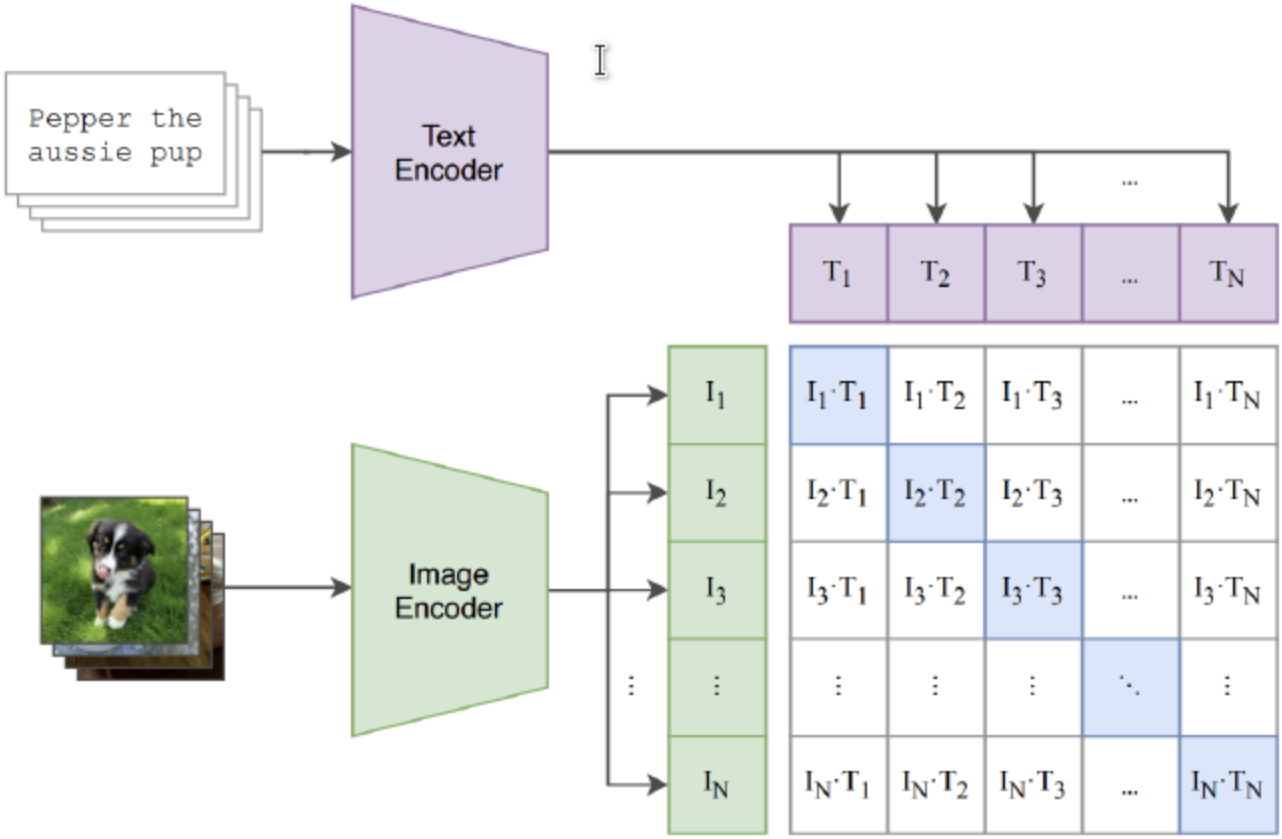



**SimCLR**: **contrastive learning framework with optimized choices**, uses a **classification objective with $N$ negative samples** $L(x) = -\mathbb{E}[log \frac{e^{s(f(x),f(x^+))}}{e^{s(f(x),f(x^+))}+\sum_{i=1}^{N} e^{s(f(x),f(x^-))}}]$ to maximize the score between $x$ and $x^+$ and minimize the score between $x$ and all $x_i^-$. Uses an **additional projector to map the encoder output to the similarity space** (possibly lower-dimensional) $f(x) = f_2 \circ f_1(x)$, **learn** $f_1$ (**encoder**) and $f_2$ (**projector**) **jointly** in an end-to-end fashion, use only $f_1$ for downstream tasks. Finally **use cosine similarity with temperature scaling** $\tau$: $s(e_1, e_2) = \frac{<e_1,e_2>}{||e_1||_2||e_2||_2\tau}$. **Generate views with data augmentation** (stronger than those used in supervised learning, like **random crop to create global to local views or adjacent views prediction tasks**)



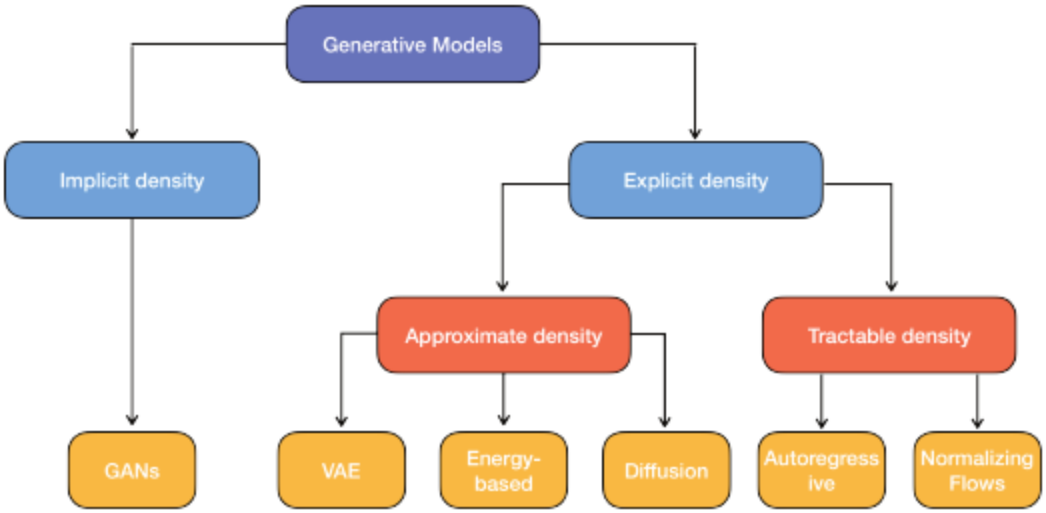


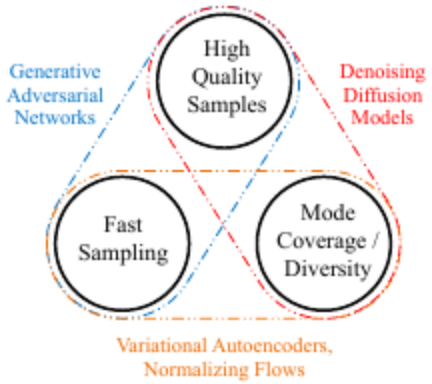

**CLIP**: use **captioned images to learn a joint multimodal embedding space**, the objective is to **maximize cosine similarity of caption and image embeddings while minimizing unrelated pairs**. Learns how to **match images to captions**, capable of **few-shot learning** (learning to classify new classes with only a few labelled examples (correct captions)) and **then zero-shot classify** an image by providing text input “A photo of [actual class in the image]” (it should say that the caption and the image match)



# 22 - Generative ML

**Generative Models**: given a data sample $x$ a **discriminative models aims at predicting its label** $y$ (it models the conditional distribution $p(y|x)$). **Generative models instead model the distribution** $p(x)$ **defined over the data points** $x$, they can be **explicit** (**modelling the probability** distribution of the data) or **implicit** (**generate samples** according to the probability distribution)




**Implicit density models**: **generate high-quality realistic samples** but often **challenging to train due to mode collapse** where the **model fails to capture the diversity** of the data distribution, e.g. **GANs**

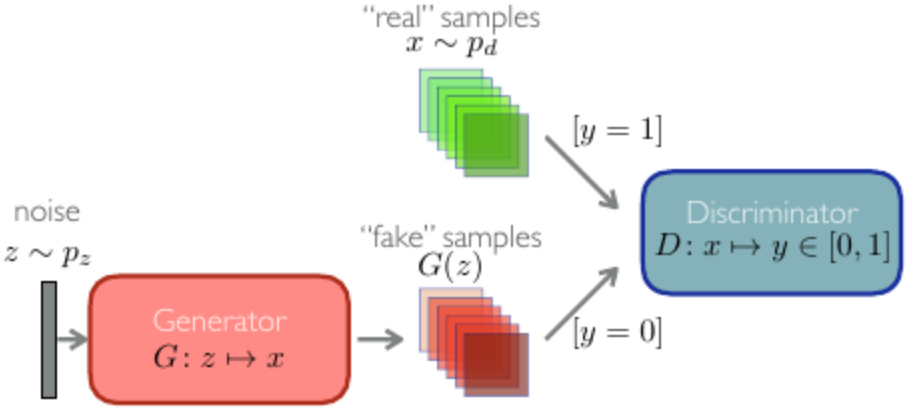
**Tractable density models**: allow for **exact likelihood computation**, e.g. **Autoregressive models** (**generate data sequentially**, can be **slow** in sampling) and **Normalizing Flows** (**more efficient** sampling and inference by **using invertible transformations**, can be **computationally intensive** to train)

**Approximate models**: includes **VAEs** (optimize a **lower bound on the likelihood**, **easier and more stable training than GANs**, but **less sharp** samples) and **Energy-based models**

**Diffusion models**: **hybrid** approach, they **iteratively convert noise into samples** via a reverse Markov process (used for **high-quality images and audio**), can be **slow**

**GANs**: **Generative Adversarial Networks**, family of **implicit generative algorithms** that are **fast to sample from**. Composed of **two models**: a **generator** $G$ with parameters $\theta$ (**learns to generate samples** from the data distribution $p_d$) and a **discriminator** $D$ with parameters $\varphi$ (learns to **distinguish between real and fake samples**). **Optimization is formulated as a differentiable two-player game** where the **generator** $G$**, and the discriminator** $D$**, aim at minimizing their own cost functions** $\mathcal{L}^\theta$ and $\mathcal{L}^\varphi$, $\theta^* = argmin_{\theta \in \Theta} \mathcal{L}^\theta(\theta, \varphi^*), \varphi^* \in argmin_{\varphi \in \Phi} \mathcal{L}^\varphi(\theta^*, \varphi)$. When $\mathcal{L}^\theta = -\mathcal{L}^\varphi$ the game is called a **zero-sum game** and finding the optimal $\theta^*, \varphi^*$ is a **minmax problem**. The **overall objective** is $min_G max_D \mathbb{E}_{x \sim p_d}[logD(x)] + \mathbb{E}_{z \sim p_z}[log(1 - D(G(z))]$, the loss for $D$ (distinguishing $x \sim p_g$ from $x \sim p_d$) is $argmin_D \mathcal{L}_D(G, D) = -[\mathbb{E}_{x \sim p_d}[logD(x)] + \mathbb{E}_{z \sim p_z}[log(1 - D(G(z))]]$, the loss for $G$ (fooling $D$ that $G(z) \sim p_d$) $argmin_G \mathcal{L}_G(G, D) = \mathbb{E}_{z \sim p_z}[log(1 - D(G(z))]$. Optimum is reached when $p_g = p_d$ and the optimal value is $-log4$

The discriminator "distinguishes" *real* vs. *fake* samples:




$p_z$ known "noise" distribution, *e.g.* $\mathcal{N}(0, 1)$

$p_d$ the real data distribution

$D$ mapping $D: x \mapsto y \in [0, 1]$,
where $y$ is an estimated probability that $x \sim p_d$

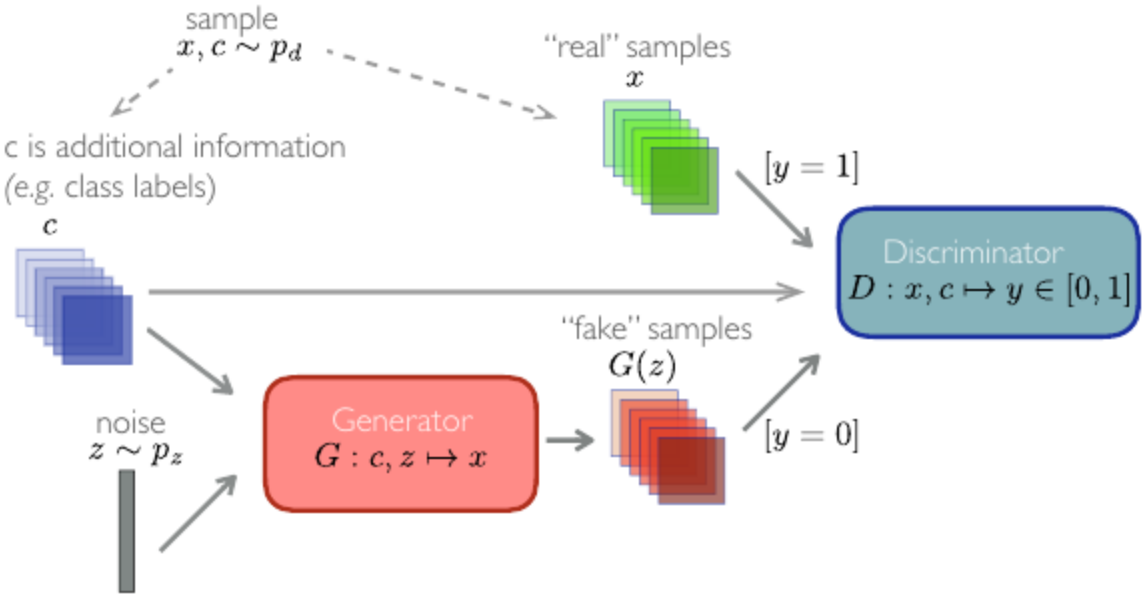
$G$ mapping $G: z \mapsto x$, such that if $z \sim p_z$,
then hopefully $x \sim p_d$

$p_g$ the "fake" data distribution

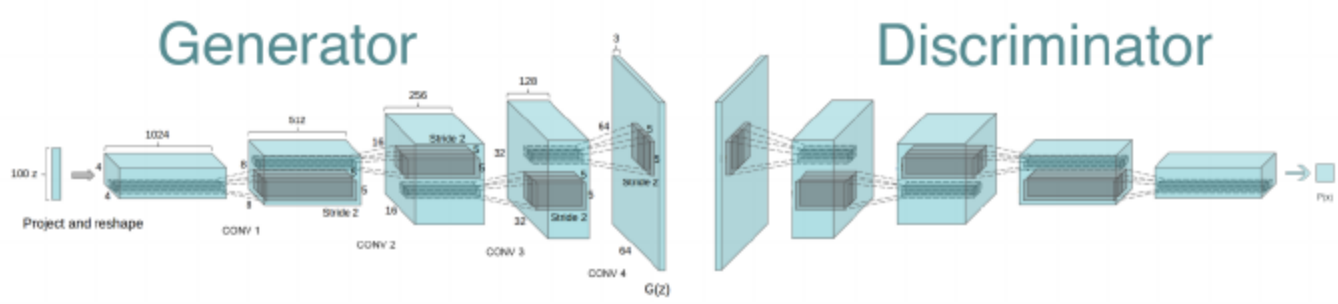
**Alternating-GAN**: most common **algorithm for training GANs**, input is dataset $\mathcal{D}$, known distribution $p_z$, learning rate $\eta$, generator loss $\mathcal{L}^\theta$, discriminator loss $\mathcal{L}^\varphi$, initialize $D(\cdot; \varphi), G(\cdot; \theta)$, for $t = 1 \rightarrow T$ do $x \sim p_x$ (sample real data), $z \sim p_z$ (sample noise), $\varphi = \varphi - \eta \nabla_\varphi \mathcal{L}^\varphi(\theta, \varphi, x, z)$ (update $D$),

$z \sim p_z$ (sample noise), $\theta = \theta - \eta \nabla_\theta \mathcal{L}^\theta(\theta, \varphi, z)$ (update $G$), final output is $\theta, \varphi$. Often optimized with **Adam**

**Conditional GAN (CGAN)**: to generate **samples of a conditional probability distribution** (next frame in a video, or image from text prompt), both the **generator and the discriminator are conditioned during training by some additional information** (typically one-hot vector encoded class labels)




**Deep Convolutional GAN (DCGAN)**: GAN architecture **specific for images**, uses **transposed convolutional layers** (fractionally strided convolutions) called **Deconvolution layers that perform image upsampling**




**Diffusion models**: **progressively add noise to input data and train a model to estimate the added noise and recover the data, generate a new sample by trying to recover an ipothetical image from noise**. Consider a **Markov chain** with $X_0 \sim p_0$ and a **forward transition** such that $X_{t+1} \sim p(\cdot|X_t)$ then we call **forward decomposition** $p(x_{0:T}) = p_0(x_0) \prod_{t=0}^{T-1} p(x_{t+1}|x_t)$ where at each step, the marginal distribution of $X_t$ satisfies $p(x_t) = \int p(x_t|x_{t-1})p(x_{t-1})dx_{t-1}$. The **backward transition** can be obtained with Bayes' rule $p(x_t|x_{t+1}) = \frac{p(x_{t+1}|x_t)p(x_t)}{p(x_{t+1})}$ and we call **backward decomposition** $p(x_{0:T}) = p_T(x_T) \prod_{t=0}^{T-1} p(x_t|x_{t+1})$. A **generative model can be constructed by sampling from $p(x_{0:T})$ using ancestral sampling**

**Ancestral sampling for Score-based Diffusion Models**: starts with a random sample $X_T \sim p_T(\cdot)$ ( $p_T$ is the probability distribution at time $T$, all noise), for $k = T-1, \dots, 0$ sample $X_t \sim p(\cdot|X_{t+1})$. We consider $p_0 = p_{data} \in \mathcal{P}(\mathbb{R}^d)$ (real-world data), we chose the **forward transition to be Gaussian** $p(x_{t+1}|x_t) = \mathcal{N}(x_{t+1}; \alpha x_t, (1-\alpha^2)\mathbb{I}_d)$, the conditional $p(x_t|x_0)$ is also Gaussian $\mathcal{N}(x_t; \alpha^t x_0, (1-\alpha^{2t})\mathbb{I}_d)$ (since we know the forward transition is Gaussian we can skips steps if we know the original image and **calculate the noise directly at any $t$ we want**), the Markov chain converges to $p_{ref} = \mathcal{N}(x; 0, \mathbb{I}_d)$ for $T \to \infty$, as you keep on **adding noise eventually you only have noise and converge to a standard Gaussian**, so **for a large enough $T$ we can assume the final step is just standard random noise**, so we can **generate samples by replacing $p_T$ with $p_{ref}$**. ****This works **perfectly for destroying the image**, but **generating** one has the problem that the **backward transition $p(x_t|x_{t+1})$ can't be computed**, we **approximate using a Taylor expansion** of $log\ p(x_t)$ around $x_{t+1}$: $p(x_t|x_{t+1}) = p(x_{t+1}|x_t)exp[log\ p(x_t) - log\ p(x_{t+1})] \approx \mathcal{N}(x_t; (2-\alpha)x_{t+1} + (1-\alpha^2)\nabla log\ p(x_{t+1}), (1-\alpha^2)\mathbb{I}_d)$ where $\nabla log\ p(x_{t+1})$ is the **score**. The **score term is intractable** (because to compute it we would need to compute an integral using $p(x_0), \dots, p(x_t)$ that we don't have going backward), we can **approximate it using a neural network** $s_\theta(x_t)$ that solves a **simple regression problem** (**Denoising Score Matching**, **DSM**, the **only model we train**) with loss $\theta^* = argmin_\theta \mathcal{L}(\theta) = \frac{1}{2}\mathbb{E}_{X_t}[||\nabla log\ p(X_t) - s_\theta(X_t)||^2]$ since we also cannot compute $p(x_t)$ for the same reason as before the problem is still intractable, so $\mathcal{L}(\theta) = C_1 \frac{1}{2}\mathbb{E}_{X_t}[||s_\theta(X_t)||^2] - \mathbb{E}_{X_t}[\nabla log\ p(X_t)^T s_\theta(X_t)]$, using that $p(X_t) = \int p_0(X_0)p(X_t|X_0)dX_0$ we get $\nabla log\ p(X_t) = \mathbb{E}_{X_0|X_t}[\nabla log\ p(X_t|X_0)]$, finally $\mathcal{L}(\theta) = C_2 \frac{1}{2}\mathbb{E}_{X_0}\mathbb{E}_{X_t|X_0}[||s_\theta(X_t) - \nabla log\ p(X_t|X_0)||^2]$. We **estimate all the scores $s_{\theta^*}(t, X_t)$ simultaneously such that** $\theta^* = argmin_\theta \frac{1}{2}\sum_{t=1}^T \mathbb{E}_{X_0}\mathbb{E}_{X_t|X_0}[||s_\theta(X_t) - \nabla log\ p(X_t|X_0)||^2]$. Recalling our initial choices for the forward side of things we have $\nabla log\ p(x_t|x_0) = -\frac{x_t - \alpha^t x_0}{1-\alpha^{2t}}$ (property of taking the gradient of the log-probability of a Gaussian), now recall how we theoretically initially computed $X_t = \alpha^t X_0 + \sqrt{1\alpha^{2t}}\xi_t,\ \xi_t \sim \mathcal{N}(0, \mathbb{I}_d)$ and substitute it in the previous formula to get $\nabla log\ p(X_t|X_0) = -\frac{\xi_t}{\sqrt{1-\alpha^{2t}}}$ (not dependant of $x_0$). We make the model learn and predict $\hat{\xi}_\theta(t, X_t)$ instead of $s_\theta(t, X_t)$ because it's more meaningful and numerically stable (it's exactly how much noise $X_t$ has), so we have $s_\theta(t, X_t) = -\frac{\hat{\xi}_\theta(t, X_t)}{\sqrt{1-\alpha^{2t}}}$. The **DSM problem becomes** $\theta^* = argmin_\theta \frac{1}{2}\sum_{t=1}^T \mathbb{E}_{X_0}\mathbb{E}_{X_t|X_0}[||\hat{\xi}_\theta(t, X_t) - \xi_t||^2]$ which allows us to **predict the added noise $\xi_t$ from the noised data $X_t$**. We can now **perform ancestral sampling** starting from $X_T \sim p_{ref}$ and at each iteration setting $X_t = (2-\alpha)X_{t+1} + (1-\alpha^2)s_{\theta^*}(t+1, X_{t+1}) + \sqrt{1-\alpha^2}\xi_{t+1},\ \xi_{t+1} \sim \mathcal{N}(0, \mathbb{I}_d)$

**Algorithm 2** Training

**Input:** Data distribution $p_0$, $\alpha$
**Output:** Learned parameters $\theta^*$
**Initialize:** Parameters $\theta$
**repeat**
  Sample initial data $X_0 \sim p_0$
  Sample time step $t \sim \text{Uniform}(\{1, \dots, T\})$
  Sample noise $\xi_t \sim \mathcal{N}(0, \mathbb{I}_d)$
  Update $\theta$ using gradient descent step on:
  $\nabla_\theta \frac{1}{2} \mathbb{E}_{X_0} \mathbb{E}_{X_t|X_0} \left[ \left\| s_\theta(t, \alpha^t X_0 + \sqrt{1-\alpha^{2t}}\xi_t) - \xi_t \right\|^2 \right]$
**until** convergence

**Algorithm 3** Sampling

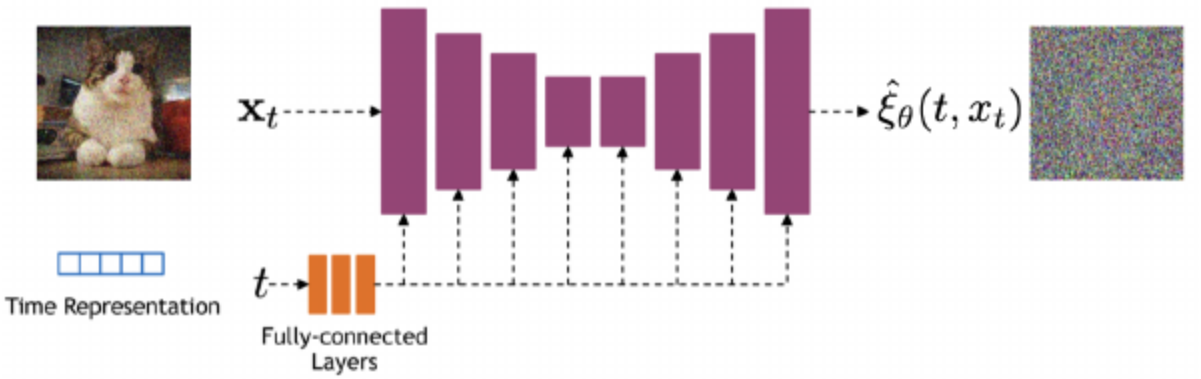
**Input:** Learned parameters $\theta^*$, $p_{\text{ref}}$, $\alpha$
**Output:** Generated sample $X_0$
Sample $X_T \sim p_{\text{ref}}$
**for** $t = T-1$ **to** $0$ **do**
  Sample noise $\xi_{t+1} \sim \mathcal{N}(0, \mathbb{I}_d)$:

$$X_t = (2-\alpha)X_{t+1} + (1-\alpha^2)s_{\theta^*}(t+1, X_{t+1}) + \sqrt{1-\alpha^2}\xi_{t+1}$$

**end for**

Return $x_0$

**U-net architecture**: implementing **diffusion models with ancestral sampling**, ****often **used with ResNet blocks and self attention layers to represent the score function** $\hat{\xi}_\theta(t, X_t)$. The **time step** can be **represented with sinusoidal positional encodings** (like transformers) that are **fed to the residual blocks** using either spatial addition or adaptive group normalization layers. Since **training of U-net via Denoising Score Matching** can be very **unstable**, an **exponential moving average is used** $\bar{\theta}_{n+1} = (1-\beta)\bar{\theta}_n + \beta\theta_n$




**Conditional Diffusion Models**: done by **adding an additional input to the score function such that a conditional score is learned** $\theta^* = argmin_\theta \frac{1}{2} \sum_{t=1}^{T} \mathbb{E}_{X_0,Y} \mathbb{E}_{X_t|X_0} [||s_\theta(t, X_t^Y; Y) - \nabla log\, p(X_t^Y|X_0^Y)||^2]$, the conditioning is fed into the network in a similar way as the time features